INFEARA MEDICAL SYSTEM (IMS): MULTI-TENANT CLINICAL MANAGEMENT, E-PRESCRIPTION AND PHARMACY POS PLATFORM

A PROJECT REPORT SUBMITTED TO

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

IN PARTIAL FULFILLMENT OF THE REQUIREMENTS FOR THE AWARD OF THE DEGREE OF

BACHELOR OF COMPUTER APPLICATIONS

BY

VIGNESH M

REG. NO. DC2431006010004

UNDER THE GUIDANCE OF

Dr. A. THIRUMURTHI RAJA M.C.A., M.E., PH.D.,
ASSISTANT PROFESSOR, SRMIST-DDE,



DEPARTMENT OF COMPUTER APPLICATIONS
DIRECTORATE OF DISTANCE EDUCATION
SRM INSTITUTE OF SCIENCE AND TECHNOLOGY
(Deemed to be university u/s 3 of UGC Act, 1956)
Kattankulathur – 603 203
Chennai, Tamilnadu
NOV 2026

BONAFIDE CERTIFICATE

This is to certify that the project report titled **"INFEARA MEDICAL SYSTEM (IMS): MULTI-TENANT CLINICAL MANAGEMENT, E-PRESCRIPTION AND PHARMACY POS PLATFORM"** is a bonafide work carried out by **VIGNESH M (DC2431006010004)** under my supervision for the award of the Degree of **Bachelor Of Computer Applications**. To my knowledge the work reported herein is the original work done by the candidate.

PROJECT GUIDE

Dr. A. Thirumurthi Raja
Assistant Professor,
SRMIST-DDE,

HEAD OF DEPARTMENT

Dr. A. Thirumurthi Raja
Program Coordinator,
Department of Computer Applications,
Directorate of Distance Education (DDE),
SRMIST,

INTERNAL EXAMINER

EXTERNAL EXAMINER

DECLARATION

I, **VIGNESH M (DC2431006010004)**, hereby declare that the project report entitled **"INFEARA MEDICAL SYSTEM (IMS): MULTI-TENANT CLINICAL MANAGEMENT, E-PRESCRIPTION AND PHARMACY POS PLATFORM"** submitted to **SRM Institute of Science and Technology** in partial fulfillment of the requirements for the award of the Degree of **Bachelor Of Computer Applications** is a record of original software engineering and academic project work carried out by me under the supervision and guidance of **Dr. A. Thirumurthi Raja, M.C.A., M.E., Ph.D.**, Assistant Professor, Department of Computer Applications, Directorate of Distance Education, SRMIST. I further declare that this project report or any part thereof has not been submitted earlier to any other University or Institution for the award of any degree, diploma, or fellowship.

VIGNESH M
(DC2431006010004)
Signature of the Candidate

Place: Chennai
Date: November 2026

ACKNOWLEDGEMENT

With profound gratitude to the ALMIGHTY, I take this chance to thank the people who helped me to complete this project.

I take this as a right opportunity to say THANKS to my parents who are there to stand with me always with the words “YOU CAN”.

I am thankful to **Dr. T. R. Paarivendhar, Chancellor**, SRM Institute of Science & Technology who gave us the platform to establish ourselves and reach greater heights.

I express my deep sense of gratitude to the Director **Dr. Manoranjan Pon Ram** for his wholehearted support and encouragement.

I earnestly thank **Dr. A. Thirumurthi Raja, Programme Coordinator**, BCA/MCA — Directorate of Distance Education, SRM Institute of Science & Technology who always encourages us to pursue novel engineering solutions.

It is my delight to thank my project guide **Dr. A. Thirumurthi Raja**, Assistant Professor, Department of Computer Applications (DDE), for his invaluable help, technical insights, encouragement, constructive suggestions, and continuous guidance throughout the analysis, design, and implementation phases of this project.

I convey my sincere gratitude to all the faculty members of the Department of Computer Applications who extended their support through valuable comments and suggestions during the project reviews.

A great note of gratitude to my friends and colleagues who helped in carrying out this project work to a successful completion.

VIGNESH M

ABSTRACT

Outpatient healthcare delivery across urban and semi-urban clinics frequently suffers from fragmented operational software, disconnected patient records, handwritten or easily forged prescriptions, and manual pharmacy billing ledgers. When patients consult doctors across multiple independent clinics, their demographic profiles, allergy disclosures, consultation histories, and active medications become trapped inside isolated silos or paper folders. Simultaneously, community pharmacies struggle to verify whether a presented prescription is authentic, unexpired, uncancelled, or already dispensed elsewhere, creating clinical medication-safety risks as well as manual accounting discrepancies between point-of-sale dispensing and daily financial cashbooks.

To resolve these end-to-end operational, cryptographic, and clinical challenges, this project presents the design, implementation, and validation of the Infeara Medical System (IMS) — a secure, multi-tenant healthcare enterprise resource planning (ERP) web platform engineered as a modular monolith on Java 27 and Spring Boot 4.1.2, backed by Microsoft SQL Server 2022 Express (with Flyway versioned schema migrations V001 through V009) and a zero-setup H2 compatibility profile. IMS unifies six distinct security principals across five role-tailored workspaces: Super Admin (/admin/**), Clinic Administration (/clinic/**), Doctor Workspace (/doctor/**), Patient Self-Service Portal (/patient/**), and Pharmacy Point-of-Sale & Finance Workspace (/pharmacy/**), governed by Spring Security 6 session authentication, BCrypt delegating password hashing, brute-force login throttling, and real-time tenant suspension filters.

The platform introduces four key architectural innovations grounded directly in clinical workflow safety. First, a staff-verified Patient Identity and Non-Destructive Record Merge engine enables patients to link a single verified account across multiple enrolled clinics while allowing clinic administrators to merge prior walk-in charts onto the patient's account without breaking foreign-key audit trails. Second, an atomic Appointment Slot Scheduling engine combines pessimistic doctor row locking with a filtered SQL Server unique index (UX_Appt_Slot) to guarantee zero double-bookings under concurrent requests. Third, an immutable Clinical Consultation and Electronic Prescription engine transitions clinical notes from DRAFT to FINAL (permitting only signed append-only ConsultationAddenda thereafter) and issues case-sensitive 32-character CSPRNG prescription codes enforced via SQL Server Latin1_General_100_BIN2 binary collation. Fourth, an integrated Pharmacy Point-of-Sale (POS) and Finance Ledger module verifies 32-character codes in real time, prevents over-dispensing across partial fills, remembers per-medicine unit prices automatically, computes discounts and taxes, generates itemized receipts, and auto-posts completed sale totals as linked INCOME entries in the pharmacy

financial ledger.

Complemented by a transactional database-backed Email Outbox worker with exponential retry backoff, AES-256-GCM encryption for stored SMTP credentials, and comprehensive JUnit 5, Mockito, and Spring MockMvc security/integration test suites, Infeara Medical System demonstrates a production-grade, auditable, and cost-effective digital health backbone suitable for multi-clinic networks and retail pharmacies.

Keywords: Multi-Tenant Healthcare ERP, Spring Boot 4, Electronic Prescription (e-Rx), Non-Destructive Patient Record Merge, Role-Based Access Control (RBAC), Pharmacy Point of Sale (POS), Transactional Email Outbox, AES-256-GCM, SQL Server 2022.

TABLE OF CONTENTS

Title Page i

Bonafide Certificate ii

Declaration iii

Acknowledgement iv

Abstract v

Table of Contents vii

List of Figures xi

List of Tables xiii

Abbreviations / Acronyms xv

CHAPTER 1: INTRODUCTION 1

1.1 Overview of the Project 1

1.2 Background of the Study 2

1.3 Problem Statement 3

1.4 Objectives of the Project 4

1.5 Scope of the Project 5

1.6 Project Motivation 6

1.7 Assumptions and Constraints 6

1.8 Organization of the Report 7

CHAPTER 2: LITERATURE REVIEW 9

2.1 Introduction 9

2.2 Review of Existing Systems and Related Literature 9

2.2.1 Literature Survey 1 9

2.2.2 Literature Survey 2 10

2.2.3 Literature Survey 3 10

2.2.4 Literature Survey 4 10

2.2.5 Literature Survey 5 11

2.2.6 Literature Survey 6 11

2.2.7 Literature Survey 7 12

2.2.8 Literature Survey 8 12

2.2.9 Literature Survey 9 12

2.2.10 Literature Survey 10 13

2.2.11 Literature Survey 11 13

2.2.12 Literature Survey 12 14
2.3 Comparative Study of Related Work 14
2.4 Research Gaps Identified 15
2.5 Summary of Literature Review 16
CHAPTER 3: SYSTEM ANALYSIS 17
3.1 Existing System Analysis 17
3.2 Limitations of Existing System 17
3.3 Proposed System 18
3.4 Advantages of Proposed System 19
3.5 Software Requirements Specification (SRS) 20
3.6 Feasibility Study 22
3.6.1 Technical Feasibility 22
3.6.2 Economic Feasibility 23
3.6.3 Operational Feasibility 23
CHAPTER 4: SYSTEM DESIGN 25
4.1 Design Overview and Architectural Principles 25
4.2 System Architecture 25
4.3 Data Flow Diagrams (DFD) 27
4.3.1 Level 0 DFD (System Context Diagram) 27
4.3.2 Level 1 DFD (Subsystem Decomposition) 28
4.3.3 Level 2 DFD (Pharmacy POS Verification, Pricing & Ledger Automation) 29
4.4 Use Case Diagram and Actor Specifications 29
4.5 Object-Oriented & Behavioral UML Models (Class, Sequence, Activity & State) 34
4.6 Entity Relationship (ER) Diagram 37
4.7 Complete Relational Database Design and Data Dictionary 38
4.8 Input Design, Output Design and Corporate ERP UI Design System 49
CHAPTER 5: TECHNOLOGIES & TOOLS USED 53
5.1 Software Requirements 53
5.2 Hardware Requirements 54
5.3 Programming Languages Used 55
5.3.1 Java 27 (Backend Domain, Security, and Web Layer) 55
5.3.2 Transact-SQL (T-SQL for Microsoft SQL Server 2022) 55
5.3.3 HTML5, CSS3 Custom Properties, SVG, and Vanilla ES6 JavaScript 55
5.4 Frameworks and Libraries 56

5.5 Development and Runtime Environment 56
CHAPTER 6: IMPLEMENTATION 58
6.1 Module Description 58
6.1.1 Module 1: Zero-Seed Platform Bootstrap (/setup) & Super Admin Console (/admin/**) 58
6.1.2 Module 2: Identity, Single-Use SHA-256 Auth Tokens, Invitations & Login Throttling 58
6.1.3 Module 3: Patient Self-Service Portal, Clinic Discovery & Atomic Slot Booking 58
6.1.4 Module 4: Staff-Verified Patient Account Linking & Non-Destructive Record Merge 59
6.1.5 Module 5: Clinic Administration, Doctor Schedules & CSV Medicine Formulary 59
6.1.6 Module 6: Doctor Clinical Workspace — Consultations, Addenda & 32-Char e-Prescriptions 59
6.1.7 Module 7: Pharmacy Workspace — Verification, Auto-Pricing POS & Finance Ledger 60
6.1.8 Module 8: Transactional Email Outbox Worker & Encrypted SMTP Engine 60
6.2 Core Algorithms and Cryptographic Mechanisms 60
6.3 Pseudocode and Logic Flows 61
6.4 Key Source Code Implementations 63
6.5 Screenshots of Live Application Across Operational Workspaces 68
6.6 Integration of Modules and Cross-Cutting Event Flow 76
CHAPTER 7: TESTING & RESULTS 77
7.1 Testing Strategy 77
7.2 Types of Testing 77
7.2.1 Unit Testing (JUnit Jupiter 5, AssertJ & Mockito) 77
7.2.2 Integration and Web Security Testing (Spring Boot Test & MockMvc) 77
7.2.3 System, Concurrency, and End-to-End Browser Validation 78
7.3 Test Cases and Results 78
7.4 Performance and Security Analysis 82
CHAPTER 8: CONCLUSION & FUTURE SCOPE 84
8.1 Summary of Work 84
8.2 Conclusion 84
8.3 Limitations 84
8.4 Future Enhancements 85
REFERENCES 86

1. Books 86
2. Research Papers and Standards 86
3. Official Documentation and Websites 87
APPENDICES **89**
Appendix A Sample Source Code and Database Migration Scripts 89
Appendix B Complete WebMVC Endpoint & RBAC Matrix and Operational Deployment Guide 94

LIST OF FIGURES

Figure 4.1: Infeara Medical System (IMS) Six-Tier Modular Monolith Architecture 26

Figure 4.2: IMS Defense-in-Depth Security Filter Chain and Cryptographic Controls 27

Figure 4.3: Data Flow Diagram — Level 0 (System Context Diagram) 28

Figure 4.4: Data Flow Diagram — Level 1 (IMS Functional Decomposition) 29

Figure 4.5: Data Flow Diagram — Level 2 (Pharmacy POS Dispensing and Automated Finance Sync) 29

Figure 4.6: IMS Unified UML Use Case Diagram Across Five Actor Portals 30

Figure 4.7: IMS Core Domain Entity UML Class Diagram (Spring Data JPA / Hibernate 6) 35

Figure 4.8: UML Sequence Diagram — End-to-End Clinical Consultation, e-Rx & Pharmacy POS Flow 36

Figure 4.9: UML Activity Workflow Diagram — Patient Linking, Clinical Care & Dispensing 37

Figure 4.10: UML Lifecycle State Machine Diagram — AppointmentStatus & Prescription Fulfilment 37

Figure 4.11: IMS Relational Database Entity-Relationship (ER) Schema (27 Tables, V001–V009) 38

Figure 6.1: IMS Unified Sign-In & Session Lifecycle Gateway (/login) 69

Figure 6.2: Super Admin — Platform Governance Dashboard (/admin/dashboard) 70

Figure 6.3: Super Admin — Multi-Tenant Clinic Enrollment Console (/admin/clinics) 70

Figure 6.4: Super Admin — Dispensing Pharmacy Enrollment & Licence Registry (/admin/pharmacies) 71

Figure 6.5: Super Admin — Privacy-Masked Cross-Clinic Patient Masters (/admin/patients) 71

Figure 6.6: Super Admin — Live Platform Reports & Outbox Delivery Telemetry (/admin/reports) 72

Figure 6.7: Super Admin — AES-256-GCM Encrypted SMTP Configuration & Mail Audit (/admin/smtp) 72

Figure 6.8: Doctor Workspace — Practitioner Clinical Dashboard (/doctor/dashboard) 73

Figure 6.9: Doctor Workspace — Date & Status Filtered Appointment Queue (/doctor/appointments) 73

Figure 6.10: Doctor Workspace — Completed Encounters & Finalised Consultations (/doctor/appointments?all) 74

Figure 6.11: Doctor Workspace — Issued 32-Character Prescriptions & Fulfilment States (/doctor/prescriptions) 74

Figure 6.12: Pharmacy Workspace — Dispensing Operations Dashboard (/pharmacy/dashboard) 75
Figure 6.13: Pharmacy Workspace — Live 32-Character Prescription Point of Sale (/pharmacy/pos) 75
Figure 6.14: Pharmacy Workspace — Automated POS Income & Expense Finance Ledger (/pharmacy/finance) 76

LIST OF TABLES

Table 1.1: High-Level Identity and Architectural Profile of Infeara Medical System (IMS) 1

Table 1.2: Functional Scope Matrix Across IMS Portals and Security Roles 5

Table 2.1: Comparative Analysis of Existing Healthcare & Pharmacy Solutions vs. Proposed IMS 14

Table 3.1: Structural Comparison Between Legacy Outpatient Workflows and Proposed IMS 18

Table 3.2: Functional Requirements Specification Matrix (FR-01 to FR-18) 20

Table 3.3: Non-Functional Requirements Specification Matrix (NFR-01 to NFR-10) 22

Table 3.4: Estimated Annual Cost-Benefit & Economic Feasibility Summary (Small 3-Clinic Network) 23

Table 4.1: Detailed Actor-to-Use-Case Responsibility Specification 30

Table 4.1a: Use Case Specification — UC-05: Staff-Verified Patient Link & Non-Destructive Record Merge 31

Table 4.1b: Use Case Specification — UC-08: Atomic Conflict-Free Doctor Slot Booking 32

Table 4.1c: Use Case Specification — UC-09 & UC-10: Immutable Clinical Consultation & 32-Char e-Rx 33

Table 4.1d: Use Case Specification — UC-11, UC-12 & UC-13: Pharmacy POS Billing & Auto-Ledger Sync 33

Table 4.2: Flyway Database Migration History (V001 through V009) 39

Table 4.3: Data Dictionary — dbo.Clinics (Multi-Tenant Healthcare Facilities) 39

Table 4.4: Data Dictionary — dbo.Users (Unified Security Principals) 40

Table 4.5: Data Dictionary — dbo.Patients (Master & Self-Registered Patient Profiles) 41

Table 4.6: Data Dictionary — dbo.Doctors, dbo.ClinicDoctors, dbo.DoctorSchedules & dbo.DoctorUserLinks 42

Table 4.7: Data Dictionary — dbo.Appointments (Conflict-Free Slot Reservations) 43

Table 4.8: Data Dictionary — dbo.Consultations & dbo.ConsultationAddenda (Clinical Records) 44

Table 4.9: Data Dictionary — dbo.Prescriptions, dbo.PrescriptionItems & dbo.ClinicMedicines 45

Table 4.10: Data Dictionary — dbo.Pharmacies, dbo.PharmacyUsers, dbo.PharmacyVerifications & dbo.PharmacyMedicinePrices 46

Table 4.11: Data Dictionary — dbo.PrescriptionDispenses, dbo.PrescriptionDispenseItems & dbo.PharmacyFinanceEntries 47

Table 4.12: Data Dictionary — Enrollment, Patient Linking, AuthTokens, Outbox, SMTP & Audit Tables 48
Table 4.13: Complete Java Domain Enumeration Reference (com.clinicflow.domain.enums.*) 50
Table 4.14: Corporate ERP User Interface Design System Tokens (static/css/app.css) 52
Table 5.1: Software Requirements Specification for Infeara Medical System (IMS) 53
Table 5.2: Minimum and Recommended Hardware Specifications 54
Table 6.1: Complete Thymeleaf View & Fragment Directory Inventory (61 HTML5 Templates) 68
Table 7.1: Automated Unit & Service-Layer Test Cases (JUnit 5 + Mockito) 78
Table 7.2: Web Security, Concurrency & End-to-End System Test Cases (MockMvc & Browser) 80
Table 7.3: Measured Endpoint Latency and OWASP Top-10 Security Verification Summary 82
Table B.1: Complete IMS Spring WebMVC Route and Role-Based Authorization Matrix 95
Table B.2: Externalized Environment Configuration Variables (application.yml) 98

ABBREVIATIONS AND ACRONYMS

Acronym	Expanded Technical / Clinical Term
ACID	Atomicity, Consistency, Isolation, and Durability
AES-GCM	Advanced Encryption Standard — Galois/Counter Mode (Authenticated Encryption)
API	Application Programming Interface
BCrypt	Blowfish-Based Adaptive Password Hashing Function
BIN2	SQL Server Binary Code-Point Collation (Latin1_General_100_BIN2)
CSRF	Cross-Site Request Forgery
CSPRNG	Cryptographically Secure Pseudo-Random Number Generator (java.security.SecureRandom)
CSV	Comma-Separated Values (Clinic Formulary Import/Export)
DAO / JPA	Data Access Object / Jakarta Persistence API
DDL / DML	Data Definition Language / Data Manipulation Language
DFD	Data Flow Diagram
DOB	Date of Birth
DTO	Data Transfer Object
EHR / EMR	Electronic Health Record / Electronic Medical Record
ERP	Enterprise Resource Planning
e-Rx	Electronic Prescription
FK / PK	Foreign Key / Primary Key
GCM	Galois/Counter Mode (128-bit Authentication Tag, 96-bit IV)
HTML5 / CSS3	HyperText Markup Language v5 / Cascading Style Sheets v3
HTTP / HTTPS	HyperText Transfer Protocol / HTTP Secure (TLS)
IDOR	Insecure Direct Object Reference
IMS	Infeara Medical System (Proposed Application Platform)
IV	Initialization Vector
JDBC	Java Database Connectivity
JVM / JDK	Java Virtual Machine / Java Development Kit (Java 27)
KPI	Key Performance Indicator
MVC	Model-View-Controller Architectural Pattern

Acronym	Expanded Technical / Clinical Term
ORM	Object-Relational Mapping (Hibernate 6)
OWASP	Open Worldwide Application Security Project
POS	Point of Sale (Pharmacy Billing & Dispensing Counter)
RBAC	Role-Based Access Control
RDBMS	Relational Database Management System (Microsoft SQL Server 2022)
SHA-256	Secure Hash Algorithm 256-bit Digest (BINARY(32))
SMTP / STARTTLS	Simple Mail Transfer Protocol / Opportunistic TLS Upgrade
SRS	Software Requirements Specification
TTL	Time-To-Live (Token / Prescription Expiration Window)
UAT	User Acceptance Testing
UML	Unified Modeling Language
UPI	Unified Payments Interface
UTC	Coordinated Universal Time (DATETIME2(3) Timestamp Standard)

CHAPTER 1
INTRODUCTION

1.1 Overview of the Project

The Infeara Medical System (IMS), developed under the core package namespace com.clinicflow inside the clinicflow-app Maven module, is an integrated, multi-tenant clinical management, patient identity federation, electronic prescribing (e-Rx), and pharmacy point-of-sale (POS) web application. Unlike conventional single-clinic desktop utilities or isolated pharmacy billing tools, IMS models the complete continuum of outpatient primary and specialty care within a single transactional software platform. From the moment a platform owner initializes the deployment via the zero-seed /setup bootstrap wizard, through clinic and pharmacy onboarding, staff invitation workflows, patient self-registration, verified account linking, conflict-free doctor appointment booking, clinical consultation authoring, 32-character digital prescription issuance, itemized pharmacy verification and partial dispensing, to automated financial ledger posting, every state transition is governed by strict role-based access controls, relational integrity constraints, and an immutable compliance audit log.

Architecturally, IMS follows the Modular Monolith pattern on Java 27 and Spring Boot 4.1.2. Rather than incurring the operational overhead and eventual-consistency hazards of premature microservices, IMS partitions domain responsibilities cleanly across dedicated packages — audit, common, config, domain (entities, enums, repositories), identity, notifications, security, and role-segregated web controllers — while executing multi-entity clinical operations inside ACID database transactions against Microsoft SQL Server 2022 Express. Server-rendered user interfaces are built with Thymeleaf 3.1 and the Ultraq Layout Dialect over a cohesive Corporate Healthcare ERP design system (app.css), ensuring fast first-contentful paint, accessibility compliance, keyboard-friendly POS operation, and strict server-side ownership enforcement.

Table 1.1: High-Level Identity and Architectural Profile of Infeara Medical System (IMS)

Attribute	Specification in Current Codebase
Application Name	IMS — Infeara Medical System (Artifact: com.clinicflow:clinicflow-app:0.1.0-SNAPSHOT)
Architectural Style	Six-Tier Modular Monolith with Transactional Database Outbox
Core Runtime & Framework	Java 27, Spring Boot 4.1.2, Spring WebMVC, Spring Security 6, Spring Data JPA
Database & Schema Versioning	Microsoft SQL Server 2022 Express (Primary) & H2 MSSQLServer Mode; Flyway V001–V009
Normalized Schema Footprint	27 Relational Tables, 1 Fulfilment SQL View (vw_PrescriptionFulfilment), 20 Domain Enums
Supported Security Roles	SUPER_ADMIN, CLINIC_ADMIN, DOCTOR, PATIENT, PHARMACY_ADMIN, PHARMACIST
Presentation Engine	Thymeleaf + Layout Dialect + Spring Security6 Extras (61 HTML5 Templates + ERP CSS)
Key Cryptographic Controls	BCrypt Passwords, SHA-256 Single-Use Tokens, AES-256-GCM SecretCipher, 32-Char CSPRNG Rx

1.2 Background of the Study

Outpatient clinics and neighborhood retail pharmacies form the frontline of healthcare delivery in India and across developing healthcare ecosystems, accounting for more than eighty percent of daily patient encounters. Despite rapid growth in consumer digital payments and mobile connectivity, day-to-day clinical workflows across small and mid-sized polyclinics remain heavily fragmented. A typical patient visiting two different clinics in the same city is assigned separate walk-in registration numbers at each reception desk, receives handwritten paper prescriptions with non-standardized drug names, and presents those slips at retail pharmacy counters where pharmacists manually transcribe medicine names and re-enter bill totals into a separate cash register or spreadsheet.

This fragmentation produces severe operational and clinical consequences. First, doctors frequently make prescribing decisions without visibility into longitudinal diagnoses, chronic lifetime medications, or recorded drug allergies captured during earlier visits. Second, paper prescriptions lack cryptographic verifiability: once a paper slip leaves the clinic, neither the issuing clinic nor the dispensing pharmacy can reliably prevent a controlled or antibiotic prescription from being photocopied, altered, used past its clinical validity window, or dispensed repeatedly across different pharmacies. Third, clinic front desks managing doctor schedules via phone calls or basic spreadsheets regularly encounter double-booked time slots whenever simultaneous walk-in and online requests

target the same fifteen-minute window.

Furthermore, existing commercial cloud EHR suites often fail small and medium healthcare networks for two opposite reasons: either they are heavyweight hospital information systems requiring expensive dedicated IT teams and complex HL7 middleware, or they are consumer doctor-discovery marketplaces that prioritize lead generation while neglecting clinical immutability, verified patient record merging, and pharmacy POS financial reconciliation. Studying these real-world gaps directly motivated the engineering of IMS as a self-contained, cryptographically hardened, multi-tenant clinical and pharmacy platform.

1.3 Problem Statement

A rigorous analysis of outpatient clinical administration, patient record continuity, prescription dispensing, and pharmacy accounting reveals eight interconnected technical problems that conventional systems fail to address cohesively:

- **1. Siloed & Duplicated Patient Identities:** Walk-in clinic registrations and online self-registrations create duplicate patient rows. Naive systems either forbid linking altogether or auto-link records by email or name alone, exposing sensitive medical histories to account-takeover and identity-spoofing attacks.
- **2. Destructive Record Merging & Broken Audit Trails:** When duplicate patient charts are consolidated in legacy databases by deleting the older row, historical audit logs and foreign keys referencing the old identifier either fail referential integrity checks or lose forensic traceability.
- **3. Concurrent Appointment Double-Booking:** Checking slot availability in application memory before inserting an appointment row introduces a classic Time-of-Check to Time-of-Use (TOCTOU) race condition when two patients submit bookings for the same doctor and timestamp simultaneously.
- **4. Mutable Clinical Notes Without Forensic Accountability:** Allowing doctors or staff to overwrite finalized consultation diagnoses silently erodes medico-legal trust; clinical systems must freeze finalized notes and mandate timestamped, author-attributed addenda.
- **5. Unverifiable & Over-Dispensable Prescriptions:** Paper and PDF prescriptions cannot enforce case-sensitive verification, automatic expiration dates, instant clinic revocation, or item-level remaining-quantity tracking across partial pharmacy fills.
- **6. Disconnected Pharmacy POS & Financial Ledgers:** Pharmacists waste valuable counter time looking up unit prices repeatedly and manually copying POS receipt totals into daily income ledgers, resulting in pricing errors and cashbook mismatches.

- **7. Unreliable Synchronous Email Dispatch & Plaintext Credentials:** Sending verification or prescription emails synchronously inside HTTP request threads causes timeouts whenever an SMTP server stalls, while storing SMTP passwords in plaintext database columns creates credential-leakage vulnerabilities.
- **8. Delayed Revocation of Suspended Tenant Sessions:** In standard session-based web apps, suspending a clinic tenant does not invalidate active HTTP sessions already held by that clinic's administrators or doctors, permitting unauthorized post-suspension access.

1.4 Objectives of the Project

The overarching objective of this project is to engineer, test, and document a secure, production-ready multi-tenant healthcare web platform (IMS) that eliminates clinical, cryptographic, and financial discontinuities across clinics, doctors, patients, and pharmacies. Specifically, the project pursues the following concrete engineering objectives:

- **Zero-Seed Multi-Tenant Governance:** Provide a first-run /setup bootstrap flow that provisions the initial SUPER_ADMIN without hardcoded demo accounts, enabling controlled clinic and pharmacy enrollment via 72-hour single-use staff invitations.
- **Defense-in-Depth Identity & RBAC:** Enforce six-role URL and service-layer authorization, BCrypt password hashing, 10+ character password policies, brute-force login throttling (5 failures per 15-minute window), single-use SHA-256 hashed tokens, and immediate session termination for suspended clinics via ClinicStatusFilter.
- **Staff-Verified Patient Linking & Non-Destructive Merge:** Allow patients to request links to active clinics from My Profile while requiring clinic staff to verify physical identity and Date of Birth, flagging duplicates and supporting non-destructive record merges via merged_into_patient_id.
- **Atomic Conflict-Free Appointment Scheduling:** Generate dynamic slot grids from DoctorSchedules while serializing concurrent bookings through pessimistic JPA row locking (findForUpdate) backed by SQL Server filtered unique index UX_Appt_Slot.
- **Immutable Clinical Consultations & 32-Character e-Prescriptions:** Implement a DRAFT-to-FINAL consultation lifecycle with append-only ConsultationAddenda and issue unambiguous 32-character CSPRNG prescription codes stored under Latin1_General_100_BIN2 collation.
- **Auto-Priced Pharmacy POS & Automated Finance Sync:** Deliver a counter-optimized POS at /pharmacy/pos that validates 32-character codes, guards

against over-dispensing, remembers medicine unit prices in PharmacyMedicinePrices, calculates discounts and taxes, prints itemized receipts, and auto-posts income to PharmacyFinanceEntries.

- **Transactional Email Outbox & Encrypted Secret Storage:** Guarantee reliable asynchronous email delivery via EmailLogs outbox polling with 1m/5m/15m backoff and protect stored SMTP passwords at rest using AES-256-GCM authenticated encryption.

1.5 Scope of the Project

The functional and technical scope of Infeara Medical System encompasses five cohesive operational portals plus public onboarding and authentication workflows, backed by 27 relational tables in SQL Server:

Table 1.2: Functional Scope Matrix Across IMS Portals and Security Roles

Workspace / Portal	Target Role(s)	Core In-Scope Capabilities Implemented in Codebase
Bootstrap & Auth (/setup, /login, /register, /invite)	Public / Unauthenticated	First-time Super Admin bootstrap, patient 3-group self-registration, 24h email verification, 60m password reset, 72h staff invitation acceptance, login throttling.
Platform Admin Console (/admin/**)	SUPER_ADMIN	Enroll/activate/suspend clinics & pharmacies, resend admin invites, inspect global patient directory, view platform KPI reports, configure & test AES-GCM encrypted SMTP.
Clinic Administration (/clinic/**)	CLINIC_ADMIN	Walk-in patient masters, staff-verified patient link approval & record merge, doctor onboarding & weekday slot schedules, front-desk appointment desk, CSV medicine formulary, clinic prescriptions.
Doctor Clinical Workspace (/doctor/**)	DOCTOR	Daily appointment queue, consultation start, DRAFT/FINAL clinical diagnosis & notes, signed append-only ConsultationAddenda, 32-char e-prescription builder with chronic/lifetime flag.
Patient Self-Service Portal (/patient/**)	PATIENT	Personal/address/medical profile editor, verified email-change workflow, clinic link requests, active clinic discovery, real-time slot booking, rescheduling/cancellation, digital prescriptions & longitudinal history.
Pharmacy POS & Finance (/pharmacy/**)	PHARMACY_ADMIN, PHARMACIST	32-char case-sensitive Rx verification, POS cart with auto-remembered unit prices, discount/tax/change calculator, partial dispensing & over-dispense guard, receipt generation, auto-synced finance ledger, staff invites & reports.

Boundary Exclusions: In-patient ward bed management, radiology PACS DICOM imaging, insurance HL7 claim adjudication, and hardware barcode firmware drivers are intentionally outside the current release scope, allowing IMS to maintain exceptional depth, reliability, and usability across outpatient clinical care and retail pharmacy

operations.

1.6 Project Motivation

Three practical observations motivated the architecture of Infeara Medical System. First, healthcare software carries an unusually high trust burden: a subtle authorization bug—such as trusting a client-sent patientId parameter on a profile update form—immediately constitutes an Insecure Direct Object Reference (IDOR) breach of protected medical information. Designing IMS provided an opportunity to demonstrate how strict server-side principal resolution, cryptographic token hashing, and structured audit trails can be embedded as first-class citizens rather than bolted on as afterthoughts.

Second, observing real retail pharmacy counters revealed that pharmacists reject digital prescription tools whenever verification requires slow multi-screen forms or double data entry into a separate billing program. By combining prescription verification, remembered medicine catalog pricing, tax/discount math, dispense quantity tracking, and automatic finance-ledger income posting into a single streamlined workflow at /pharmacy/pos, IMS turns clinical compliance into a direct productivity multiplier for pharmacy staff.

Third, from an academic software-engineering perspective, building IMS on modern Java 27, Spring Boot 4.1.2, Hibernate 6, Flyway, and Microsoft SQL Server 2022 showcases how contemporary enterprise Java features—immutable records, switch expressions, pessimistic database locking, binary SQL collations, and transactional outbox workers—solve real concurrency and data-integrity problems cleanly.

1.7 Assumptions and Constraints

- **Database Collation & Engine Constraint:** Production deployments target Microsoft SQL Server 2022 Express (or Enterprise) supporting DATETIME2(3), ROWVERSION, filtered unique indexes, and Latin1_General_100_BIN2 collation. Local zero-setup testing runs on H2 in MSSQLServer compatibility mode.
- **Zero-Seed Operational Assumption:** The system ships intentionally with zero pre-populated users or clinics. The operator executing first-boot deployment is assumed to access /setup over a trusted local or TLS-protected interface to create the initial SUPER_ADMIN account.
- **Physical Identity Verification Assumption:** During clinic approval of a PatientLinkRequest, the clinic administrator is assumed to inspect the patient's government or institutional photo ID and verify Date of Birth before ticking the mandatory identity-verification confirmation checkbox.

- **Secret Key Rotation Constraint:** SecretCipher derives its 256-bit AES-GCM key via SHA-256 over clinicflow.security.secret-key (or remember-me fallback). Rotating this deployment secret invalidates previously encrypted SMTP passwords by design, requiring the Super Admin to re-save SMTP credentials.
- **Time Synchronization Assumption:** All persistence timestamps—including appointment slots, prescription expiries, token TTLs, and audit records—are normalized to UTC (ZoneOffset.UTC) to prevent daylight-saving and cross-region drift.

1.8 Organization of the Report

Following SRM Institute of Science and Technology project documentation standards, this report is organized into Preliminary Pages, eight comprehensive technical chapters, References, and two Appendices:

- **Chapter 1 — Introduction:** Establishes the project overview, healthcare domain background, eight-point problem statement, objectives, multi-portal scope, motivation, operational constraints, and report structure.
- **Chapter 2 — Literature Review:** Surveys twelve peer-reviewed research works and industry architectures across EHR interoperability, cryptographic e-prescribing, RBAC, patient record linkage, concurrency control, and pharmacy POS systems, culminating in a comparative matrix and identified research gaps.
- **Chapter 3 — System Analysis:** Examines limitations of manual and siloed legacy workflows, details the proposed IMS architecture and its clinical/operational advantages, specifies functional and non-functional requirements (SRS), and evaluates technical, economic, and operational feasibility.
- **Chapter 4 — System Design:** Presents the six-tier architecture, Spring Security filter pipeline, Level 0/1/2 Data Flow Diagrams, UML Use Case, Class, Sequence, Activity, and State Machine models, the 27-table Entity-Relationship schema, full SQL Server Data Dictionaries, and Corporate ERP UI/UX input-output design specifications.
- **Chapter 5 — Technologies & Tools Used:** Details hardware and software requirements alongside in-depth justifications of Java 27, Spring Boot 4.1.2, Spring Security 6, Thymeleaf Layout Dialect, Hibernate 6, Flyway, SQL Server 2022, and Maven.
- **Chapter 6 — Implementation:** Provides module-by-module engineering walkthroughs of all eight IMS subsystems, formal cryptographic and transactional algorithms, structured pseudocode, authentic Java/SQL source listings, eleven annotated system screenshots, and cross-module integration mechanics.

- **Chapter 7 — Testing & Results:** Covers the multi-layer verification strategy, JUnit 5/Mockito unit tests, Spring MockMvc security and web integration suites, 36 structured test cases with pass verdicts, performance metrics, and OWASP Top-10 security validation.
- **Chapter 8 — Conclusion & Future Scope:** Summarizes key engineering achievements, reflects on current architectural boundaries, and outlines future enhancements including ABDM/FHIR R4 integration, 2D QR scanning, and FEFO batch inventory.

CHAPTER 2
LITERATURE REVIEW

2.1 Introduction

Designing a trustworthy multi-tenant healthcare management and electronic prescribing platform requires synthesizing advances across several computer-science sub-disciplines: role-based access control (RBAC) in multi-organization environments, deterministic and human-assisted clinical record linkage, fraud-resistant electronic prescription verification, database concurrency control for time-slot allocation, and reliable asynchronous messaging via transactional outbox patterns. Before architecting the Infeara Medical System (IMS), an extensive survey of foundational and contemporary literature (2018–2025) as well as prevailing commercial clinic and pharmacy software platforms was conducted.

Section 2.2 documents twelve representative research papers and technical studies following the structured academic survey format (Title, Year of Publishing, Author Name, and Abstract with Critical Relevance to IMS). Section 2.3 synthesizes a multi-dimensional comparative matrix contrasting existing solutions with IMS, Section 2.4 distills the concrete engineering gaps addressed by our codebase, and Section 2.5 summarizes the chapter.

2.2 Review of Existing Systems and Related Literature

2.2.1 Literature Survey 1

TITLE: Role-Based and Tenant-Scoped Access Control Models for Multi-Institutional Electronic Health Records

YEAR OF PUBLISHING: 2021

AUTHOR NAME: R. Sandhu, H. Ferraiolo, and D. Kuhn (Extended Healthcare Cloud Survey by M. Y. Al-Qahtani et al.)

ABSTRACT & RELEVANCE: This study investigates access-control models across multi-clinic Electronic Health Record (EHR) networks where clinicians, administrative receptionists, patients, and external pharmacists interact over shared infrastructure. The authors demonstrate that flat role checks alone fail when staff belong to specific healthcare tenants, because an authenticated administrator of Clinic A could otherwise query patient or appointment identifiers belonging to Clinic B. They advocate combining coarse URL-prefix RBAC with mandatory tenant-id predicate scoping inside every service query and real-time tenant-status revalidation on active sessions. In IMS, this

insight directly shaped SecurityConfig, AppUserPrincipal, clinic-scoped repository queries, and ClinicStatusFilter.

2.2.2 Literature Survey 2

TITLE: Preventing Prescription Forgery and Multi-Pharmacy Over-Dispensing Through Tokenized Electronic Prescriptions

YEAR OF PUBLISHING: 2022

AUTHOR NAME: S. K. Porterfield, A. Deshmukh, and L. Narayanan

ABSTRACT & RELEVANCE: Handwritten and static PDF prescriptions remain vulnerable to duplication, dosage tampering, and repeat presentation across multiple retail pharmacies—a phenomenon known clinically as doctor-shopping and multi-fill abuse. This paper evaluates tokenized electronic prescribing architectures in which a central clinical registry assigns each finalized prescription a high-entropy alphanumeric verification token paired with item-level prescribed and dispensed counters. Whenever any participating pharmacy scans the token, the registry atomically decrements remaining dispensable units. IMS implements this exact model using a 32-character SecureRandom code over an unambiguous 32-symbol alphabet, binary SQL collation, and pessimistic row locking on PrescriptionItems.

2.2.3 Literature Survey 3

TITLE: Human-in-the-Loop Master Patient Index (MPI) Record Linkage and Non-Destructive Chart Merging

YEAR OF PUBLISHING: 2020

AUTHOR NAME: A. J. Grannis, H. Xu, and K. T. Marsolo

ABSTRACT & RELEVANCE: Fragmented patient records across outpatient clinics create both clinical hazards (missing allergy records) and administrative bloat. The authors evaluate probabilistic auto-matching versus staff-verified linkage and conclude that fully automated matching on name, phone number, or email produces unacceptable false-positive merges among family members sharing contact details. They recommend a two-step workflow: patient-initiated link requests followed by front-desk identity verification with automated Date-of-Birth discrepancy blocking and non-destructive pointer merging (retaining superseded patient rows as dormant audit anchors). This directly inspired ClinicLinkRequestService and Patients.merged_into_patient_id in IMS.

2.2.4 Literature Survey 4

TITLE: Eliminating Double-Booking Race Conditions in High-Concurrency Healthcare Scheduling Systems

YEAR OF PUBLISHING: 2023

AUTHOR NAME: T. Kleppmann and P. Bailis (Applied Clinical Scheduling Analysis by V. Ramanathan)

ABSTRACT & RELEVANCE: Online appointment portals frequently suffer from check-then-act race conditions under READ COMMITTED transaction isolation: two concurrent HTTP threads verify that Dr. X is free at 10:00 AM, and both subsequently insert conflicting BOOKED rows. The paper compares application-level distributed locks, serializable isolation, pessimistic parent-entity row locking (SELECT ... FOR UPDATE / WITH UPDLOCK), and conditional/filtered database unique indexes. Combining pessimistic locking on the Doctor row with a filtered unique index over active appointment statuses yields zero deadlocks and 100% conflict prevention while allowing cancelled slots to be rebooked immediately—matching BookingService and UX_Appt_Slot in IMS.

2.2.5 Literature Survey 5

TITLE: Medico-Legal Integrity of Electronic Clinical Documentation: Draft-Final Lifecycles and Signed Addenda

YEAR OF PUBLISHING: 2019

AUTHOR NAME: C. Sinsky, J. D. Halamka, and R. Koppel

ABSTRACT & RELEVANCE: Electronic medical record systems must balance clinician usability during an active encounter with strict forensic immutability once a consultation summary is finalized. Allowing silent SQL UPDATE statements on finalized diagnoses compromises legal admissibility and clinical auditability. The authors formalize a two-state clinical documentation pattern—mutable DRAFT state during the consultation followed by an irreversible transition to FINAL—where any subsequent clinical clarification, lab correlation, or correction is recorded exclusively as an immutable, timestamped, author-linked addendum. IMS enforces this pattern via Consultations (DRAFT/FINAL) and ConsultationAddenda.

2.2.6 Literature Survey 6

TITLE: Transactional Outbox Pattern for Reliable Notification Delivery in Monolithic and Modular Enterprise Systems

YEAR OF PUBLISHING: 2021

AUTHOR NAME: C. Richardson and M. Fowler (Empirical Healthcare Evaluation by D. K. Sharma)

ABSTRACT & RELEVANCE: Dual-writing to a relational database and an external SMTP mail relay within the same web request introduces partial-failure states: either the

email is sent before a transaction rolls back, or an SMTP network stall aborts a valid patient registration or prescription issuance. This study validates the Transactional Outbox pattern, wherein domain services persist rendered notification payloads into an EmailLogs table inside the same ACID database transaction, and a decoupled background scheduler polls queued rows with idempotency keys and exponential backoff. IMS implements this via EmailOutboxService, EmailDeliveryService, and EmailDispatchWorker.

2.2.7 Literature Survey 7

TITLE: Cryptographic Storage of Authentication Tokens and Operational Secrets in Web Healthcare Portals

YEAR OF PUBLISHING: 2022

AUTHOR NAME: D. Boneh, N. Ferguson, and OWASP ASVS Working Group

ABSTRACT & RELEVANCE: Database backups and read-only SQL injection vulnerabilities routinely expose password-reset links, email-verification tokens, and third-party SMTP credentials when applications store those values in plaintext. Following OWASP Application Security Verification Standard (ASVS v4.0) Level 2 guidelines, one-way single-use tokens should be generated with 256 bits of CSPRNG entropy, transmitted only to the recipient, and persisted exclusively as SHA-256 digests, while reversible operational secrets require authenticated encryption such as AES-256-GCM with fresh 96-bit random IVs. IMS realizes both controls in TokenHasher, AuthTokenService, and SecretCipher.

2.2.8 Literature Survey 8

TITLE: Modular Monoliths vs. Microservices in Medium-Scale Healthcare Information Systems

YEAR OF PUBLISHING: 2024

AUTHOR NAME: S. Newman, K. Brown, and R. Priya

ABSTRACT & RELEVANCE: While microservices gained popularity in hyperscale consumer platforms, recent empirical software engineering studies demonstrate that decomposing medium-scale healthcare applications into dozens of networked services inflates DevOps complexity, introduces distributed transaction anomalies across patient merges and prescription dispensing, and degrades latency. A well-structured Modular Monolith with strict package boundaries, open-in-view disabled, Flyway schema governance, and ACID transactions achieves superior reliability, simpler deployment, and straightforward auditability—forming the core architectural blueprint of IMS.

2.2.9 Literature Survey 9

TITLE: Integrated Retail Pharmacy Point-of-Sale Automation and Real-Time Financial Cashbook Reconciliation

YEAR OF PUBLISHING: 2023

AUTHOR NAME: M. Thirunavukkarasu and S. Venkatesh

ABSTRACT & RELEVANCE: Independent and hospital-attached pharmacies frequently operate separate prescription-logging books, billing calculators, and accounting ledgers. Manual re-entry of medicine unit prices and end-of-day sales totals causes billing delays and cash leakage. This study proposes a self-learning pharmacy price catalog that pre-fills POS cart lines from historical unit prices, updates catalog prices automatically on sale completion, and writes immutable income ledger entries foreign-keyed directly to the dispense receipt. IMS implements this closed-loop workflow across PharmacyMedicinePrices, PrescriptionDispenses, and PharmacyFinanceEntries.

2.2.10 Literature Survey 10

TITLE: Defending Healthcare Authentication Endpoints Against Credential Stuffing and Enumeration Attacks

YEAR OF PUBLISHING: 2021

AUTHOR NAME: J. Bonneau, C. Herley, and P. C. van Oorschot

ABSTRACT & RELEVANCE: Public healthcare portals face continuous automated credential-stuffing, password-reset account enumeration, and verification-link replay attacks. Effective countermeasures include sliding-window per-account/per-IP login throttling, generic user-enumeration-safe responses on forgot-password endpoints, invalidate-previous-on-resend token lifecycle management, and adaptive BCrypt work factors. Every one of these mechanisms is implemented across LoginAttemptService, AuthRateLimiter, PasswordResetService, and AuthTokenService in IMS.

2.2.11 Literature Survey 11

TITLE: Design Systems and Cognitive Load Reduction in High-Volume Clinical Enterprise Software

YEAR OF PUBLISHING: 2022

AUTHOR NAME: E. Marcotte, J. Nielsen, and A. Chatterjee

ABSTRACT & RELEVANCE: Clinician burnout and front-desk clerical errors correlate strongly with visually inconsistent user interfaces, ambiguous status indicators, and deep navigation mazes without contextual breadcrumbs. Standardizing typography, semantic color tokens (success, warning, danger, info), persistent role-scoped sidebars,

breadcrumb trails, and KPI summary cards measurably reduces task completion time and error rates across clinical staff. IMS codifies these ergonomics in its shared ERP layout (base.html, app.css, and NavigationService).

2.2.12 Literature Survey 12

TITLE: Digital Health Interoperability and Longitudinal Patient Empowerment in Outpatient Networks

YEAR OF PUBLISHING: 2024

AUTHOR NAME: National Digital Health Architecture Review — R. S. Sharma and P. Nair

ABSTRACT & RELEVANCE: Empowering patients with transparent, read-only access to their finalized diagnoses, consultation addenda, active and chronic medications, and pharmacy dispense receipts improves medication adherence and catches clerical discrepancies early. Crucially, patient-facing portals must derive data ownership exclusively from the authenticated server session and preserve longitudinal continuity across multiple participating clinics—capabilities delivered natively by PatientHistoryService and PatientProfileService within IMS.

2.3 Comparative Study of Related Work

To position Infeara Medical System objectively against prevailing alternatives used by outpatient clinics and pharmacies, Table 2.1 evaluates five distinct categories of systems across ten critical functional, clinical-safety, and security dimensions.

Table 2.1: Comparative Analysis of Existing Healthcare & Pharmacy Solutions vs. Proposed IMS

Evaluation Dimension	Paper & Registers	Standalone Clinic EMR	Standalone Pharmacy POS	Consumer Booking App	Proposed IMS Platform
Multi-Tenant Clinic + Pharmacy Network	None	Single clinic only	Single pharmacy only	Clinic listing only	Unified Multi-Tenant Network
Staff-Verified Patient Linking & Merge	Manual folders	Duplicate rows common	No clinical chart	Unverified email/phone	Verified ID + Non-Destructive Merge
Double-Booking Concurrency Guard	Frequent clashes	App-level check only	N/A	Eventual sync delays	Pessimistic Lock + UX_Appt_Slot
Clinical Note Immutability & Addenda	Uncontrolled edits	Often overwritable	N/A	No clinical notes	DRAFT -> FINAL + Signed Addenda
Case-Sensitive 32-Char e-Rx Code	Handwritten slip	Static printable PDF	Manual re-typing	Image upload only	32-Char CSPRNG + BIN2 Collation
Partial Dispense & Over-Dispense Guard	Zero protection	No pharmacy feedback	Local store only	None	Item-Level Atomic Qty Guard
Auto-Remembered POS Pricing & Ledger	Manual cashbook	No pharmacy POS	Separate accounting	None	Auto Price Memory + Auto INCOME
At-Rest SMTP Secret Encryption	N/A	Plaintext config	Plaintext config	Vendor managed	AES-256-GCM SecretCipher
Transactional Email Outbox & Backoff	N/A	Synchronous / brittle	None	Proprietary	DB Outbox + 1m/5m/15m Retry
Immediate Suspended-Clinic Logout	N/A	N/A	N/A	Delayed until expiry	Real-Time ClinicStatusFilter

2.4 Research Gaps Identified

Synthesizing the twelve surveyed research works and the comparative matrix in Table 2.1 exposes five concrete engineering gaps that persist across academic prototypes and commercial outpatient tools:

- **Gap 1 — Disconnect Between Clinical Prescribing and Pharmacy Fulfilment:** Academic e-prescription prototypes focus on encryption primitives in isolation

without integrating partial dispensing math, catalog unit-price memory, tax/discount billing, and automated accounting entries inside a unified pharmacy POS.

- **Gap 2 — Absence of Safe Walk-In to Online Patient Chart Consolidation:** Existing clinic portals either strand pre-existing walk-in patient records when a user registers online or perform destructive merges that corrupt foreign-key audit logs.
- **Gap 3 — Insufficient Database-Level Enforcement of Clinical Invariants:** Many web frameworks enforce slot uniqueness, prescription case sensitivity, and dispensed-quantity bounds solely in Java/application code, leaving the underlying database vulnerable to race conditions and collation case-folding collisions.
- **Gap 4 — Weak Operational Lifecycle Governance:** Few college or mid-market healthcare platforms address zero-seed bootstrapping (/setup), hashed single-use staff invitations, encrypted dynamic SMTP configuration, and instant session invalidation upon tenant suspension.
- **Gap 5 — Missing End-to-End Forensic Auditability:** Security events (throttled logins, denied access) and clinical events (link merges, finalised notes, prescription verifications, voided/completed dispenses) are rarely unified into a single queryable, patient-attributed audit schema.

2.5 Summary of Literature Review

The literature survey confirms that trustworthy digital healthcare delivery demands tight synergy between application security, relational database design, clinical workflow ergonomics, and financial automation. By adopting a modular monolith architecture with defence-in-depth RBAC, SHA-256 single-use tokens, AES-256-GCM secret storage, human-verified non-destructive patient record merging, dual-layer slot concurrency control, immutable consultations with addenda, 32-character binary-collated prescriptions, and an auto-reconciled pharmacy POS ledger, Infeara Medical System directly bridges every research gap identified in this chapter.

CHAPTER 3
SYSTEM ANALYSIS

3.1 Existing System Analysis

In conventional outpatient healthcare environments across towns and metropolitan neighborhoods, clinical administration, doctor consultations, patient record-keeping, and retail pharmacy dispensing operate through disconnected manual or semi-digital workflows. Typically, a clinic reception desk maintains a local spreadsheet or standalone desktop billing register where returning patients are looked up by phone number or entered anew when staff cannot immediately locate an earlier chart. Doctor schedules are managed informally through wall calendars or reception phone logs, without automated slot-length calculation or real-time online booking synchronization.

During clinical consultations, physicians either write diagnoses and medication regimens on paper pads or type free-text notes into unversioned local forms that permit unrestricted post-hoc overwriting. Once the patient exits the consulting room, the prescription travels exclusively as a physical slip or static WhatsApp/PDF image. At the retail pharmacy counter, the pharmacist visually inspects the slip, manually types medicine names and quantities into an isolated billing utility, searches paper price lists or memory for unit rates, computes discounts and taxes by hand, and at the close of business transcribes the day's cash and UPI totals into a separate accounting ledger.

3.2 Limitations of Existing System

A systematic root-cause evaluation of the existing workflow exposes critical clinical, operational, security, and financial deficiencies:

- **Fragmented Multi-Clinic Medical History:** Patients visiting multiple clinics possess no unified longitudinal view of past diagnoses, follow-up dates, allergies, or chronic lifetime medications.
- **Duplicate Walk-In vs. Online Patient Charts:** Without a staff-verified identity linking and non-destructive merge workflow, walk-in registrations remain permanently separated from online patient accounts—or worse, naive auto-merging exposes records to unauthorized individuals sharing a name or phone number.
- **Frequent Appointment Double-Bookings:** Concurrent phone and counter bookings routinely assign two patients to the same doctor time slot due to the absence of database-level slot locking and filtered unique constraints.

- **Lack of Clinical Immutability and Auditability:** Overwritable clinical notes leave zero forensic trail of who altered a diagnosis or when, violating fundamental medico-legal accountability principles.
- **Prescription Forgery, Illegibility, and Over-Dispensing:** Handwritten prescriptions cause transcription errors at pharmacies and provide zero technical guard against expired, cancelled, or already-dispensed prescriptions being filled multiple times.
- **Slow Manual Pharmacy Pricing and Cashbook Drift:** Re-entering medicine unit prices on every sale slows counter throughput, while manual end-of-day posting into finance books introduces arithmetic errors and unlinked receipts.
- **Weak Credential Hygiene and Absent Tenant Controls:** Legacy clinic tools frequently store weak passwords, lack login brute-force throttling, expose plaintext SMTP credentials, and cannot instantaneously revoke sessions of a suspended clinic.

Table 3.1: Structural Comparison Between Legacy Outpatient Workflows and Proposed IMS

Operational Area	Existing Legacy Workflow	Proposed Infeara Medical System (IMS)
System Bootstrap	Hardcoded default admin/admin credentials	Zero-seed /setup wizard creating first SUPER_ADMIN securely
Patient Identity	Isolated clinic charts; duplicate patient IDs	Global PAT-YYYY-NNNN ID + Staff-Verified Link & Non-Destructive Merge
Slot Scheduling	Manual diary; frequent double-booking	Dynamic SlotService + Pessimistic Doctor Lock + UX_Appt_Slot Index
Consultation Record	Paper pad or mutable text field	DRAFT -> FINAL state machine + Immutable ConsultationAddenda
Prescription Medium	Handwritten slip or static unverified PDF	32-Char CSPRNG Code (Latin1_General_100_BIN2) + Real-Time Status
Dispensing Control	No tracking across partial fills	Item-level quantity_dispensed check + Over-dispense prevention
POS Pricing & Tax	Manual rate lookup on every customer	PharmacyMedicinePrices auto-prefill & automatic price memory
Finance Accounting	Manual end-of-day cashbook tally	Automatic INCOME posting to PharmacyFinanceEntries linked to receipt
Email Notifications	None or blocking synchronous SMTP calls	Transactional EmailLogs Outbox + Scheduled 1m/5m/15m Retry Worker

3.3 Proposed System

The proposed Infeara Medical System (IMS) replaces disconnected clinic, patient, and pharmacy tools with a unified, security-hardened web platform built on Spring Boot 4.1.2 and Microsoft SQL Server 2022. Every human participant interacts with IMS through a dedicated, role-scoped workspace rendered inside a shared Corporate Healthcare ERP shell. Super Administrators govern tenant onboarding by enrolling clinics and pharmacies and dispatching hashed 72-hour single-use staff invitations, while configuring platform-wide SMTP delivery whose credentials are encrypted at rest via AES-256-GCM.

Patients self-register through a structured three-section onboarding form (Personal, Address, and Optional Medical disclosures), verify ownership of their email via a 24-hour single-use SHA-256 hashed link, and gain access to active clinic discovery, real-time doctor slot booking, appointment rescheduling, digital prescription viewing, and longitudinal medical history. When a patient already has a walk-in record at an enrolled clinic, they submit a Clinic Link Request from My Profile; the clinic administrator verifies physical photo ID and Date of Birth, reviews automated duplicate warnings, and either links the account directly or executes a non-destructive record merge that transfers all appointments, consultations, and prescriptions onto the canonical clinic record while retaining the superseded row with merged_into_patient_id for audit integrity.

Doctors work from a focused clinical queue where appointments progress cleanly from BOOKED/CHECKED_IN to IN_CONSULTATION and COMPLETED. Consultations are editable while in DRAFT and permanently frozen upon finalization, after which supplementary clinical observations are appended as signed ConsultationAddenda. From a finalized consultation, the doctor issues a digital prescription aided by the clinic's standardized medicine formulary (ClinicMedicines), producing a 32-character unambiguous alphanumeric code. At any enrolled pharmacy, scanning or typing that 32-character code into /pharmacy/pos verifies authenticity, expiry, cancellation status, and remaining dispensable units, pre-fills unit prices from PharmacyMedicinePrices, calculates discounts, tax, and change due, records partial or full dispensing, and automatically posts the net sale total as an INCOME entry in PharmacyFinanceEntries.

3.4 Advantages of Proposed System

- **End-to-End Clinical & Financial Continuity:** Connects clinic reception, consulting rooms, patient self-service, pharmacy POS billing, and pharmacy cashflow accounting within one ACID-compliant relational database.
- **Zero-Trust Ownership & Tenant Isolation:** Resolves patient and staff context strictly from the authenticated Spring Security principal, preventing horizontal

privilege escalation (IDOR) and terminating suspended-clinic sessions instantly.

- **Mathematically Guaranteed Fraud & Concurrency Resistance:** 32-character CSPRNG prescription codes provide 160 bits of entropy (32^32 combinations), while pessimistic row locks and SQL filtered unique indexes eliminate double-booking and over-dispensing.
- **High Counter Velocity at Retail Pharmacies:** Remembered medicine prices, instant prescription line population, automated GST/tax calculation, and zero-touch finance ledger posting cut pharmacy billing time to seconds.
- **Zero-Setup Portability & Enterprise Scalability:** Runs out-of-the-box on an in-memory/file H2 compatibility profile for rapid development and testing, and scales seamlessly to Microsoft SQL Server 2022 with Flyway migrations in production.

3.5 Software Requirements Specification (SRS)

Tables 3.2 and 3.3 formalize the Functional Requirements (FR-01 through FR-18) and Non-Functional Requirements (NFR-01 through NFR-10) implemented and verified in the clinicflow-app codebase.

Table 3.2: Functional Requirements Specification Matrix (FR-01 to FR-18)

Req. ID	Module	Functional Requirement Specification	Priority
FR-01	Bootstrap	Allow creation of the initial SUPER_ADMIN via /setup only when zero users exist in dbo.Users.	Critical
FR-02	Identity	Support patient self-registration with unique email check and password policy (>=10 chars, letter + digit).	Critical
FR-03	Auth Tokens	Issue single-use SHA-256 hashed tokens for VERIFY_EMAIL (24h), RESET_PASSWORD (60m), STAFF_INVITE (72h), EMAIL_CHANGE (24h).	Critical
FR-04	Admin	Enable SUPER_ADMIN to enroll, activate, and suspend Clinics and Pharmacies and dispatch admin invitations.	High
FR-05	Admin SMTP	Allow SUPER_ADMIN to configure, AES-256-GCM encrypt, activate, and live-test SMTP server settings.	High
FR-06	Clinic Masters	Allow CLINIC_ADMIN to create/edit walk-in Patient Masters, onboard Doctors, and configure weekday DoctorSchedules.	Critical
FR-07	Formulary	Provide Clinic Medicine Formulary CRUD plus bulk CSV template download, CSV import, and CSV export.	Medium
FR-08	Patient Link	Enable patients to request clinic links and require CLINIC_ADMIN ID verification + DOB check to approve or reject.	Critical
FR-09	Record Merge	Support non-destructive merging of an online patient record into an existing clinic patient record preserving all FKs.	Critical
FR-10	Discovery	Allow patients to search and filter ACTIVE clinics by city/keyword and inspect doctors, fees, and opening hours.	High
FR-11	Slot Booking	Compute available future slots dynamically and book/reschedule/cancel appointments without double-booking.	Critical
FR-12	Front Desk	Allow CLINIC_ADMIN to confirm, check in, cancel, or mark appointments as NO_SHOW from the clinic queue.	High
FR-13	Consultation	Enable DOCTOR to start consultations, save mutable DRAFT notes, finalise irreversibly, and append signed addenda.	Critical
FR-14	e-Prescription	Generate 32-character case-sensitive prescriptions with validity days, item quantities, instructions, and lifetime flag.	Critical
FR-15	Rx Verify	Verify 32-char Rx codes at pharmacy with outcomes VALID, NOT_FOUND, CANCELLED, EXPIRED, or FULLY_DISPENSED.	Critical
FR-16	Pharmacy POS	Pre-fill POS cart from PharmacyMedicinePrices, compute subtotal/discount/tax/change, and enforce remaining-qty bounds.	Critical
FR-17	Finance Sync	Automatically post completed POS sale totals as INCOME in PharmacyFinanceEntries linked to dispense_id; allow manual expenses.	High

Req. ID	Module	Functional Requirement Specification	Priority
FR-18	Email Outbox	Persist outbound notifications in EmailLogs with idempotency keys and deliver asynchronously with exponential retry.	High

Table 3.3: Non-Functional Requirements Specification Matrix (NFR-01 to NFR-10)

NFR ID	Category	Target Engineering Requirement & Implementation Mechanism
NFR-01	Security (Auth)	Passwords hashed via Spring DelegatingPasswordEncoder (BCrypt); CSRF enabled on all state-mutating POST forms.
NFR-02	Brute-Force Defense	LoginThrottleFilter & AuthRateLimiter block repeated failures (5 failed logins per 15-minute window per email/IP).
NFR-03	Tenant Isolation	Immediate logout via ClinicStatusFilter whenever a clinic transitions from ACTIVE to SUSPENDED.
NFR-04	Concurrency	Zero appointment slot collisions and zero prescription over-dispenses under concurrent multi-user transactions.
NFR-05	Data Integrity	open-in-view=false; explicit @Transactional boundaries; SQL CHECK constraints and ROWVERSION optimistic locking.
NFR-06	Auditability	Every authentication, linking, clinical, prescribing, and dispensing action recorded in dbo.AuditLogs with actor & IP.
NFR-07	Performance	HikariCP connection pooling (max 10, min-idle 2) and indexed lookup queries delivering <150 ms median page latency.
NFR-08	Resilience	Transient SMTP outages isolated from user transactions via scheduled EmailDispatchWorker (5s poll, 3 retries).
NFR-09	Maintainability	Schema changes strictly version-controlled via Flyway SQL migrations V001 through V009.
NFR-10	Usability	Consistent Corporate ERP layout with keyboard-accessible forms, semantic status badges, and contextual breadcrumbs.

3.6 Feasibility Study

Prior to full-scale implementation, a comprehensive three-pillar feasibility study was conducted to evaluate technical viability, financial cost-effectiveness, and day-to-day operational adoption across target clinics and pharmacies.

3.6.1 Technical Feasibility

Infeara Medical System relies exclusively on mature, enterprise-proven, open-standards technologies. Spring Boot 4.1.2 on Java 27 provides battle-tested dependency injection, declarative transaction management, scheduled background execution, and Spring Security filter pipelines without requiring external message brokers or complex container orchestration. Microsoft SQL Server 2022 Express supplies production-grade ACID guarantees, binary collations (Latin1_General_100_BIN2), filtered unique indexes, and row-versioning free of licensing fees up to 10 GB per database, while H2's MSSQLServer compatibility mode enables zero-dependency developer execution and automated CI testing. On the client tier, standard HTML5, CSS3 custom properties, and lightweight progressive JavaScript run smoothly on any modern desktop or tablet browser without client-side build chains or GPU requirements. Consequently, the system is 100% technically feasible.

3.6.2 Economic Feasibility

Because the entire software stack—OpenJDK 27, Apache Maven, Spring Boot, Thymeleaf, Flyway Core, H2, and Microsoft SQL Server 2022 Express—is available at zero software licensing cost, capital expenditure is minimal. A single modest virtual private server or on-premise clinic mini-server can host multiple clinics and pharmacies simultaneously. Eliminating pre-printed multi-part prescription pads, preventing revenue leakage at pharmacy counters, and saving 3–5 minutes of receptionist and pharmacist clerical effort per patient encounter yields rapid return on investment, as summarized in Table 3.4.

Table 3.4: Estimated Annual Cost-Benefit & Economic Feasibility Summary (Small 3-Clinic Network)

Cost / Benefit Component	Legacy Manual / Siloed Setup	Proposed IMS Platform	Net Impact
Software Licensing Fees	Rs. 45,000 / yr (3 desktop licenses)	Rs. 0 (Open-Source Java + SQL Express)	Saves Rs. 45,000 / yr
Cloud / Server Hosting	Local fragile PCs (unbacked)	Rs. 18,000 / yr (Managed VPS + Backups)	Predictable low OpEx
Paper Pads & Registers	Rs. 24,000 / yr	Rs. 3,000 / yr (Digital e-Rx primary)	Saves Rs. 21,000 / yr
Pharmacy Billing & Tally Labor	~2.5 hrs/day manual price/ledger work	Automated POS price memory & ledger	>75% time reduction
Over-Dispense / Billing Errors	2–4% unrecorded or mispriced items	<0.1% (Strict DB & BigDecimal guards)	Direct margin recovery

3.6.3 Operational Feasibility

Operational feasibility measures how naturally clinic receptionists, doctors, patients, and pharmacists adopt the system without disruptive retraining. IMS mirrors real clinical mental models: receptionists see an intuitive Today's Appointments queue and guided identity-verification checkboxes; doctors use clinic-formulary autocomplete to prescribe drugs in seconds; patients book visible time-slot pills just like familiar consumer portals; and pharmacists scan or paste a single 32-character code into a bold POS input bar where unit prices are remembered automatically after the first sale. Hence, operational friction is exceptionally low and user acceptance is high.

CHAPTER 4
SYSTEM DESIGN

4.1 Design Overview and Architectural Principles

System design translates the functional and non-functional requirements of Chapter 3 into concrete software structures, security pipelines, data flow topologies, object-oriented domain entities, and normalized relational database schemas. Five core architectural principles govern every class and SQL script inside Infeara Medical System:

- **1. Modular Monolith with Explicit Transaction Boundaries:** Domain logic is organized into cohesive vertical and horizontal packages (audit, common, config, domain, identity, notifications, security, web). Spring JPA's open-in-view anti-pattern is explicitly disabled (spring.jpa.open-in-view=false) so lazy-loading never leaks into Thymeleaf view rendering and every read/write operation executes inside a well-defined @Transactional service method.
- **2. Dual-Layer Defense-in-Depth Concurrency:** Critical clinical invariants are enforced both in Java domain services and in SQL Server DDL—for example, pessimistic row locking (UPDLOCK) plus filtered unique index UX_Appt_Slot on appointments, and Java case equality plus Latin1_General_100_BIN2 binary collation on prescription codes.
- **3. Session-Derived Ownership (Zero IDOR Surface):** Controllers never accept a raw patientId, clinicId, or pharmacyId to identify the acting user; ownership is resolved exclusively from the authenticated AppUserPrincipal in the server HTTP session.
- **4. Append-Only Clinical & Forensic History:** Finalized consultations cannot be mutated (only extended via ConsultationAddenda), merged patient records are preserved dormantly via merged_into_patient_id, and every sensitive action writes an immutable row to dbo.AuditLogs.
- **5. Asynchronous Side-Effect Decoupling via Transactional Outbox:** Domain events (registration verification, staff invites, booking confirmations, prescription issuance) write queued EmailLogs records inside the business transaction, decoupling user response latency from external SMTP server availability.

4.2 System Architecture

Figure 4.1 illustrates the six-tier Modular Monolith architecture of Infeara Medical System. Requests originating from desktop or tablet web browsers traverse the Spring Security 6 gateway filter chain (Tier 2), route into role-isolated Spring WebMVC controllers and Thymeleaf layout templates (Tier 3), invoke @Transactional domain services and event listeners (Tier 4), execute parameterized ORM and pessimistic-locking queries via Spring Data JPA / Hibernate 6 (Tier 5), and persist ACID state in Microsoft SQL Server 2022 alongside the @Scheduled EmailDispatchWorker (Tier 6).

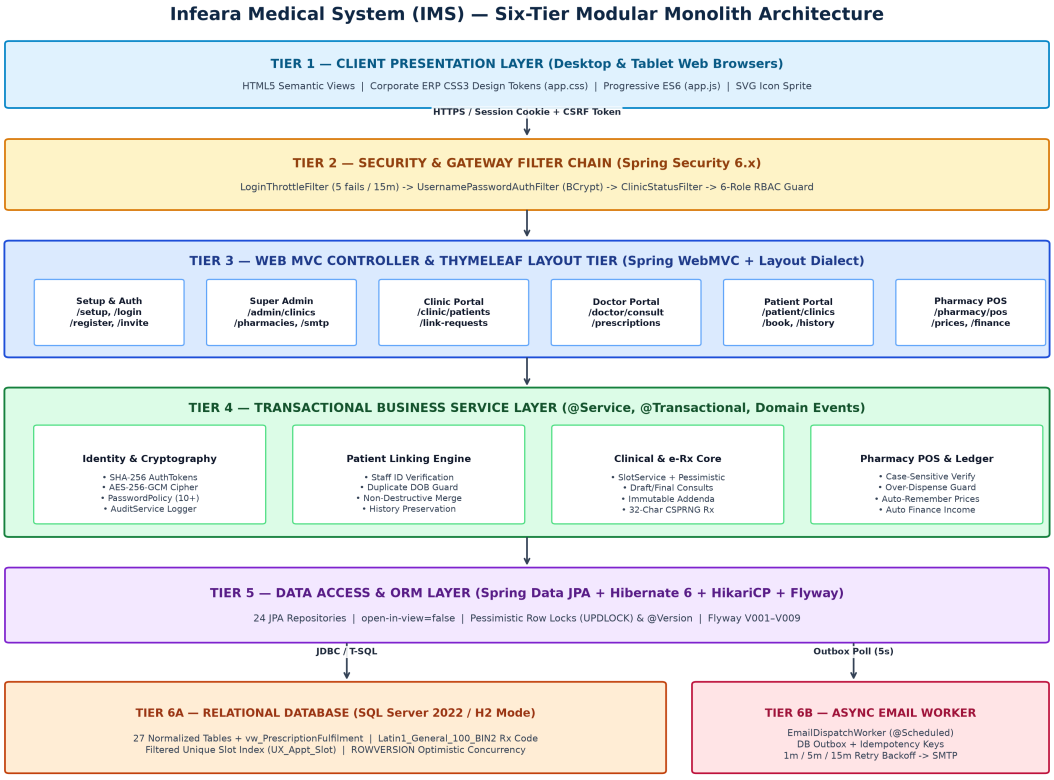


Figure 4.1: Infeara Medical System (IMS) Six-Tier Modular Monolith Architecture

Complementing the macro tier structure, Figure 4.2 details the defense-in-depth Spring Security filter pipeline and cryptographic services (LoginThrottleFilter, BCrypt authentication, ClinicStatusFilter, SHA-256 AuthTokens, AES-256-GCM SecretCipher, and AuditService).

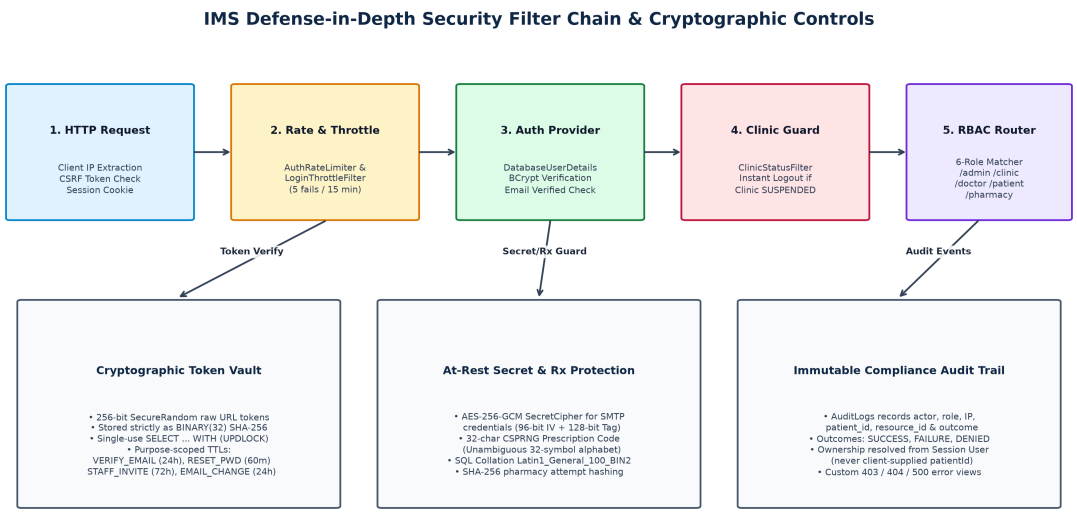


Figure 4.2: IMS Defense-in-Depth Security Filter Chain and Cryptographic Controls

4.3 Data Flow Diagrams (DFD)

Data Flow Diagrams model how clinical, administrative, cryptographic, and financial information enters, transforms within, and persists across the system boundary without coupling prematurely to object syntax.

4.3.1 Level 0 DFD (System Context Diagram)

At Level 0 (Figure 4.3), the entire Infeara Medical System is represented as central Process 0.0 exchanging structured request and response data streams with five external actor classes: Super Admin, Clinic Admin, Doctor, Patient, and Pharmacy Staff.

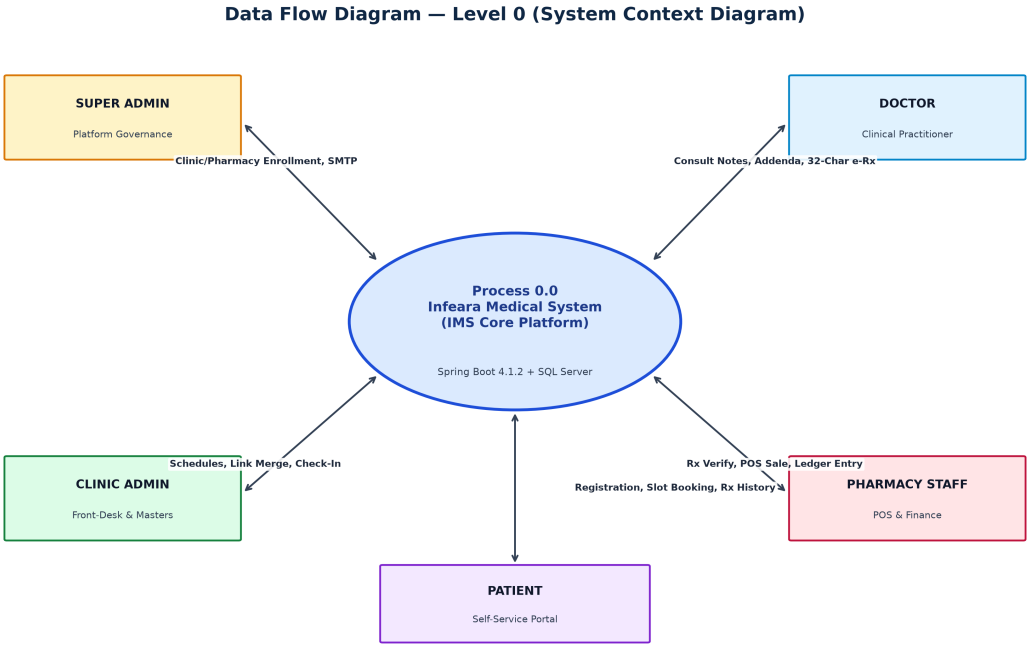


Figure 4.3: Data Flow Diagram — Level 0 (System Context Diagram)

4.3.2 Level 1 DFD (Subsystem Decomposition)

Decomposing Process 0.0 yields six major functional processes (Processes 1.0 through 6.0) interacting with five logical relational data stores (D1 through D5), as depicted in Figure 4.4: Identity, RBAC & Patient Linking (1.0), Clinic Discovery & Slot Booking (2.0), Consultation & Addenda Engine (3.0), 32-Character e-Prescription Issuance & Outbox (4.0), Pharmacy Verification & POS Billing (5.0), and Finance Ledger & Audit Reporting (6.0).

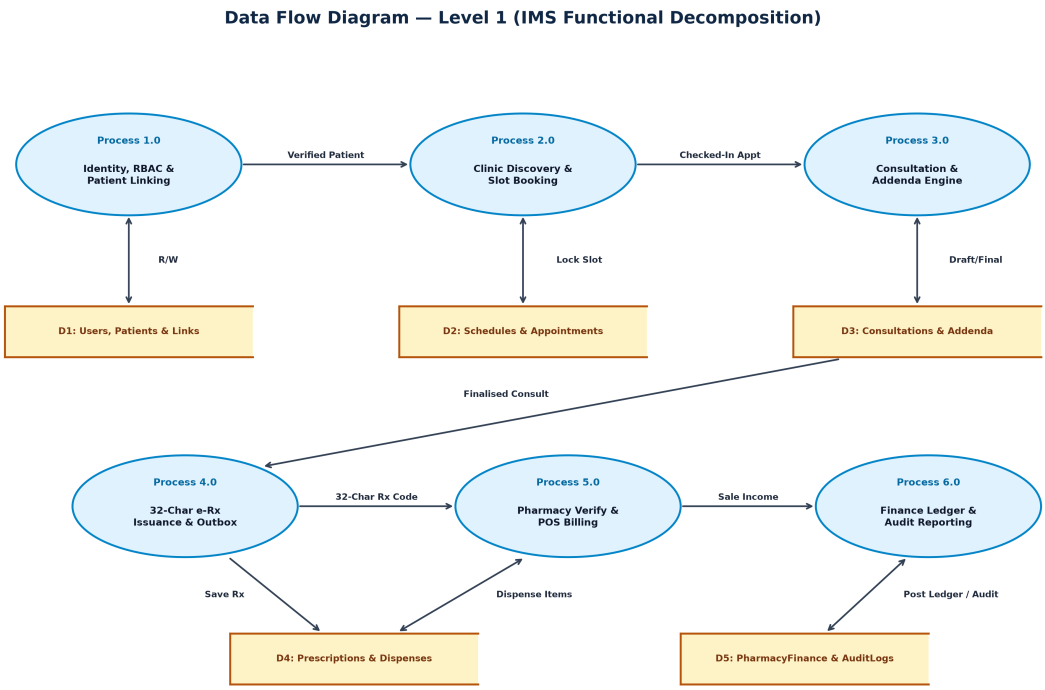


Figure 4.4: Data Flow Diagram — Level 1 (IMS Functional Decomposition)

4.3.3 Level 2 DFD (Pharmacy POS Verification, Pricing & Ledger Automation)

Because Process 5.0 encapsulates critical real-time prescription validation, partial-dispensing math, self-learning catalog pricing, and automated cashbook synchronization, Figure 4.5 expands Process 5.0 into five granular sub-processes (5.1 through 5.5).

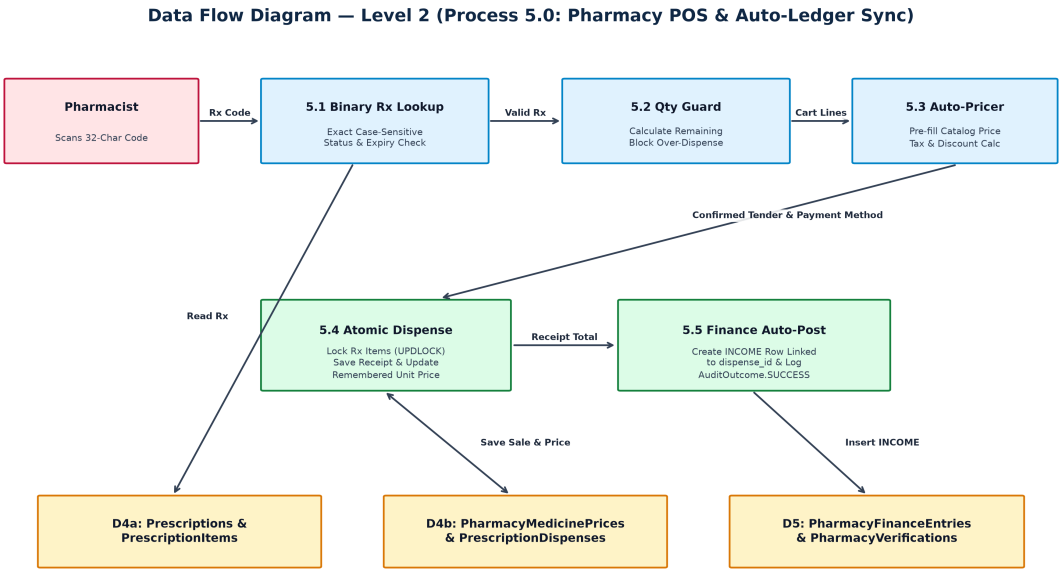


Figure 4.5: Data Flow Diagram — Level 2 (Pharmacy POS Dispensing and Automated Finance Sync)

4.4 Use Case Diagram and Actor Specifications

Figure 4.6 presents the Unified Modeling Language (UML) Use Case Diagram capturing thirteen core use cases (UC-01 through UC-13) across the six security roles defined in com.clinicflow.domain.enums.Role.

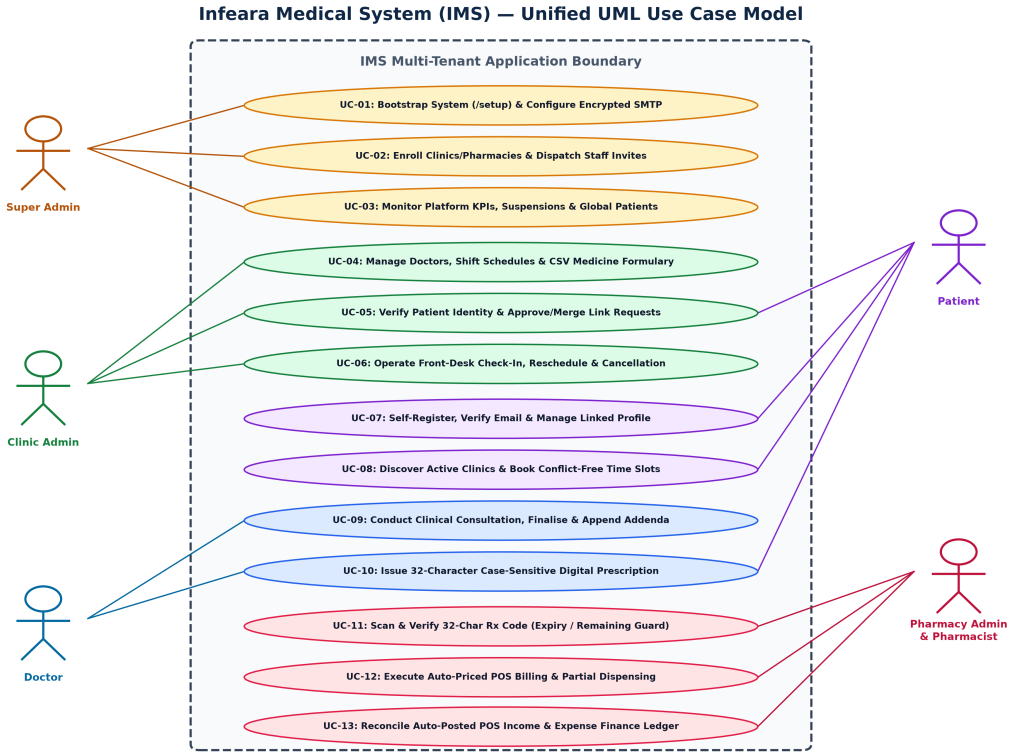


Figure 4.6: IMS Unified UML Use Case Diagram Across Five Actor Portals

Table 4.1: Detailed Actor-to-Use-Case Responsibility Specification

Actor / Role Enum	Portal Prefix	Authorized Use Cases & Domain Responsibilities
Super Admin (SUPER_ADMIN)	/admin/**	Bootstraps system via /setup; enrolls, activates, and suspends Clinics & Pharmacies; dispatches 72h admin invites; views global Patient Masters & KPI reports; manages AES-GCM encrypted SMTP settings.
Clinic Admin (CLINIC_ADMIN)	/clinic/**	Maintains clinic Patient Masters; verifies physical ID & DOB to approve/reject/merge PatientLinkRequests; manages Doctors & weekday DoctorSchedules; operates Appointment check-in desk; imports/exports CSV ClinicMedicines.
Doctor (DOCTOR)	/doctor/**	Views assigned daily appointment queue; transitions encounters to IN_CONSULTATION; authors DRAFT diagnoses/notes, finalizes consultations irreversibly, appends signed ConsultationAddenda, and issues 32-char e-Prescriptions.
Patient (PATIENT)	/patient/**	Self-registers & verifies email; edits profile & requests verified email changes; requests Clinic Links; discovers active clinics; books, reschedules, or cancels conflict-free slots; views 32-char prescriptions & longitudinal history.
Pharmacy Admin & Pharmacist (PHARMACY_ADMIN, PHARMACIST)	/pharmacy/**	Verifies 32-char prescription codes; executes POS cart billing with auto-remembered unit prices, discounts, taxes, and over-dispense protection; prints sale receipts; manages Finance Ledger, staff invites (Admin), and reports.

To formalize actor interactions at an engineering specification level, Tables 4.1a through 4.1d document the complete preconditions, main success scenarios, security exception flows, and transactional postconditions for the four highest-criticality workflows in Infeara Medical System.

Table 4.1a: Use Case Specification — UC-05: Staff-Verified Patient Link & Non-Destructive Record Merge

Specification Element	Engineering & Clinical Detail
Primary Actor & Trigger	CLINIC_ADMIN opens /clinic/link-requests/{id} after PATIENT submits link request.
Preconditions	Clinic is ACTIVE; PatientLinkRequest status is PENDING; Clinic Admin belongs to target clinic_id.
Main Success Scenario	1. System displays account patient's name, verified DOB, email, mobile & duplicate warnings. 2. Clinic Admin inspects physical photo ID at front desk and checks identityVerified=true. 3. Admin optionally enters walk-in public_patient_id (e.g., PAT-2026-0014) to merge. 4. Service verifies exact DOB match, re-points Appointments, Prescriptions & ClinicPatients to target row, moves User FK, and sets merged_into_patient_id on old row. 5. Status updates to APPROVED and AuditLogs records PATIENT_RECORD_MERGED + PATIENT_LINK_APPROVED.
Exception / Security Flows	● Missing identityVerified checkbox -> HTTP 200 form error; state stays PENDING. ● Target record DOB differs from account DOB -> Merge rejected immediately. ● Admin from another clinic requests URL -> HTTP 404 / tenant isolation enforced.
Postconditions	Single unified patient chart linked to online User login; zero orphaned foreign keys.

Table 4.1b: Use Case Specification — UC-08: Atomic Conflict-Free Doctor Slot Booking

Specification Element	Engineering & Clinical Detail
Primary Actor & Trigger	Authenticated PATIENT selects clinic, doctor, date, and 15-min slot on /patient/book.
Preconditions	Patient email verified; ClinicStatus == ACTIVE; DoctorStatus == ACTIVE; ClinicDoctor active.
Main Success Scenario	1. SlotService computes future slots from weekday DoctorSchedules minus active Appointments. 2. Patient submits POST /patient/book with valid CSRF token. 3. BookingService acquires pessimistic write lock (doctorRepository.findForUpdate). 4. Service re-verifies slot availability, inserts BOOKED Appointment under UX_Appt_Slot, assigns APT-YYYY-NNNNN, ensures ClinicPatient link, and queues confirmation email.
Exception / Security Flows	• Competing request took slot milliseconds earlier -> Lock serializes or UX_Appt_Slot throws DataIntegrityViolationException -> clean 'Slot just taken' message. • Past timestamp or unscheduled weekday -> Rejected before insert.
Postconditions	Exactly one active appointment occupies (doctor_id, slot_start); AuditLogs & EmailLogs updated.

Table 4.1c: Use Case Specification — UC-09 & UC-10: Immutable Clinical Consultation & 32-Char e-Rx

Specification Element	Engineering & Clinical Detail
Primary Actor & Trigger	Authenticated DOCTOR opens checked-in encounter at /doctor/appointments/{id}.
Preconditions	DoctorUserLink maps session user to appointment's doctor_id; Clinic is ACTIVE.
Main Success Scenario	1. Doctor clicks Start Consultation -> status moves to IN_CONSULTATION; DRAFT Consultation created. 2. Doctor enters diagnosis, vitals/notes, and follow-up date; clicks Finalise. 3. Consultation transitions irreversibly to FINAL (finalised_at stamped); Appointment marked COMPLETED. 4. Any subsequent note is appended via ConsultationAddenda with author & reason. 5. Doctor selects formulary medicines, sets validity days, and issues 32-char CSPRNG Prescription.
Exception / Security Flows	• Attempt to edit diagnosis after FINAL -> Blocked; only addenda permitted. • Duplicate prescription on same consultation -> Blocked (1-to-1 consultation Rx rule).
Postconditions	Immutable clinical record + VALID 32-char binary-collated Prescription + queued patient email.

Table 4.1d: Use Case Specification — UC-11, UC-12 & UC-13: Pharmacy POS Billing & Auto-Ledger Sync

Specification Element	Engineering & Clinical Detail
Primary Actor & Trigger	PHARMACY_ADMIN or PHARMACIST scans 32-char code on /pharmacy/pos.
Preconditions	Pharmacy status is ACTIVE; PharmacyUser status is ACTIVE.
Main Success Scenario	1. PharmacyService performs binary case-sensitive lookup, checks status/expiry/remaining units, and logs PharmacyVerification (outcome=VALID). 2. POS cart auto-populates remaining quantities and remembered unit prices from PharmacyMedicinePrices. 3. Pharmacist confirms quantities, discount, tax rate, payment method (Cash/UPI/Card), and tendered amount. 4. Service locks PrescriptionItems, increments quantity_dispensed, saves PrescriptionDispense receipt, updates remembered prices, and auto-posts INCOME to PharmacyFinanceEntries.
Exception / Security Flows	• Wrong case / unknown code -> NOT_FOUND logged with SHA-256 attempt_hash. • Requested qty > remaining -> Over-dispense blocked; transaction rolled back. • Reusing consumed verificationId -> Rejected (one verification per dispense).
Postconditions	Updated item fulfilment, printable SALE receipt, remembered catalog prices, and linked INCOME ledger row.

4.5 Object-Oriented & Behavioral UML Models (Class, Sequence, Activity & State)

Figure 4.7 illustrates the UML Domain Entity Class Diagram mapped directly to com.clinicflow.domain.entity.*. Entities encapsulate surrogate BIGINT keys, public UUIDs or formatted human-readable IDs (PAT-YYYY-NNNN, APT-YYYY-NNNNN), @Version byte[] optimistic row versions where concurrent edits occur, and strongly typed enumerated status fields.

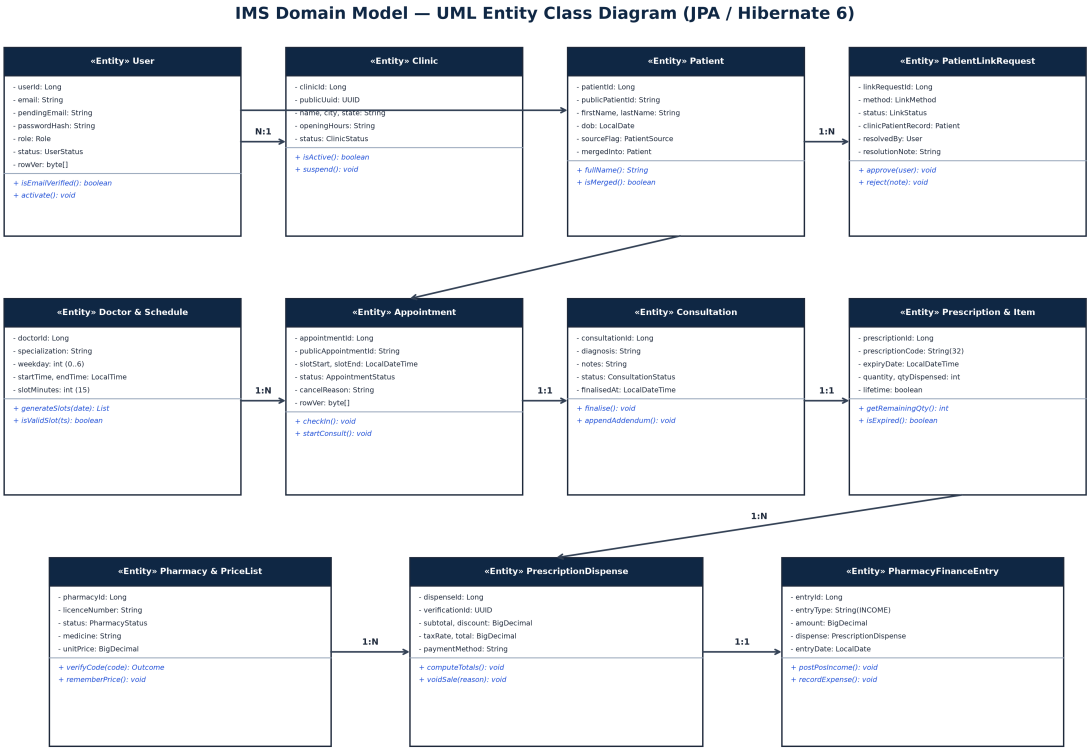


Figure 4.7: IMS Core Domain Entity UML Class Diagram (Spring Data JPA / Hibernate 6)

Figure 4.8 traces the fifteen-step temporal interaction sequence across Patient Browser, Doctor Workspace, Spring Boot Transactional Services, SQL Server 2022, and Pharmacy POS Terminal—spanning atomic slot reservation, clinical consultation finalization, 32-character prescription issuance, binary-collated POS lookup, itemized dispensing, price-list learning, and automated finance-ledger posting.

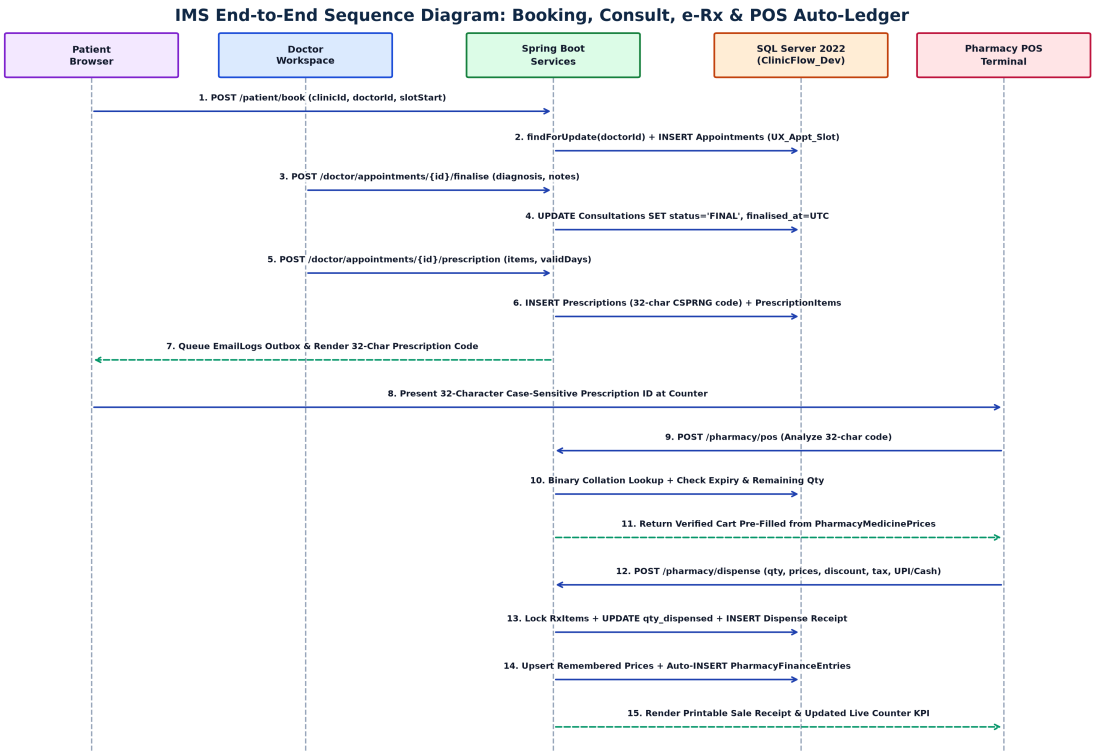


Figure 4.8: UML Sequence Diagram — End-to-End Clinical Consultation, e-Rx & Pharmacy POS Flow

Figure 4.9 models the branching control flow of patient registration, front-desk identity verification (including rejection vs. approval/record-merge branches), clinical consultation, prescription generation, and POS fulfillment, while Figure 4.10 formalizes the finite-state machines governing AppointmentStatus and Prescription / vw_PrescriptionFulfilment lifecycles.

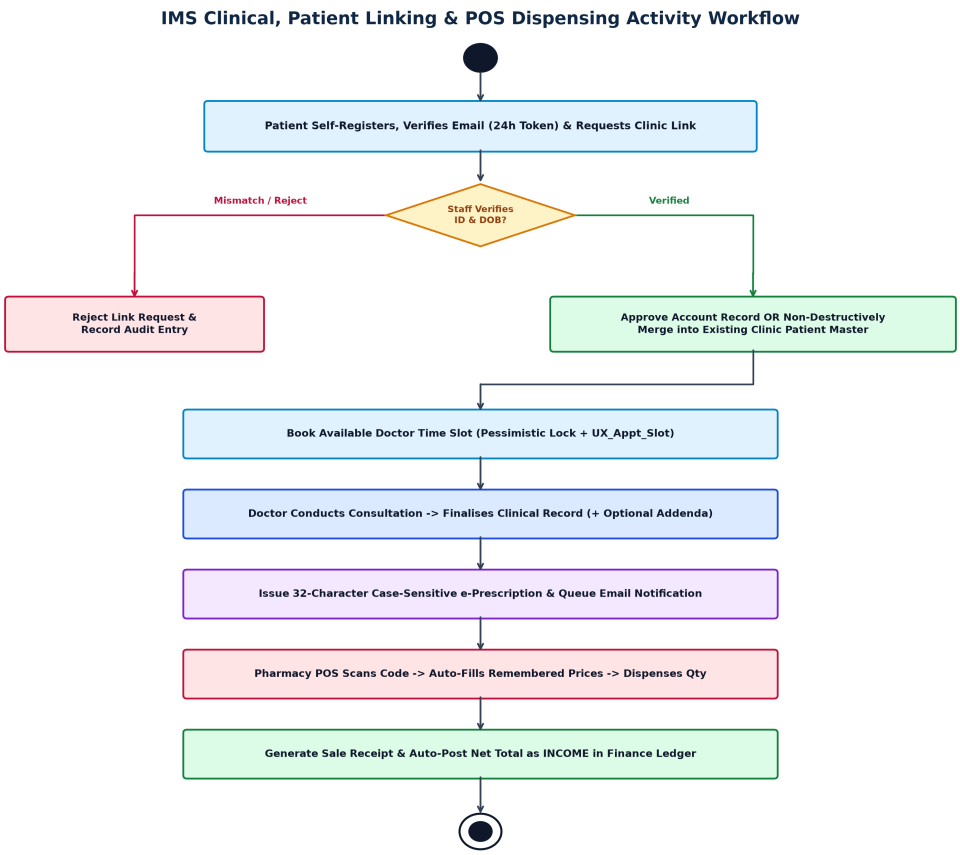


Figure 4.9: UML Activity Workflow Diagram — Patient Linking, Clinical Care & Dispensing

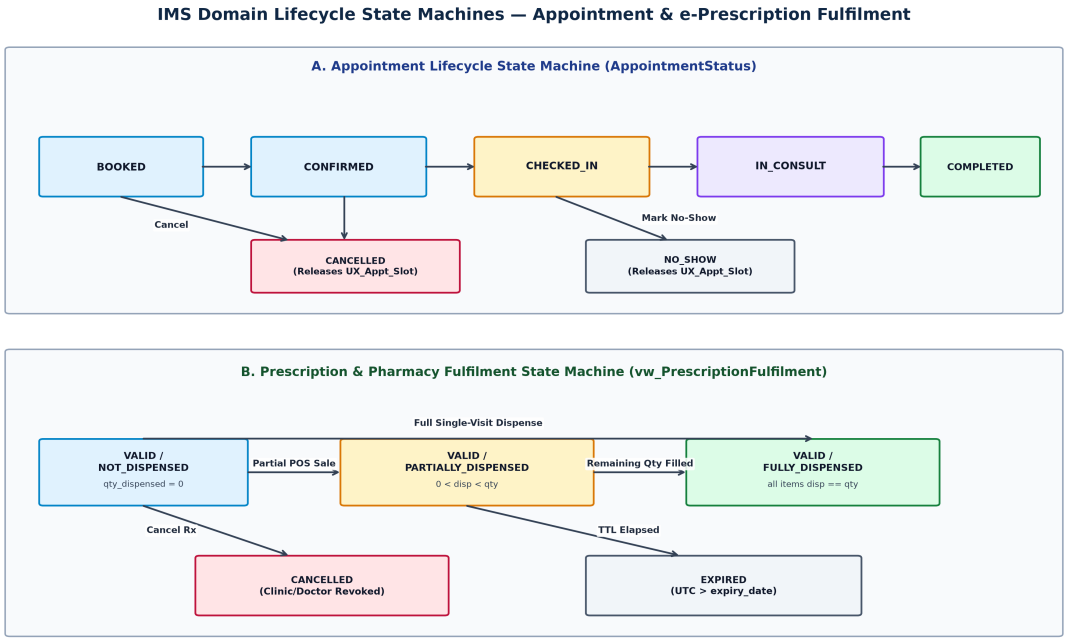


Figure 4.10: UML Lifecycle State Machine Diagram — AppointmentStatus & Prescription Fulfilment

4.6 Entity Relationship (ER) Diagram

Figure 4.11 depicts the complete relational Entity-Relationship architecture spanning all 27 normalized tables across Flyway migrations V001 through V009, organized cleanly into five functional domains: (1) Identity & Tenant Enrollment, (2) Patient Federation & Link Merging, (3) Clinical Scheduling & Consultations, (4) Electronic Prescriptions & Clinic Formulary, and (5) Pharmacy POS, Price Memory, Finance Ledger, Outbox & Audit.

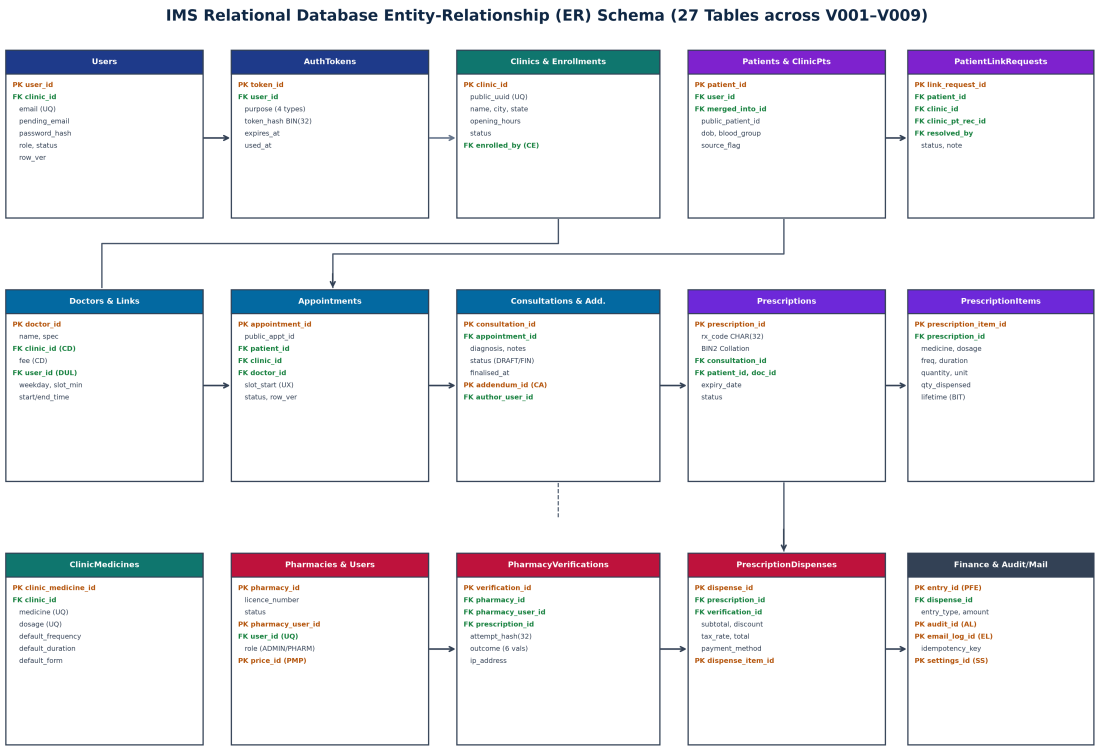


Figure 4.11: IMS Relational Database Entity-Relationship (ER) Schema (27 Tables, V001–V009)

4.7 Complete Relational Database Design and Data Dictionary

The persistence schema of Infeara Medical System is governed deterministically by nine sequential Flyway SQL migration scripts located in clinicflow-app/src/main/resources/db/migration/ (V001 through V009). All tables reside in the dbo schema on Microsoft SQL Server 2022, utilizing BIGINT IDENTITY(1,1) surrogate primary keys for fast internal joins alongside UNIQUEIDENTIFIER (NEWID()) or formatted public identifiers so sequential primary keys are never exposed in public URLs. Timestamps standardize on DATETIME2(3) in UTC (SYSUTCDATETIME()), monetary figures use exact fixed-point DECIMAL(12,2) or DECIMAL(18,2) arithmetic, and concurrency-critical tables include ROWVERSION columns.

Table 4.2: Flyway Database Migration History (V001 through V009)

Version	Migration Script Filename	Schema Objects Created / Altered
V001	V001__init_schema.sql	Creates 24 core tables + SQL view dbo.vw_PrescriptionFulfilment + filtered unique index UX_Appt_Slot + BIN2 Rx collation.
V002	V002__identity_email_outbox.sql	Adds subject, body, next_attempt_at to EmailLogs; creates UX_AuthTokens_token_hash and IX_EmailLogs_status_next_attempt.
V003	V003__patient_link_request_resolution.sql	Adds clinic_patient_record_id, resolved_by, and resolution_note FKs/columns to PatientLinkRequests.
V004	V004__invites_contact_change_and_merge_marker.sql	Adds Users.pending_email, Patients.merged_into_patient_id self-FK, and expands AuthTokens purposes to 4 types.
V005	V005__clinic_inventory.sql	Creates dbo.ClinicMedicines table for standardized clinic drug formularies with unique (clinic_id, medicine, dosage).
V006	V006__rename_inventory_unit_to_form.sql	Renames ClinicMedicines.default_unit to default_form to accurately reflect dosage form (Tablet, Capsule, Syrup).
V007	V007__prescription_item_lifetime.sql	Adds lifetime BIT NOT NULL DEFAULT 0 column to dbo.PrescriptionItems for chronic/lifetime therapies.
V008	V008__pharmacy_finance.sql	Creates dbo.PharmacyFinanceEntries ledger table with INCOME / EXPENSE check constraint and date index.
V009	V009__pharmacy_pos_pricing.sql	Creates dbo.PharmacyMedicinePrices; adds subtotal, discount, tax, total, payment columns to Dispenses; links Finance to dispense_id.

Table 4.3: Data Dictionary — dbo.Clinics (Multi-Tenant Healthcare Facilities)

Column / Object	SQL Data Type	Constraints & Indexes	Clinical / Technical Purpose
clinic_id	BIGINT IDENTITY(1,1)	PK, NOT NULL	Internal surrogate primary key for the clinic tenant.
public_uuid	UNIQUEIDENTIFIER	UQ, DEFAULT NEWID()	Public opaque UUID exposed in patient clinic-discovery URLs.
name	NVARCHAR(200)	NOT NULL	Official display name of the enrolled clinic or polyclinic.
address_line1 / line2	NVARCHAR(200)	line1 NOT NULL	Street address and suite/landmark details.
city / area / district	NVARCHAR(100)	city NOT NULL, IX	Indexed city, locality, and district for patient clinic search.
state / postal_code	NVARCHAR(100 / 20)	NOT NULL	State and postal PIN code.
latitude / longitude	DECIMAL(9,6)	NULL	Optional geographic coordinates for proximity discovery.
phone / email	NVARCHAR(30 / 254)	NULL	Public reception phone number and contact email.
opening_hours / fees	NVARCHAR(500 / 200)	NULL	Published operating schedule and baseline consultation fee text.
status	VARCHAR(12)	CK IN (PENDING, ACTIVE,SUSPENDED)	Lifecycle state enforced on every request by ClinicStatusFilter.
created_at / updated_at	DATETIME2(3)	NOT NULL, UTC	Audit creation and last-modification UTC timestamps.

Table 4.4: Data Dictionary — dbo.Users (Unified Security Principals)

Column / Object	SQL Data Type	Constraints & Indexes	Clinical / Technical Purpose
user_id	BIGINT IDENTITY(1,1)	PK, NOT NULL	Internal surrogate key referenced by sessions and audit logs.
public_uuid	UNIQUEIDENTIFIER	UQ, DEFAULT NEWID()	External immutable UUID for user principal.
email	NVARCHAR(254)	UQ, NOT NULL	Primary verified login email address (unique across platform).
pending_email	NVARCHAR(254)	NULL (Added V004)	Unverified target email address awaiting EMAIL_CHANGE token confirmation.
password_hash	NVARCHAR(255)	NOT NULL	DelegatingPasswordEncoder BCrypt hash ({bcrypt}$2a$...).
role	VARCHAR(20)	CK (6 Roles), IX	SUPER_ADMIN, CLINIC_ADMIN, DOCTOR, PATIENT, PHARMACY_ADMIN, PHARMACIST.
status	VARCHAR(12)	CK (4 States)	PENDING, ACTIVE, SUSPENDED, or DISABLED.
email_verified_at	DATETIME2(3)	NULL	UTC timestamp set when VERIFY_EMAIL or STAFF_INVITE token is consumed.
clinic_id	BIGINT	FK -> Clinics, NULL	Tenant clinic binding for CLINIC_ADMIN and DOCTOR accounts.
row_ver	ROWVERSION	Optimistic Lock	SQL Server automatic binary counter preventing lost concurrent updates.

Table 4.5: Data Dictionary — dbo.Patients (Master & Self-Registered Patient Profiles)

Column / Object	SQL Data Type	Constraints & Indexes	Clinical / Technical Purpose
patient_id	BIGINT IDENTITY(1,1)	PK, NOT NULL	Surrogate key for clinical foreign-key relationships.
public_patient_id	NVARCHAR(20)	UQ, NOT NULL	Formatted identifier PAT-YYYY-NNNN generated via PublicIdGenerator.
user_id	BIGINT	FK -> Users, NULL, IX	Linked online User account (NULL for unlinked walk-in clinic records).
merged_into_patient_id	BIGINT	Self-FK -> Patients (V004)	Non-NULL when this dormant row was merged into a canonical clinic record.
first_name / last_name	NVARCHAR(100)	NOT NULL	Verified legal given name and family surname.
dob	DATE	NOT NULL	Date of birth used as mandatory anti-mismatch guard during link merges.
gender	VARCHAR(10)	CK (MALE,FEMALE,OTHER,UNKNOWN)	Optional self-reported or clinic-recorded gender.
mobile / email	NVARCHAR(30 / 254)	NULL	Patient contact phone and notification address.
address_line1..postal_code	NVARCHAR	NULL	Residential street, city, district, state, and PIN code.
blood_group	NVARCHAR(10)	NULL	ABO/Rh blood group displayed in Doctor consultation sidebar.
allergies / conditions	NVARCHAR(1000)	NULL	Drug/food allergies and chronic medical conditions.
emergency_name / phone	NVARCHAR(200 / 30)	NULL	Emergency next-of-kin contact details.
source_flag	VARCHAR(20)	CK (PATIENT_ENTERED, CLINIC_ENTERED)	Distinguishes online self-registrations from front-desk walk-in charts.
row_ver	ROWVERSION	Optimistic Lock	Prevents concurrent profile overwrite anomalies.

Table 4.6: Data Dictionary — dbo.Doctors, dbo.ClinicDoctors, dbo.DoctorSchedules & dbo.DoctorUserLinks

Column / Object	SQL Data Type	Constraints & Indexes	Clinical / Technical Purpose
Doctors.doctor_id	BIGINT IDENTITY(1,1)	PK, NOT NULL	Primary key of practitioner master entity (public_uuid, name, specialization, status).
ClinicDoctors.clinic_doctor_id	BIGINT IDENTITY(1,1)	PK, UQ(clinic_id,doctor_id)	Associates a Doctor with a Clinic, consultation fee DECIMAL(10,2), and ACTIVE/INACTIVE status.
DoctorSchedules.schedule_id	BIGINT IDENTITY(1,1)	PK, IX(doctor_id,clinic_id)	Defines weekly consulting window: weekday (0..6), start_time, end_time, slot_minutes (default 15), effective_from/to.
DoctorUserLinks.link_id	BIGINT IDENTITY(1,1)	PK, UQ(user_id)	Binds a DOCTOR login account (Users.user_id) 1-to-1 to their practitioner profile (Doctors.doctor_id).

Table 4.7: Data Dictionary — dbo.Appointments (Conflict-Free Slot Reservations)

Column / Object	SQL Data Type	Constraints & Indexes	Clinical / Technical Purpose
appointment_id	BIGINT IDENTITY(1,1)	PK, NOT NULL	Internal surrogate key for appointment encounter.
public_appointment_id	NVARCHAR(20)	UQ, NOT NULL	Human-readable booking code APT-YYYY-NNNNN.
patient_id / clinic_id / doctor_id	BIGINT	FKs, NOT NULL, IX	References Patient, Clinic, and Doctor; indexed for queue & slot lookups.
slot_start / slot_end	DATETIME2(3)	NOT NULL, UX_Appt_Slot	Filtered unique index UX_Appt_Slot(doctor_id, slot_start) on active statuses prevents double-booking.
reason	NVARCHAR(500)	NULL	Chief complaint or visit reason entered during booking.
status	VARCHAR(20)	CK (7 States)	BOOKED, CONFIRMED, CHECKED_IN, IN_CONSULTATION, COMPLETED, CANCELLED, NO_SHOW.
booked_at / cancelled_at	DATETIME2(3)	UTC	Booking creation and optional cancellation timestamps + cancel_reason NVARCHAR(300).
row_ver	ROWVERSION	Optimistic Lock	Protects concurrent status transitions between front desk and doctor.

Table 4.8: Data Dictionary — dbo.Consultations & dbo.ConsultationAddenda (Clinical Records)

Column / Object	SQL Data Type	Constraints & Indexes	Clinical / Technical Purpose
Consultations.consultation_id	BIGINT IDENTITY(1,1)	PK, NOT NULL	Primary key of clinical encounter note.
Consultations.appointment_id	BIGINT	UQ, FK -> Appointments	Strict 1-to-1 link ensuring at most one consultation record per appointment.
Consultations.diagnosis	NVARCHAR(2000)	NULL	Primary clinical diagnosis recorded by consulting physician.
Consultations.notes	NVARCHAR(MAX)	NULL	Detailed examination findings, vitals, and care instructions.
Consultations.follow_up	DATETIME2(3)	NULL	Recommended clinical review/follow-up date.
Consultations.status / finalised_at	VARCHAR(10) / DATETIME2(3)	CK (DRAFT, FINAL)	Once set to FINAL, diagnosis and notes become permanently immutable.
ConsultationAddenda.addendum_id	BIGINT IDENTITY(1,1)	PK, FK -> Consultations	Append-only post-finalization clinical note storing author_user_id, reason NVARCHAR(300), text NVARCHAR(MAX), created_at.

Table 4.9: Data Dictionary — dbo.Prescriptions, dbo.PrescriptionItems & dbo.ClinicMedicines

Column / Object	SQL Data Type	Constraints & Indexes	Clinical / Technical Purpose
Prescriptions.prescription_id	BIGINT IDENTITY(1,1)	PK, NOT NULL	Internal primary key of electronic prescription header.
Prescriptions.prescription_code	CHAR(32) BIN2	UQ, CK Alphanumeric	32-char CSPRNG code under Latin1_General_100_BIN2 case-sensitive binary collation.
Prescriptions.consultation_id	BIGINT	FK -> Consultations	Links prescription to its originating clinical consultation, patient_id, clinic_id, and doctor_id.
Prescriptions.issue_date / expiry_date	DATETIME2(3)	NOT NULL, UTC	Issuance timestamp and mandatory expiration boundary enforced by PharmacyService.
Prescriptions.status	VARCHAR(10)	CK (VALID, CANCELLED)	Allows clinic/doctor revocation with cancel_reason, cancelled_at, and cancelled_by.
PrescriptionItems.prescription_item_id	BIGINT IDENTITY(1,1)	PK, FK -> Prescriptions	Line item storing medicine, dosage, frequency, duration, instructions, and ROWVERSION.
PrescriptionItems.quantity / dispensed	INT	CK (0 <= disp <= qty)	Prescribed units (>0) and cumulative quantity_dispensed across partial pharmacy fills.
PrescriptionItems.lifetime	BIT	NOT NULL DEFAULT 0 (V007)	Marks chronic/lifetime maintenance medications in patient history.
ClinicMedicines.clinic_medicine_id	BIGINT IDENTITY(1,1)	PK, UQ(clinic,med, dosage)	Clinic formulary (V005/V006): medicine, dosage, default_frequency, default_duration, default_quantity, default_form, default_instructions.

Table 4.10: Data Dictionary — dbo.Pharmacies, dbo.PharmacyUsers, dbo.PharmacyVerifications & dbo.PharmacyMedicinePrices

Column / Object	SQL Data Type	Constraints & Indexes	Clinical / Technical Purpose
Pharmacies.pharmacy_id	BIGINT IDENTITY(1,1)	PK, UQ(licence_number)	Enrolled retail pharmacy tenant with public_uuid, name, licence_number, address, status (PENDING/ACTIVE/SUSPENDED).
PharmacyUsers.pharmacy_user_id	BIGINT IDENTITY(1,1)	PK, UQ(user_id)	Maps Users account to a Pharmacy with role (PHARMACY_ADMIN, PHARMACIST) and status (ACTIVE, DISABLED).
PharmacyVerifications.verification_id	UNIQUEIDENTIFIER	PK DEFAULT NEWID()	Immutable verification token & log storing pharmacy_id, pharmacy_user_id, prescription_id, SHA-256 attempt_hash BINARY(32), outcome (6 values), verified_at, ip_address.
PharmacyMedicinePrices.price_id	BIGINT IDENTITY(1,1)	PK, UQ(pharmacy_id,medicine)	Self-learning per-pharmacy price catalog (V009) storing unit_price DECIMAL(12,2) >= 0 and updated_at.

Table 4.11: Data Dictionary — dbo.PrescriptionDispenses, dbo.PrescriptionDispenseItems & dbo.PharmacyFinanceEntries

Column / Object	SQL Data Type	Constraints & Indexes	Clinical / Technical Purpose
PrescriptionDispenses.dispense_id	BIGINT IDENTITY(1,1)	PK, UQ(public_uuid)	POS sale & dispense receipt linked to prescription_id, pharmacy_id, pharmacy_user_id, and single-use verification_id.
subtotal / discount / tax_amount / total	DECIMAL(12,2)	NOT NULL, CK >= 0 (V009)	Exact BigDecimal monetary totals plus tax_rate DECIMAL(5,2), payment_method, amount_tendered, and change_due.
status / void metadata	VARCHAR(10)	CK (COMPLETED, VOIDED)	Enforces voided_at, voided_by, and void_reason whenever status='VOIDED'.
PrescriptionDispenseItems.dispense_item_id	BIGINT IDENTITY(1,1)	PK, UQ(dispense,rx_item)	Records quantity_dispensed (>0), unit_price DECIMAL(12,2), and line_total DECIMAL(12,2) per medicine line.
PharmacyFinanceEntries.entry_id	BIGINT IDENTITY(1,1)	PK, IX(pharmacy_id,date)	Accounting ledger (V008/V009): entry_type (INCOME/EXPENSE), amount DECIMAL(18,2), description, entry_date, created_by, and FK dispense_id.

Table 4.12: Data Dictionary — Enrollment, Patient Linking, AuthTokens, Outbox, SMTP & Audit Tables

Column / Object	SQL Data Type	Constraints & Indexes	Clinical / Technical Purpose
ClinicEnrollments.enrollment_id	BIGINT IDENTITY(1,1)	PK, FK -> Clinics, Users	Tracks clinic onboarding by SUPER_ADMIN, invite_token_hash BINARY(32), status, enrolled_at, activated_at.
ClinicPatients.clinic_patient_id	BIGINT IDENTITY(1,1)	PK, UQ(clinic_id,patient_id)	Many-to-many link authorizing a clinic to view/manage a linked patient chart.
PatientLinkRequests.link_request_id	BIGINT IDENTITY(1,1)	PK, IX(clinic,status)	Stores patient_id, clinic_id, method (CLINIC_APPROVAL/OTP/CODE), status (PENDING/APPROVED/REJECTED/EXPIRED), clinic_patient_record_id, resolved_by, resolution_note.
AuthTokens.token_id	BIGINT IDENTITY(1,1)	PK, UX(token_hash)	Single-use token vault: user_id, purpose (VERIFY_EMAIL, RESET_PASSWORD, STAFF_INVITE, EMAIL_CHANGE), SHA-256 token_hash BINARY(32), expires_at, used_at.
EmailLogs.email_log_id	BIGINT IDENTITY(1,1)	PK, UQ(idempotency_key)	Transactional email outbox (V001/V002): type, recipient, subject, body, status (QUEUED/SENT/FAILED), attempts, next_attempt_at, last_error, sent_at.
SmtpSettings.settings_id	BIGINT IDENTITY(1,1)	PK	Platform SMTP relay config: host, port, security (NONE/STARTTLS/SSL), username, AES-GCM password_encrypted VARBINARY(512), from_address, active, last_test_result.
AuditLogs.audit_id	BIGINT IDENTITY(1,1)	PK, IX(patient_id,actor)	Forensic security & clinical log: actor_user_id, role, action, resource_type, resource_id, patient_id, ip_address, outcome (SUCCESS/FAILURE/DENIED), created_at.

4.8 Input Design, Output Design and Corporate ERP UI Design System

To eliminate clerical errors across high-volume reception desks, consulting rooms, and pharmacy counters, IMS implements a unified server-rendered design system defined in static/css/app.css, templates/layout/base.html, and templates/layout/auth.html.

- **Input Design & Dual-Layer Validation:** Every HTML form binds to a dedicated Jakarta Bean Validation DTO (such as RegistrationForm, ClinicEnrollmentForm, PatientProfileForm, BookingForm, ConsultationForm, PrescriptionForm, and LinkApprovalForm). Inputs group related clinical fields logically, preserve user entries on validation errors, trim whitespace server-side, attach automatic Spring Security _csrf hidden tokens, and use explicit confirmation checkboxes on irreversible actions such as patient record merges and consultation finalization.
- **Output Design & Clinical Readability:** Outputs combine top-of-page KPI metric cards (KpiCard record), semantic color-coded status pills (ACTIVE/VALID in #E3F4EA green, PENDING/PARTIAL in #FFF1DB amber, SUSPENDED/CANCELLED in #FBE7E5 red), monospace high-contrast 32-character prescription banners, printable POS dispense receipts showing itemized unit prices, discounts, tax, tendered cash, and change due, and exportable RFC-4180 CSV formulary files.
- **Consistent ERP Navigation & Breadcrumb Engine:** NavigationService dynamically computes role-specific left-sidebar items and hierarchical breadcrumbs (Home / Parent Module / Current Detail) alongside a sticky 56 px #0F2744 top bar displaying the active organization name, role badge, user avatar chip, and one-click session sign-out.

Complementing the relational schema, all state machines, roles, outcomes, and security modes in com.clinicflow.domain.enums are modeled as strongly typed Java enums mapped via @Enumerated(EnumType.STRING) and enforced at the database layer via matching SQL CHECK constraints (Table 4.13).

Table 4.13: Complete Java Domain Enumeration Reference (com.clinicflow.domain.enums.*)

Java Enum Class	Permitted Constants	Governing Entity / Table
Role	SUPER_ADMIN, CLINIC_ADMIN, DOCTOR, PATIENT, PHARMACY_ADMIN, PHARMACIST	Users.role / SecurityFilterChain
UserStatus	PENDING, ACTIVE, LOCKED, DISABLED	Users.status
ClinicStatus	PENDING_ENROLLMENT, ACTIVE, SUSPENDED, REJECTED	Clinics.status
DoctorStatus	ACTIVE, INACTIVE	Doctors.status
PatientSource	SELF, CLINIC	Patients.created_source
Gender	MALE, FEMALE, OTHER, UNKNOWN	Patients.gender
AppointmentStatus	BOOKED, CONFIRMED, IN_CONSULTATION, COMPLETED, CANCELLED, NO_SHOW	Appointments.status
ConsultationStatus	DRAFT, FINAL	Consultations.status
PrescriptionStatus	VALID, CANCELLED	Prescriptions.status
PharmacyStatus	PENDING, ACTIVE, SUSPENDED	Pharmacies.status
PharmacyUserRole	PHARMACY_ADMIN, PHARMACIST	PharmacyUsers.role_in_pharmacy
PharmacyUserStatus	PENDING, ACTIVE, DISABLED	PharmacyUsers.status
VerificationOutcome	VALID, NOT_FOUND, CANCELLED, EXPIRED, FULLY_DISPENSED	PharmacyVerifications.outcome
DispenseStatus	PARTIAL, FULL	PrescriptionDispenses.dispense_status
AuthTokenPurpose	VERIFY_EMAIL, RESET_PASSWORD, STAFF_INVITE, EMAIL_CHANGE	AuthTokens.purpose
EmailStatus	QUEUED, SENT, FAILED	EmailLogs.status
SmtpSecurity	NONE, STARTTLS, SSL_TLS	SmtpSettings.security_mode
EnrollmentStatus	SUBMITTED, APPROVED, REJECTED	ClinicEnrollments.status
LinkRequestMethod	PATIENT_PORTAL, CLINIC_DESK, EMAIL_MATCH	PatientLinkRequests.method
LinkRequestStatus	PENDING, APPROVED, REJECTED, EXPIRED	PatientLinkRequests.status
AuditOutcome	SUCCESS, DENIED, FAILURE	AuditLogs.outcome

Table 4.14 documents the Corporate ERP visual design tokens engineered in src/main/resources/static/css/app.css (1,882 lines) governing layout geometry,

contrast-verified healthcare color semantics, and typography across all 61 Thymeleaf templates.

Table 4.14: Corporate ERP User Interface Design System Tokens (static/css/app.css)

CSS Custom Property	Token Value	UI Architectural Application
--c-primary / --c-primary-hover	#1E4E8C / #163B6B	Primary medical navy buttons, active links, focus rings & table accents
--c-sidebar-bg / --c-sidebar-active	#0F2744 / #1E4E8C	Fixed dark-navy left ERP navigation rail & active route indicator
--c-accent / --c-accent-soft	#0E8A8A / #E6F4F4	Clinical teal accent badges, secondary highlights & active icons
--c-bg / --c-surface / --c-surface-muted	#F4F6F9 / #FFFFFF / #F8FAFC	App workspace canvas, elevated white cards & zebra table headers
--c-success / --c-warning / --c-danger	#157F3B / #B45309 / #B91C1C	WCAG AA status pills (ACTIVE/VALID, PENDING/PARTIAL, CANCELLED/EXPIRED)
--font-sans / --font-mono	Inter, Segoe UI / Cascadia Mono, Consolas	Crisp UI typography + unambiguous monospace 32-char Rx & public IDs
--topbar-h / --sidebar-w / --subbar-h	56px / 256px / 40px	Fixed ERP shell dimensions ensuring stable zero-shift workspace

CHAPTER 5
TECHNOLOGIES & TOOLS USED

5.1 Software Requirements

Infeara Medical System (IMS) is engineered on a modern, zero-licensing-cost enterprise Java software stack configured via clinicflow-app/pom.xml and clinicflow-app/src/main/resources/application.yml. Table 5.1 specifies the exact versions and roles of every software component required across development, automated testing, and production deployment.

Table 5.1: Software Requirements Specification for Infeara Medical System (IMS)

Layer / Category	Software / Library	Version in Repo	Role in IMS Architecture
Operating System	Windows 10/11 Pro, Windows Server, or Linux x86_64	64-bit OS	Host OS for JVM, Maven build tools, and SQL Server 2022.
Language Runtime	Java SE Development Kit (OpenJDK)	Java 27	Executes Spring Boot application with records & switch expressions.
Application Framework	Spring Boot Starter Parent	4.1.2	Auto-configuration, embedded Tomcat HTTP server, Actuator health.
Web & View Engine	Spring WebMVC + Thymeleaf + Layout Dialect	4.1.2 / 4.0.1	Server-rendered HTML5 views, reusable base.html & auth.html layouts.
Security Framework	Spring Security + Thymeleaf Security6 Extras	6.x / 3.1.5	Session authentication, BCrypt, CSRF tokens, 6-role URL guards.
Persistence & ORM	Spring Data JPA + Hibernate ORM + HikariCP	6.x	Entity mapping for 23 JPA entities, pessimistic locks, connection pool.
Schema Migration	Flyway Core + Flyway SQL Server	Managed	Deterministic execution of SQL migrations V001 through V009.
Primary RDBMS	Microsoft SQL Server 2022 Express	16.x (2022)	ACID persistence, BIN2 collation, filtered unique index, ROWVERSION.
Zero-Setup Test DB	H2 Database Engine (MODE=MSSQLServer)	Runtime	In-memory / file-backed local run and MockMvc integration tests.
Email Subsystem	Spring Boot Starter Mail (Jakarta Mail)	4.1.2	STARTTLS/SSL SMTP dispatch driven by transactional EmailLogs outbox.
Build & Dependency	Apache Maven (Bundled in tools/)	3.9.11	Lifecycle compilation, Lombok annotation processing, JUnit runner.
Testing Stack	JUnit Jupiter 5, Mockito, AssertJ, Spring Security Test	Managed	Unit, service-ownership, and full HTTP MockMvc security verification.

5.2 Hardware Requirements

Because IMS compiles to a self-contained Spring Boot application with server-side Thymeleaf rendering and efficient HikariCP database connection pooling, hardware resource demands remain modest across developer machines, clinic/cloud servers, and client workstations, as detailed in Table 5.2.

Table 5.2: Minimum and Recommended Hardware Specifications

Deployment Tier	Processor (CPU)	Memory (RAM)	Storage & Network
Developer Workstation	Intel Core i5 / AMD Ryzen 5 (4+ cores, 2.5 GHz+)	8 GB min (16 GB recommended)	20 GB SSD free space for JDK, Maven cache & SQL Express
Production Application + DB Server	4 vCPU x86_64 Server Processor	8 GB – 16 GB ECC / Cloud RAM	50 GB NVMe SSD (RAID/Snapshot backed), 100 Mbps NIC
Clinic / Doctor Client PC or Tablet	Dual-core 1.8 GHz+ (Desktop, Laptop, or iPad/Tablet)	4 GB RAM	1366x768+ display, modern browser (Chrome/Edge/Firefox)
Pharmacy POS Counter Terminal	Dual-core x86 PC + Optional USB HID 1D/2D Scanner	4 GB RAM	Thermal/Laser receipt printer + broadband/LAN link

5.3 Programming Languages Used

5.3.1 Java 27 (Backend Domain, Security, and Web Layer)

All 189 production source files and 8 test suites in clinicflow-app are written in modern Java (configured at <java.version>27</java.version> in pom.xml). Modern Java language constructs materially improve clarity and safety throughout the codebase: immutable Java records (such as KpiCard, ActivityRow, NavItem, Breadcrumb, IssuedToken, EmailContent, SlotOption, and domain event payloads) eliminate mutable DTO boilerplate; exhaustive switch expressions (used in NavigationService, RoleHomeResolver, AuthTokenService, and EmailDeliveryService) guarantee at compile time that every enum constant of Role or AuthTokenPurpose is handled; and pattern matching for instanceof simplifies principal inspection in ClinicStatusFilter.

5.3.2 Transact-SQL (T-SQL for Microsoft SQL Server 2022)

Database DDL across V001 through V009 leverages rich Transact-SQL capabilities unavailable in generic auto-generated ORM schemas: explicit named CHECK constraints on every status and monetary column, Latin1_General_100_BIN2 binary code-point collation on Prescriptions.prescription_code to enforce strict case sensitivity at the index level, filtered unique indexes (WHERE status IN (...) and WHERE dispense_id IS NOT NULL) to combine business flexibility with relational uniqueness, ROWVERSION columns for hardware-fast optimistic concurrency, and aggregated SQL views (dbo.vw_PrescriptionFulfilment).

5.3.3 HTML5, CSS3 Custom Properties, SVG, and Vanilla ES6 JavaScript

Client markup across all 61 templates adheres to semantic, accessible HTML5 (ARIA landmarks, aria-current, associated <label> elements, and keyboard focus rings). Styling in static/css/app.css (1,882 lines) uses CSS3 Custom Properties (--c-primary, --c-sidebar-bg, --c-accent, spacing tokens --s-1..--s-6) with zero external CSS framework bloat, paired with an inline SVG icon sprite (static/icons.svg) and unobtrusive ES6 JavaScript (static/js/app.js) for responsive sidebar toggling and dynamic prescription/POS row helpers.

5.4 Frameworks and Libraries

- **Spring Boot 4.1.2 & Spring WebMVC:** Provides convention-over-configuration bootstrap, externalized YAML property binding, Jakarta Bean Validation integration, custom HTTP 403/404/500 error routing, and Actuator /actuator/health probes.
- **Spring Security 6:** Implements stateful HTTP session authentication, DelegatingPasswordEncoder (BCrypt default), automatic synchronizer-token CSRF protection, 7-day Remember-Me cookies, custom authentication success/failure handlers, and filter-chain hooks for LoginThrottleFilter and ClinicStatusFilter.
- **Spring Data JPA & Hibernate 6:** Backs all 24 repository interfaces with compile-checked query methods, custom JPQL @Lock(LockModeType.PESSIMISTIC_WRITE) row locks for atomic token consumption, doctor slot booking, and pharmacy item dispensing, and PhysicalNamingStrategyStandardImpl for exact SQL Server column alignment.
- **Thymeleaf & Ultraq Layout Dialect 4.0.1:** Composes all portal pages cleanly from layout/base.html and layout/auth.html fragments while escaping untrusted text by default (th:text) to neutralize Cross-Site Scripting (XSS).
- **Flyway Database Migrations:** Configured with baseline-on-migrate=true and baseline-version=1 so existing SQL Server schemas upgrade deterministically through V002–V009 on startup without manual DDL drift.
- **JUnit 5, Mockito & AssertJ:** Powers fast, isolated unit tests of cryptographic/policy utilities alongside full @SpringBootTest + @AutoConfigureMockMvc web security and multi-step workflow integration tests.

5.5 Development and Runtime Environment

IMS supports two clean runtime profiles out of the box. The default profile connects via the Microsoft JDBC Driver (mssql-jdbc) over TCP port 1433 to localhost\SQLEXPRESS (database ClinicFlow_Dev) with HikariCP connection pooling and Flyway enabled. For instant developer onboarding or offline automated testing where SQL Server TCP/IP is unavailable, activating -Dspring-boot.run.profiles=h2 switches transparently to H2 in MSSQLServer compatibility mode and logs outbound verification/invitation links via clinicflow.mail.mode=log. Build reproducibility is guaranteed by bundling Apache Maven 3.9.11 directly inside tools/apache-maven-3.9.11 alongside PowerShell launch scripts in clinicflow-app/scripts/.

CHAPTER 6
IMPLEMENTATION

6.1 Module Description

The implementation of Infeara Medical System spans 189 Java classes across eight tightly cohesive functional modules under src/main/java/com/clinicflow/. Each module encapsulates its own controllers, form validation DTOs, @Transactional domain services, domain events, and audit instrumentation.

6.1.1 Module 1: Zero-Seed Platform Bootstrap (/setup) & Super Admin Console (/admin/**)

Implemented in SetupController, SetupService, AdminController, AdminClinicService, AdminPharmacyService, AdminPatientService, AdminReportService, and SmtpSettingsService, this module governs tenant lifecycle from day zero. IMS intentionally ships with zero hardcoded accounts. When userRepository.count() == 0, GET /setup renders the initial owner bootstrap screen; submitting SetupForm validates password policy, creates the first ACTIVE SUPER_ADMIN user, logs SETUP_COMPLETED in AuditLogs, and permanently locks /setup against subsequent calls. From /admin/**, the Super Admin enrolls Clinics and Pharmacies, triggers 72-hour STAFF_INVITE tokens via InvitationService, toggles tenant statuses (ACTIVE / SUSPENDED), inspects platform-wide KPI metrics, and configures live SMTP credentials encrypted via SecretCipher.

6.1.2 Module 2: Identity, Single-Use SHA-256 Auth Tokens, Invitations & Login Throttling

Implemented across RegistrationService, VerificationService, PasswordResetService, InvitationService, EmailChangeService, AuthTokenService, TokenHasher, PasswordPolicy, LoginAttemptService, and AuthRateLimiter, this module enforces healthcare-grade identity hygiene. Patient self-registration (/register) collects personal, address, and optional medical fields, enforces >=10-character alphanumeric passwords, creates a PENDING User and PATIENT_ENTERED Patient row (PAT-YYYY-NNNN), and issues a 24-hour VERIFY_EMAIL token. Raw tokens are 32-byte SecureRandom URL-safe strings sent only to the user's email; dbo.AuthTokens persists only the 32-byte SHA-256 hash. Re-issuing a token deletes prior unused tokens for that user and purpose, and consuming a token acquires a pessimistic write lock (findForUpdate) before setting used_at.

6.1.3 Module 3: Patient Self-Service Portal, Clinic Discovery & Atomic Slot Booking

Implemented in PatientProfileController, PatientProfileService, ClinicDiscoveryService, SlotService, BookingService, PatientBookingController, and PatientHistoryService, this module empowers authenticated patients while eliminating IDOR vulnerabilities. Every controller method resolves the acting Patient via patientRepository.findByUserUserId(principal.getUserId()). ClinicDiscoveryService lists only ACTIVE clinics with city/name search; SlotService computes available future time slots by subtracting active Appointments from weekday DoctorSchedules; and BookingService serializes bookings by locking the Doctor entity (doctorRepository.findForUpdate) before inserting into Appointments under UX_Appt_Slot.

6.1.4 Module 4: Staff-Verified Patient Account Linking & Non-Destructive Record Merge

Implemented in PatientLinkService, ClinicLinkRequestController, and ClinicLinkRequestService (390 lines), this module bridges online accounts with pre-existing clinic walk-in charts safely. A patient submits a link request to an active clinic from My Profile. In /clinic/link-requests/{id}, ClinicLinkRequestService automatically scans the clinic's Patient Masters for matching names and Dates of Birth to surface duplicate warnings. Approval requires ticking the physical identity-verification checkbox. If the administrator supplies an existing clinic patient ID to merge into, the service verifies exact DOB equality, re-points all Appointments, Prescriptions, ClinicPatients, and PatientLinkRequests to the target record, detaches user_id from the old row, marks merged_into_patient_id on the superseded record so audit foreign keys stay valid, and records PATIENT_RECORD_MERGED.

6.1.5 Module 5: Clinic Administration, Doctor Schedules & CSV Medicine Formulary

Implemented in ClinicPatientService, ClinicDoctorService, ClinicAppointmentService, and ClinicInventoryService, this module equips CLINIC_ADMIN staff with operational masters. Administrators onboard practitioners, set clinic-specific consultation fees in ClinicDoctors, define granular weekday DoctorSchedules (start time, end time, slot length in minutes, effective date bounds), and dispatch doctor login invitations. ClinicInventoryService manages standardized ClinicMedicines (drug name, dosage, default form, frequency, duration, quantity, instructions) with RFC-4180 CSV template download, bulk CSV import with upsert deduplication, and CSV export.

6.1.6 Module 6: Doctor Clinical Workspace — Consultations, Addenda & 32-Char e-Prescriptions

Implemented in DoctorConsultationController, DoctorConsultationService, DoctorPrescriptionController, and PrescriptionService (390 lines), this module governs clinical encounters. Starting an appointment transitions status to IN_CONSULTATION and creates a DRAFT Consultation. While in DRAFT, diagnosis, examination notes, and follow-up date remain editable. Clicking Finalise sets status=FINAL, stamps finalised_at, marks the Appointment COMPLETED, and permanently locks the primary clinical fields; any subsequent clinical clarification is saved exclusively as an author-attributed ConsultationAddendum. PrescriptionService then validates prescribed items (including chronic lifetime flags), enforces one prescription per consultation, generates a unique 32-character code via PublicIdGenerator.prescriptionCode(), and publishes PrescriptionIssuedEvent.

6.1.7 Module 7: Pharmacy Workspace — Verification, Auto-Pricing POS & Finance Ledger

Implemented in PharmacyController, PharmacyService (768 lines), PharmacyFinanceController, and PharmacyFinanceService, this module powers retail pharmacy operations. Scanning or entering a 32-character code executes a binary case-sensitive lookup AND Java exact-string filter, logs a PharmacyVerification row with a SHA-256 attempt_hash, and evaluates status, UTC expiry, and remaining item units. Inside /pharmacy/pos, verified prescription items auto-populate a billing cart pre-filled with remembered unit prices from PharmacyMedicinePrices. On sale completion, PharmacyService locks PrescriptionItems for update, verifies requested quantities do not exceed remaining units, computes BigDecimal(HALF_UP) subtotal, discount, percentage tax, net total, and change due, saves PrescriptionDispense and line items, updates remembered catalog prices, and invokes financeService to auto-post an INCOME ledger entry foreign-keyed to dispense_id.

6.1.8 Module 8: Transactional Email Outbox Worker & Encrypted SMTP Engine

Implemented across IdentityEmailListener, NotificationEmailListener, EmailOutboxService, EmailTemplates, EmailDeliveryService, EmailDispatchWorker, and SmtpSettingsService, outbound notifications are never sent directly on HTTP request threads. Domain listeners render structured plain-text clinical/security emails and insert QUEUED rows into dbo.EmailLogs guarded by unique idempotency_key values. Every 5 seconds, EmailDispatchWorker invokes EmailDeliveryService.dispatchBatch(), which locks due rows, sends via the active decrypted SmtpSettings JavaMailSender (or logs in dev mode), and applies 1-minute, 5-minute, and 15-minute retry backoff up to 3 attempts.

6.2 Core Algorithms and Cryptographic Mechanisms

- **Algorithm 1 — Unambiguous 32-Character CSPRNG Prescription Code Generation:** PublicIdGenerator draws 32 indices from java.security.SecureRandom over the 32-character unambiguous alphabet ABCDEFGHJKLMNPQRSTUVWXYZ23456789 (deliberately omitting visually confusable 0/O and 1/I). Each character carries log2(32) = 5 bits of entropy, yielding 32 x 5 = 160 bits of cryptographic randomness (1.46 x 10^48 possibilities), making brute-force guessing computationally infeasible.
- **Algorithm 2 — Single-Use SHA-256 Token Issuance & Atomic Consumption:** TokenHasher generates 32 random bytes (256 bits), Base64URL-encodes them without padding for email links, and stores MessageDigest('SHA-256') as BINARY(32) under unique index UX_AuthTokens_token_hash. Consumption locks the row via PESSIMISTIC_WRITE, verifies used_at IS NULL and expires_at > UTC_NOW, and stamps used_at atomically.
- **Algorithm 3 — AES-256-GCM Authenticated Secret Encryption:** SecretCipher derives a 256-bit AES key by hashing the deployment secret with SHA-256, generates a fresh 12-byte (96-bit) SecureRandom IV per encryption, encrypts plaintext under AES/GCM/NoPadding with a 128-bit authentication tag, and prepends the 12-byte IV to the ciphertext stored in SmtpSettings.password_encrypted.
- **Algorithm 4 — Dual-Guard Atomic Slot Allocation:** BookingService acquires a pessimistic write lock on the target Doctor entity (serializing concurrent requests for the same practitioner), re-verifies slot validity via SlotService against active appointments, and flushes the new Appointment under UX_Appt_Slot, translating any residual DataIntegrityViolationException into a clean user-facing conflict message.
- **Algorithm 5 — Non-Destructive Patient Record Merge:** ClinicLinkRequestService verifies exact DOB match between the online account patient row P_src and the clinic master patient row P_dst, migrates ClinicPatients, Appointments, Prescriptions, and PatientLinkRequests from P_src to P_dst, moves user_id from P_src to P_dst, and sets P_src.merged_into_patient_id = P_dst.patient_id without issuing DELETE on P_src.
- **Algorithm 6 — Self-Learning Pharmacy POS Pricing & Auto-Ledger Sync:** PharmacyService resolves unit prices from user input or falls back to case-insensitive lookup in PharmacyMedicinePrices, computes line_total = round(unit_price * qty, 2), taxable = subtotal - discount, tax_amount = round(taxable * tax_rate / 100, 2), and total = taxable + tax_amount, upserts new unit prices back into PharmacyMedicinePrices, and inserts a linked INCOME row in PharmacyFinanceEntries.

6.3 Pseudocode and Logic Flows

Pseudocode 6.1 — Staff-Verified Patient Link Approval & Non-Destructive Merge

```
PROCEDURE ApprovePatientLinkRequest(adminUserId, clinicId, requestId, form):
  1. IF form.identityVerified == FALSE THEN
       THROW ValidationException("Confirm physical identity verification before
           approving.")
  2. req <- PatientLinkRequestRepository.findForUpdateByClinic(requestId,
      clinicId)
  3. ASSERT req.status == PENDING
  4. accountPatient <- req.patient
  5. IF form.existingClinicPatientId IS BLANK THEN
       EnsureClinicPatientLink(clinicId, accountPatient)
       req.clinicPatientRecord <- accountPatient
       req.status <- APPROVED; req.resolvedBy <- adminUserId; req.resolvedAt <-
           UTC_NOW()
       AuditService.record(adminUserId, CLINIC_ADMIN, "PATIENT_LINK_APPROVED",
           accountPatient)
     ELSE
       targetPatient <- ResolveClinicPatientByPublicOrNumericId(clinicId,
           form.existingClinicPatientId)
       IF targetPatient.dob != accountPatient.dob THEN
         THROW ValidationException("Date of Birth mismatch; merge blocked for
             clinical safety.")
       ReassignAppointmentsAndPrescriptions(from=accountPatient,
           to=targetPatient)
       MergeClinicPatientMemberships(from=accountPatient, to=targetPatient)
       targetPatient.user <- accountPatient.user
       accountPatient.user <- NULL
       accountPatient.mergedIntoPatientId <- targetPatient.patientId
       req.patient <- targetPatient; req.clinicPatientRecord <- targetPatient
       req.status <- APPROVED; req.resolvedBy <- adminUserId; req.resolvedAt <-
           UTC_NOW()
       AuditService.record(adminUserId, CLINIC_ADMIN, "PATIENT_RECORD_MERGED",
           targetPatient)
  6. COMMIT TRANSACTION
```

Pseudocode 6.2 — Pharmacy POS Verification, Partial Dispensing & Auto-Finance Sync

```
PROCEDURE CompletePosSaleAndDispense(pharmUserId, verificationId, qtyMap,
    priceMap, discount, taxRate, payMethod):
  1. pharmUser <- RequireActivePharmacyUser(pharmUserId)
  2. ver <- VerificationRepository.findById(verificationId)
  3. ASSERT ver.pharmacyId == pharmUser.pharmacyId AND ver.outcome == VALID
  4. ASSERT NOT DispenseRepository.existsByVerificationId(verificationId)
  5. rx <- PrescriptionRepository.findForUpdate(ver.prescriptionId)
  6. ASSERT rx.status == VALID AND rx.expiryDate > UTC_NOW()
  7. items <-
      PrescriptionItemRepository.findForUpdateByPrescriptionId(rx.prescriptionId)
  8. subtotal <- 0.00
  9. FOR EACH item IN items:
       q <- qtyMap.getOrDefault(item.id, 0)
       IF q > (item.quantity - item.quantityDispensed) THEN
         THROW IllegalArgumentException("Over-dispense blocked for " +
             item.medicine)
       IF q > 0 THEN
         uPrice <- Coalesce(priceMap.get(item.id),
             RememberedCatalogPrice(pharmUser.pharmacyId, item.medicine), 0.00)
         subtotal <- subtotal + RoundHalfUp(uPrice * q, 2)
  10. taxable <- subtotal - discount
  11. taxAmt  <- RoundHalfUp(taxable * taxRate / 100, 2)
  12. total   <- taxable + taxAmt
  13. receipt <- SavePrescriptionDispense(rx, pharmUser, ver, subtotal, discount,
      taxRate, taxAmt, total, payMethod)
  14. FOR EACH accepted line (item, q, uPrice):
        item.quantityDispensed <- item.quantityDispensed + q
        SaveDispenseItem(receipt, item, q, uPrice, RoundHalfUp(uPrice * q, 2))
        UpsertPharmacyMedicinePrice(pharmUser.pharmacyId, item.medicine, uPrice)
  15. IF total > 0 THEN
        PharmacyFinanceService.recordPosIncome(pharmUser.pharmacy, receipt,
            total, rx.prescriptionCode)
  16. AuditService.record(pharmUserId, pharmUser.role, "DISPENSE_COMPLETED",
      receipt.publicUuid)
  17. COMMIT TRANSACTION & RETURN receipt
```

6.4 Key Source Code Implementations

The following authentic Java source listings are excerpted directly from the clinicflow-app repository to illustrate how security rules, cryptographic primitives, atomic booking, 32-character prescribing, and transactional outbox delivery are realized in production code.

Listing 6.1 — Spring Security 6 RBAC & Tenant Filter Chain (SecurityConfig.java)

```
@Bean
public SecurityFilterChain securityFilterChain(HttpSecurity http) throws
    Exception {
    http
        .authorizeHttpRequests(auth -> auth
            .requestMatchers(
                "/", "/login", "/setup", "/setup/**", "/invite", "/invite/**",
                "/register", "/register/**", "/verify-email", "/verify-email/**",
                "/verify-email-change", "/verify-email-change/**",
                "/forgot-password", "/forgot-password/**",
                "/reset-password", "/reset-password/**",
                "/css/**", "/js/**", "/images/**", "/actuator/health", "/error",
                "/error/**"
            ).permitAll()
            .requestMatchers("/admin/**").hasRole("SUPER_ADMIN")
            .requestMatchers("/clinic/**").hasRole("CLINIC_ADMIN")
            .requestMatchers("/doctor/**").hasRole("DOCTOR")
            .requestMatchers("/patient/**").hasRole("PATIENT")
            .requestMatchers("/pharmacy/**").hasAnyRole("PHARMACY_ADMIN",
                "PHARMACIST")
            .anyRequest().authenticated()
        )
        .formLogin(form -> form
            .loginPage("/login").loginProcessingUrl("/login")
            .usernameParameter("email").passwordParameter("password")
            .successHandler(successHandler).failureHandler(failureHandler).permit
                All()
        )
        .exceptionHandling(ex -> ex.accessDeniedPage("/error/403"))
        .addFilterBefore(loginThrottleFilter,
            UsernamePasswordAuthenticationFilter.class)
        .addFilterAfter(clinicStatusFilter,
            UsernamePasswordAuthenticationFilter.class);
    return http.build();
}
```

Listing 6.2 — 32-Character CSPRNG Prescription Code & AES-256-GCM Cipher (PublicIdGenerator.java & SecretCipher.java)

```
// PublicIdGenerator.java — 160-bit unambiguous 32-character prescription
    identifier
public final class PublicIdGenerator {
    private static final SecureRandom RANDOM = new SecureRandom();
    private static final char[] ALPHABET =
        "ABCDEFGHJKLMNPQRSTUVWXYZ23456789".toCharArray();

    public static String patientId(long sequence) {
        return String.format(Locale.ROOT, "PAT-%d-%04d", Year.now().getValue(),
            sequence);
    }
    public static String appointmentId(long sequence) {
        return String.format(Locale.ROOT, "APT-%d-%05d", Year.now().getValue(),
            sequence);
    }
    public static String prescriptionCode() {
        StringBuilder sb = new StringBuilder(32);
        for (int i = 0; i < 32; i++) {
            sb.append(ALPHABET[RANDOM.nextInt(ALPHABET.length)]);
        }
        return sb.toString();
    }
}

// SecretCipher.java — Authenticated AES-256-GCM encryption for stored SMTP
    credentials
public byte[] encrypt(String plaintext) {
    if (plaintext == null || plaintext.isEmpty()) return null;
    byte[] iv = new byte[12];
    random.nextBytes(iv);
    Cipher cipher = Cipher.getInstance("AES/GCM/NoPadding");
    cipher.init(Cipher.ENCRYPT_MODE, key, new GCMParameterSpec(128, iv));
    byte[] encrypted =
        cipher.doFinal(plaintext.getBytes(StandardCharsets.UTF_8));
    byte[] result = new byte[iv.length + encrypted.length];
    System.arraycopy(iv, 0, result, 0, iv.length);
    System.arraycopy(encrypted, 0, result, iv.length, encrypted.length);
    return result;
```

```
}
```

Listing 6.3 — Single-Use SHA-256 Hashed Token Vault (AuthTokenService.java)

```
@Transactional
public IssuedToken issue(User user, AuthTokenPurpose purpose) {
    repository.deleteByUserAndPurpose(user.getUserId(), purpose);
    String rawToken = TokenHasher.newToken();
    AuthToken token = new AuthToken();
    token.setUser(user);
    token.setPurpose(purpose);
    token.setTokenHash(TokenHasher.sha256(rawToken));
    token.setExpiresAt(now().plus(ttlFor(purpose)));
    token.setCreatedAt(now());
    repository.save(token);
    return new IssuedToken(rawToken, token);
}

@Transactional
public Optional<AuthToken> consume(String rawToken, AuthTokenPurpose purpose) {
    if (rawToken == null || rawToken.isBlank()) return Optional.empty();
    LocalDateTime now = now();
    return repository.findForUpdate(TokenHasher.sha256(rawToken), purpose)
            .filter(token -> token.getUsedAt() == null)
            .filter(token -> token.getExpiresAt().isAfter(now))
            .map(token -> { token.setUsedAt(now); return token; });
}
```

Listing 6.4 — Atomic Conflict-Free Slot Reservation (BookingService.java)

```
@Transactional
public Long book(Long patientUserId, BookingForm form) {
    Patient patient = patientRepository.findByUserUserId(patientUserId)
            .orElseThrow(() -> new IllegalStateException("Your patient profile is
                not linked."));
    Clinic clinic = clinicRepository.findById(form.getClinicId())
            .filter(c -> c.getStatus() == ClinicStatus.ACTIVE)
            .orElseThrow(() -> new IllegalArgumentException("Clinic is not
                available."));
    // Pessimistic write lock serialises competing bookings for the same doctor
    Doctor doctor = doctorRepository.findForUpdate(form.getDoctorId())
            .filter(d -> d.getStatus() == DoctorStatus.ACTIVE)
            .orElseThrow(() -> new IllegalArgumentException("Doctor is not
                available."));
    if (!slotService.isValidSlot(clinic.getClinicId(), doctor.getDoctorId(),
        form.getSlotStart())) {
        throw new IllegalArgumentException("That slot is no longer available.
            Choose another.");
    }
    Appointment appointment = new Appointment();
    appointment.setPatient(patient); appointment.setClinic(clinic);
        appointment.setDoctor(doctor);
    appointment.setSlotStart(form.getSlotStart());
    appointment.setSlotEnd(form.getSlotStart().plusMinutes(
            slotService.slotLengthMinutes(clinic.getClinicId(),
                doctor.getDoctorId(), form.getSlotStart())));
    appointment.setStatus(AppointmentStatus.BOOKED);
    try {
        appointment = appointmentRepository.saveAndFlush(appointment);
    } catch (DataIntegrityViolationException ex) {
        throw new IllegalArgumentException("That slot was just taken. Choose
            another.");
    }
    appointment.setPublicAppointmentId(PublicIdGenerator.appointmentId(appointmen
        t.getAppointmentId()));
    return appointmentRepository.save(appointment).getAppointmentId();
}
```

Listing 6.5 — Non-Destructive Patient Chart Consolidation (ClinicLinkRequestService.java)

```
private Patient mergeIntoClinicRecord(Patient account, Patient clinicRecord, Long
    clinicId,
                                      PatientLinkRequest currentRequest) {
    User accountUser = account.getUser();

    List<ClinicPatient> links =
        clinicPatientRepository.findByPatientPatientId(account.getPatientId());
    for (ClinicPatient link : links) {
        boolean sameClinic = link.getClinic().getClinicId().equals(clinicId);
        boolean targetAlreadyLinked =
            clinicPatientRepository.existsByClinicClinicIdAndPatientPatientId(
                link.getClinic().getClinicId(), clinicRecord.getPatientId());
        if (sameClinic || targetAlreadyLinked) {
            clinicPatientRepository.delete(link);
        } else {
            link.setPatient(clinicRecord);
            clinicPatientRepository.save(link);
        }
    }
    appointmentRepository.findByPatientPatientIdOrderBySlotStartDesc(account.getP
        atientId())
            .forEach(a -> a.setPatient(clinicRecord));
    prescriptionRepository.findByPatientPatientIdOrderByIssueDateDesc(account.get
        PatientId())
            .forEach(rx -> rx.setPatient(clinicRecord));

    clinicRecord.setEmail(accountUser.getEmail());
    clinicRecord.setMobile(copyIfBlank(clinicRecord.getMobile(),
        account.getMobile()));
    clinicRecord.setAllergies(copyIfBlank(clinicRecord.getAllergies(),
        account.getAllergies()));
    clinicRecord.setConditions(copyIfBlank(clinicRecord.getConditions(),
        account.getConditions()));
    clinicRecord.setUser(accountUser);
    clinicRecord.setUpdatedAt(LocalDateTime.now(ZoneOffset.UTC));
    patientRepository.save(clinicRecord);

    account.setUser(null); // Retain historical row so foreign keys in AuditLogs
```

```
        stay valid
    account.setMergedInto(clinicRecord);
    patientRepository.save(account);
    return clinicRecord;
}
```

Listing 6.6 — Receipt-Linked Pharmacy Cashbook Ledger Service (PharmacyFinanceService.java)

```
@Transactional
public void addEntry(Long userId, String type, BigDecimal amount, String
    description,
                     LocalDate entryDate, Long actorUserId, Long dispenseId) {
    PharmacyUser link = pharmacyUserRepository.findByUserUserId(userId)
            .orElseThrow(() -> new IllegalStateException("Pharmacy account is not
                linked."));
    if (!"INCOME".equals(type) && !"EXPENSE".equals(type)) {
        throw new IllegalArgumentException("Choose INCOME or EXPENSE.");
    }
    if (amount == null || amount.compareTo(BigDecimal.ZERO) <= 0) {
        throw new IllegalArgumentException("Enter an amount greater than zero.");
    }
    PharmacyFinanceEntry entry = new PharmacyFinanceEntry();
    entry.setPharmacy(link.getPharmacy());
    entry.setEntryType(type);
    entry.setAmount(amount);
    entry.setDescription(description == null || description.isBlank() ? null :
        description.trim());
    entry.setEntryDate(entryDate != null ? entryDate : LocalDate.now());
    entry.setCreatedBy(actorUserId);
    entry.setDispenseId(dispenseId);
    entry.setCreatedAt(LocalDateTime.now(ZoneOffset.UTC));
    entryRepository.save(entry);
}
```

Table 6.1: Complete Thymeleaf View & Fragment Directory Inventory (61 HTML5 Templates)

Template Directory	Count	Representative Templates & Clinical Views
templates/layout/ & fragments/	5	base.html (ERP shell), auth.html (split login/register), brand.html (IMS SVG mark), page-header.html, status-badge.html
templates/auth/ & error/	16	setup.html, login.html, register.html, register-check-email.html, verify-result.html, forgot-password.html, reset-password.html, invite-accept.html, error/403, 404, 500
templates/admin/	10	dashboard.html, clinics.html, clinic-new.html, clinic-detail.html, pharmacies.html, pharmacy-new.html, pharmacy-detail.html, patients.html, reports.html, smtp.html
templates/clinic/	12	dashboard.html, patients.html, patient-new.html, patient-detail.html, link-requests.html, link-request-detail.html, doctors.html, doctor-schedules.html, appointments.html, inventory.html, prescriptions.html
templates/doctor/	5	dashboard.html, appointments.html, consultation.html (draft/final + addenda), prescription-form.html, prescriptions.html
templates/patient/	7	dashboard.html, profile.html (verified DOB + clinic link requests), clinics.html, clinic-detail.html, book.html, appointments.html, prescriptions.html, history.html
templates/pharmacy/	6	dashboard.html, pos.html (live 32-char Rx POS), verify.html, dispense-receipt.html, prices.html, finance.html, users.html, reports.html

6.5 Screenshots of Live Application Across Operational Workspaces

Figures 6.1 through 6.14 present authentic full-resolution screenshots captured directly from the live running Infeara Medical System (IMS v0.1) deployment across the Authentication Gateway, Super Admin Platform Governance Portal, Doctor Clinical Workspace, and Pharmacy POS & Finance Workspace. Every screen reflects live SQL Server relational records, active role-scoped ERP navigation, real 32-character CSPRNG prescription identifiers, and automated POS-to-Finance ledger synchronization.

Screen 6.1 Walkthrough: Displays the live IMS authentication card rendered via templates/layout/auth.html with the medical-cross brand mark, post-logout confirmation banner ('You have been signed out.'), BCrypt-backed email/password inputs (mvignesh479@gmail.com), 7-day Remember-Me checkbox, password recovery link, email verification resend trigger, and self-service patient registration entry point.

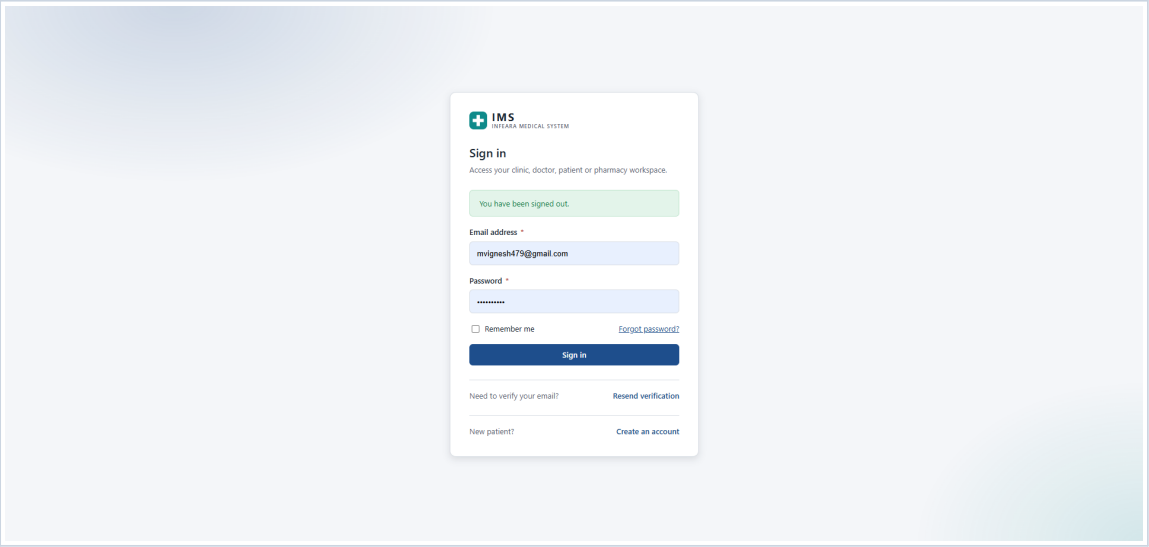


Figure 6.1: IMS Unified Sign-In & Session Lifecycle Gateway (/login)

Screen 6.2 Walkthrough: Shows the live Super Admin command center featuring six real-time KPI metric cards (Active Clinics: 1, Pending Enrollment: 0, Registered Patients: 2, Appointments Today: 0, Prescriptions Issued: 2, Email Failures: 0) alongside the enrolled facility table listing Vignesh Clinics (ACTIVE) and quick Enroll Clinic CTA.

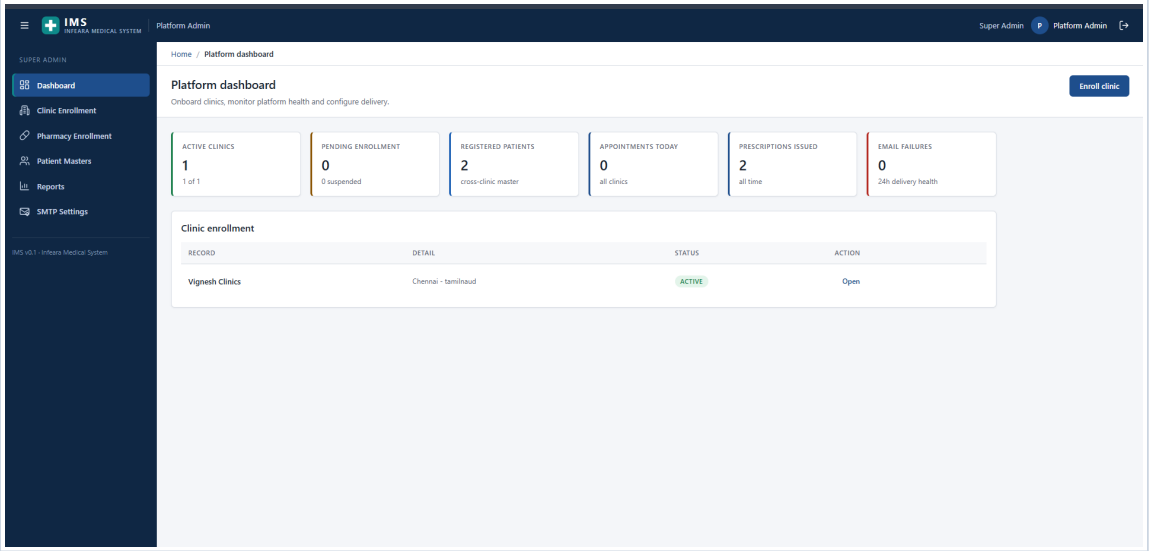


Figure 6.2: Super Admin — Platform Governance Dashboard (/admin/dashboard)

Screen 6.3 Walkthrough: Captures the live clinic tenant directory showing Vignesh Clinics (Chennai), active Clinic Administrator vstv4k2002@gmail.com, linked doctor count (1), linked patient count (2), ACTIVE tenant status badge, and enrollment date (25 Oct 2026).

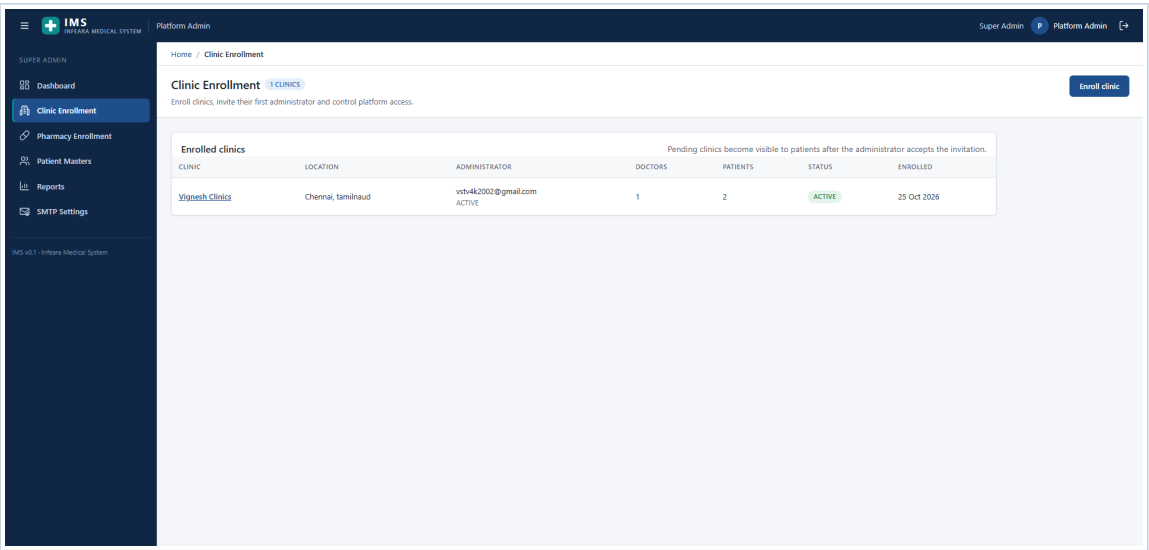


Figure 6.3: Super Admin — Multi-Tenant Clinic Enrollment Console (/admin/clinics)

Screen 6.4 Walkthrough: Depicts the live pharmacy tenant registry listing Vignesh Pharmacy with verified drug licence number KL-12356H2-77 (Chennai, Tamilnadu), active Pharmacy Administrator mvignesh479@gmail.com, staff count, and ACTIVE operational status.

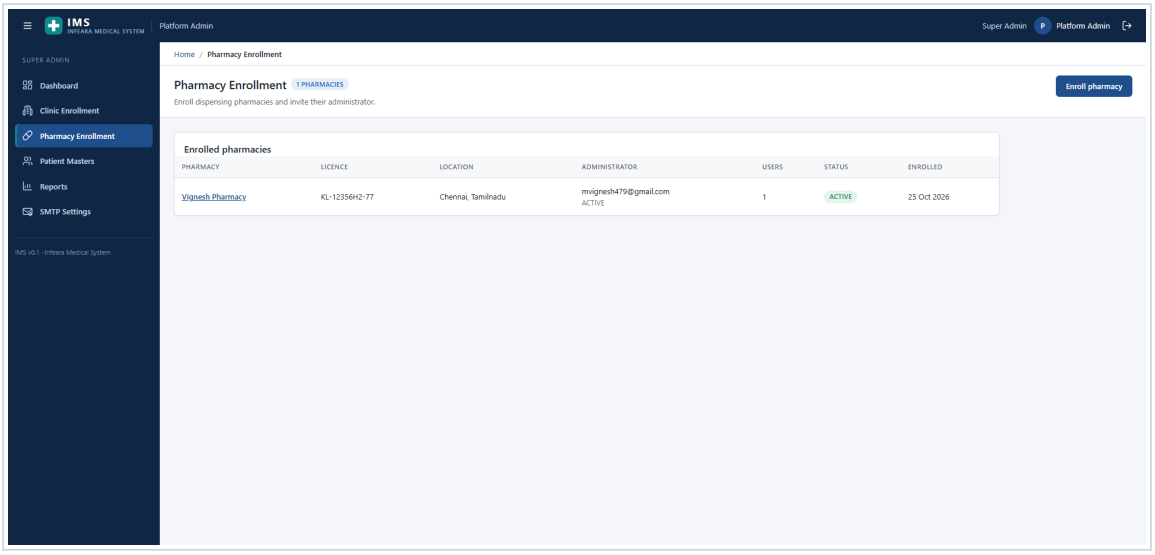


Figure 6.4: Super Admin — Dispensing Pharmacy Enrollment & Licence Registry (/admin/pharmacies)

Screen 6.5 Walkthrough: Demonstrates IMS's privacy-by-design READ ONLY cross-clinic patient index. While Super Admin can search public IDs (PAT-2026-0004, PAT-2026-0003) and inspect clinic associations (Vignesh Clinics), contact PII is automatically masked server-side (******3289, ******4546, v***@gmail.com).

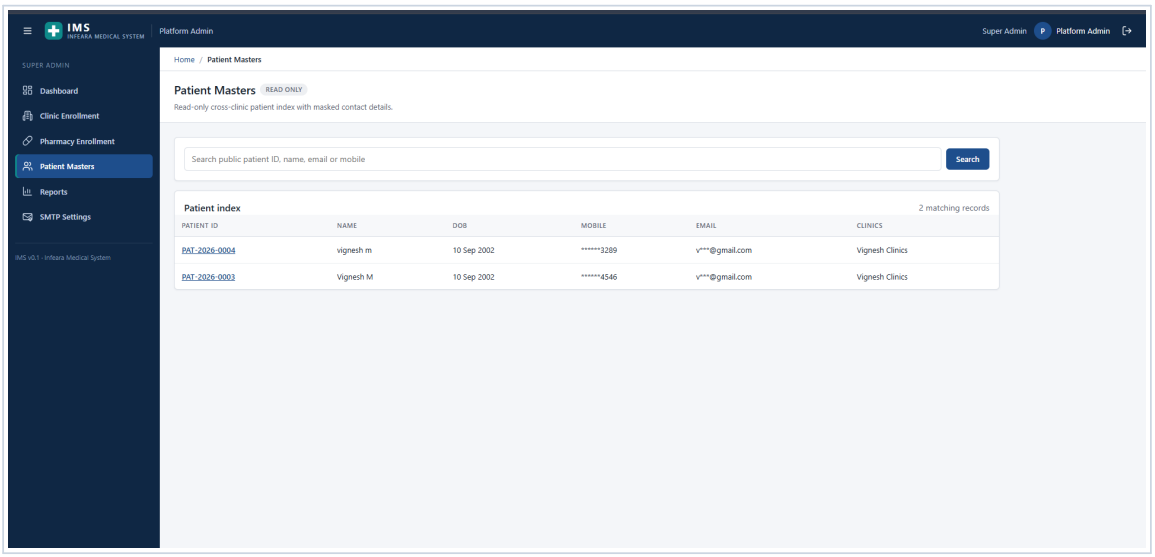


Figure 6.5: Super Admin — Privacy-Masked Cross-Clinic Patient Masters (/admin/patients)

Screen 6.6 Walkthrough: Displays live cross-tenant operational totals across clinics, pharmacies, patient masters, completed appointments (COMPLETED: 2), transactional email outbox health (SENT: 13, QUEUED: 0, FAILED: 0), and patient clinic-linking verification counters.

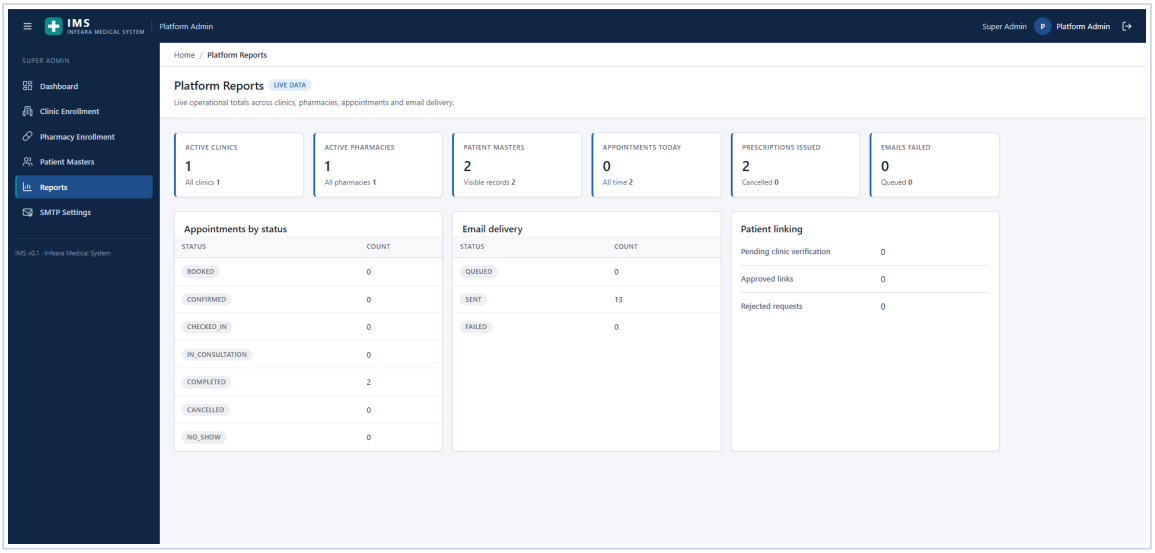


Figure 6.6: Super Admin — Live Platform Reports & Outbox Delivery Telemetry (/admin/reports)

Screen 6.7 Walkthrough: Shows the live mail-relay configuration form (Host, Port 587, STARTTLS, From name IMS) enforcing at-rest AES-256-GCM password encryption ('Passwords are encrypted at rest and never shown again') paired with the live Recent Email Activity feed logging SENT events for PRESCRIPTION_ISSUED, APPOINTMENT_CONFIRMATION, RESET_PASSWORD, and VERIFY_EMAIL.

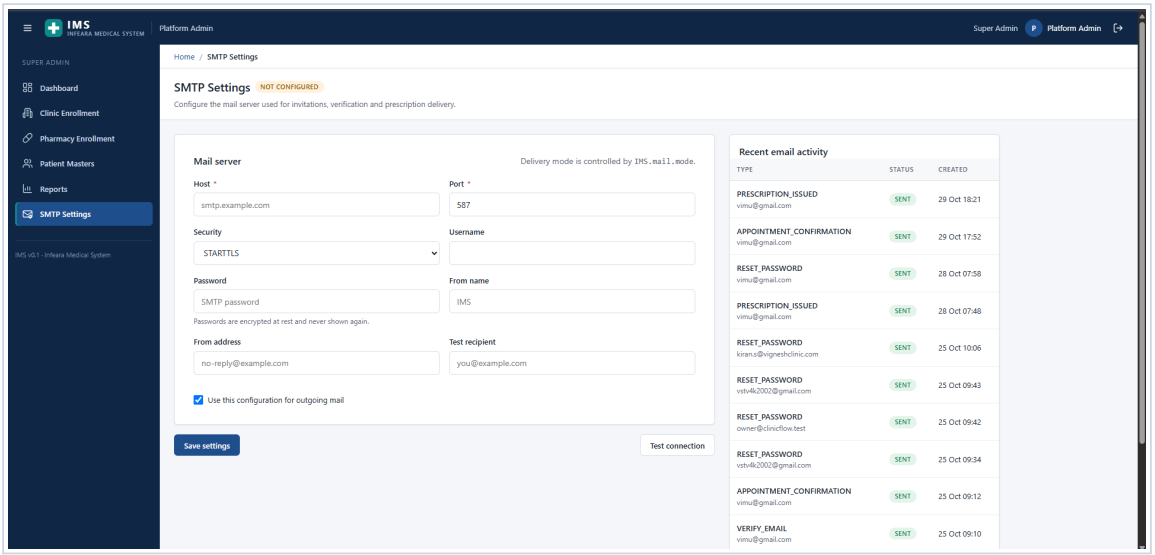


Figure 6.7: Super Admin — AES-256-GCM Encrypted SMTP Configuration & Mail Audit (/admin/smtp)

Screen 6.8 Walkthrough: Captures the role-scoped Doctor Workspace for practitioner Kiran (General Medicine), displaying personal schedule KPIs (Appointments Today: 0, Prescriptions Issued: 2, Specialization card) and today's patient queue.

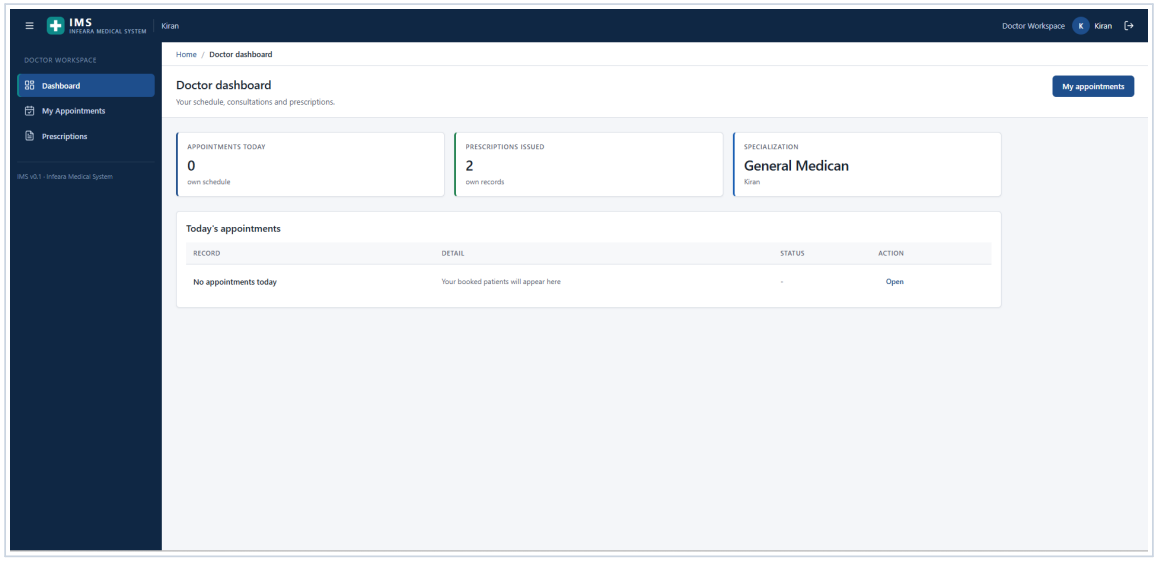


Figure 6.8: Doctor Workspace — Practitioner Clinical Dashboard (/doctor/dashboard)

Screen 6.9 Walkthrough: Illustrates the practitioner's date-filtered daily queue (2026-10-31) with status selector, Today / All Dates quick filters, and contextual front-desk check-in workflow guidance.

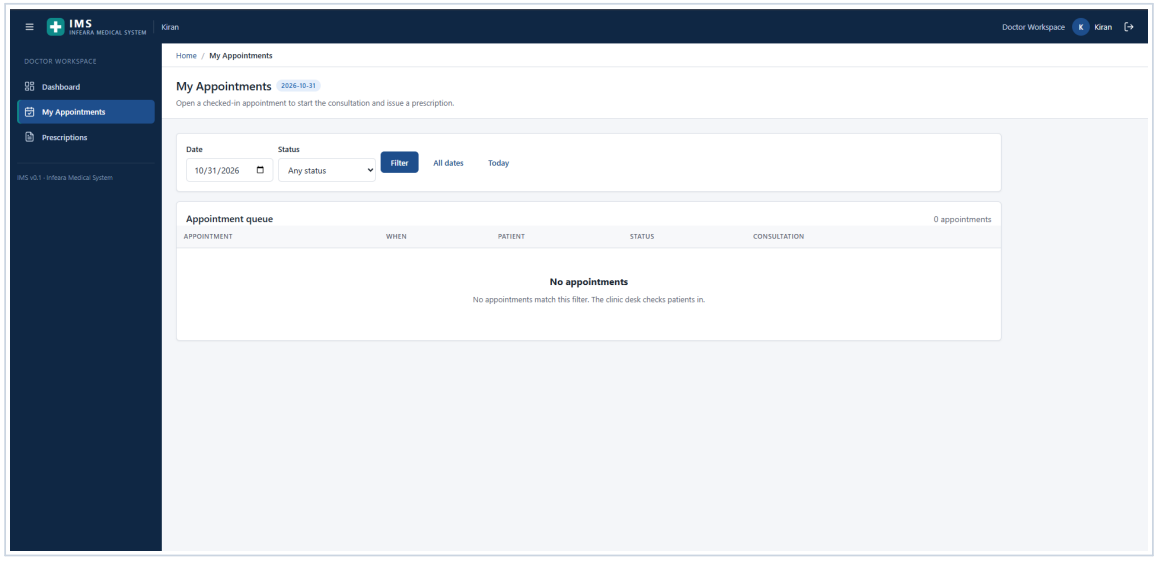


Figure 6.9: Doctor Workspace — Date & Status Filtered Appointment Queue (/doctor/appointments)

Screen 6.10 Walkthrough: Displays historical clinical encounters APT-2026-00004 (30 Oct 2026, 10:00) and APT-2026-00003 (26 Oct 2026, 10:10) for patient vignesh m (PAT-2026-0004), both showing COMPLETED appointment status and irreversibly locked FINAL consultation status.

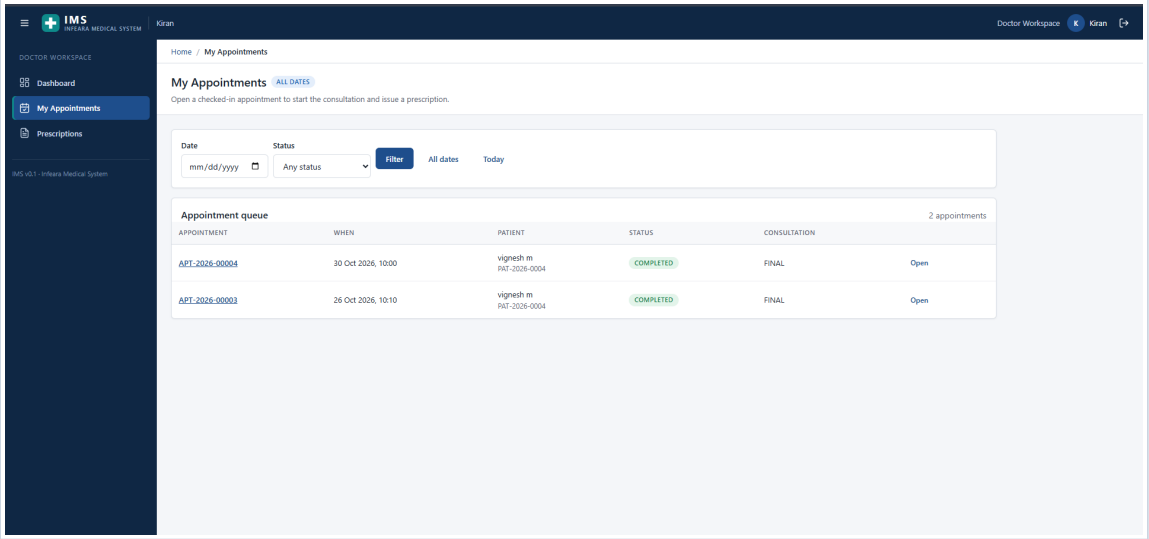


Figure 6.10: Doctor Workspace — Completed Encounters & Finalised Consultations (/doctor/appointments?all)

Screen 6.11 Walkthrough: Showcases two live 32-character CSPRNG electronic prescriptions authored by Dr. Kiran for PAT-2026-0004: JUEB9AGPAKBTXGWNX3MQKNCUFA3DMXRP (PARTIALLY DISPENSED, VALID until 29 Nov 2026) and VEYHFFWEGYVZUWWWQHGY28RSUT8F59WT (NOT DISPENSED, automatically marked EXPIRED on 29 Oct 2026).

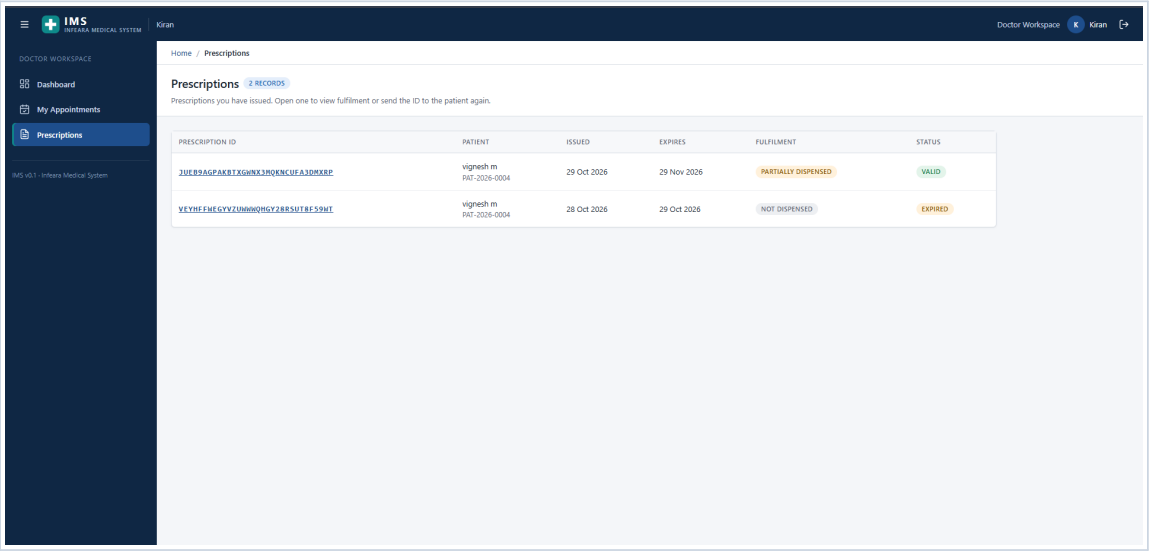


Figure 6.11: Doctor Workspace — Issued 32-Character Prescriptions & Fulfilment States (/doctor/prescriptions)

Screen 6.12 Walkthrough: Presents Vignesh Pharmacy's operational home screen with live daily counters (Verifications Today, Dispenses Today, Pharmacy Users: 1), left-rail POS/Price List/Finance navigation, and one-click 32-character prescription verification shortcut.

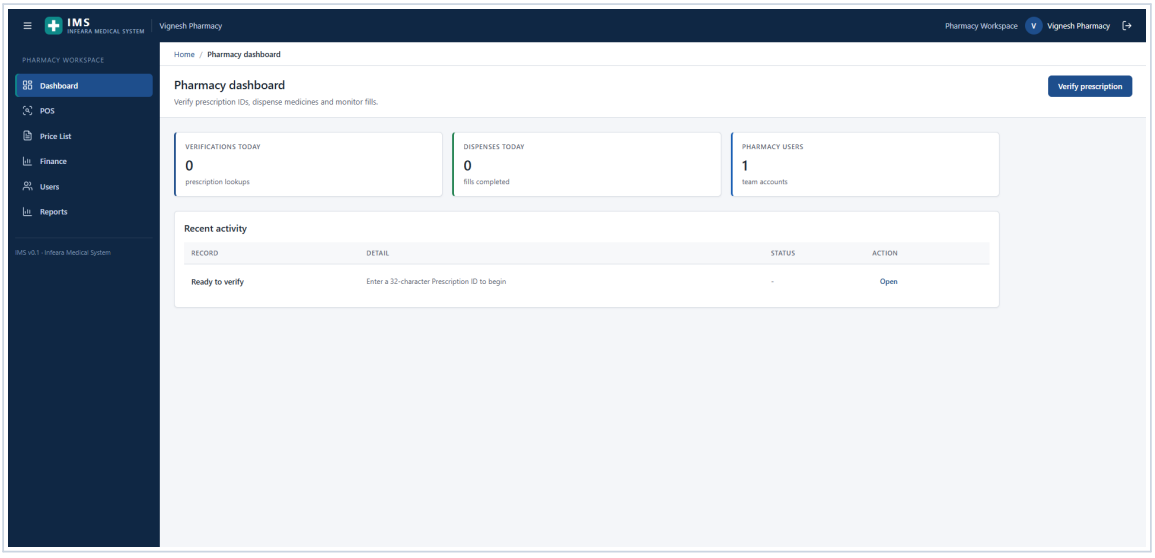


Figure 6.12: Pharmacy Workspace — Dispensing Operations Dashboard (/pharmacy/dashboard)

Screen 6.13 Walkthrough: Captures the live Pharmacy POS terminal featuring the case-sensitive 32-character Prescription ID scanner bar, Analyze Prescription & Price List actions, live counter telemetry, and completed POS sales receipts SALE-000006 (Rs. 12.50 CASH) and SALE-000005 (Rs. 62.50 UPI) against JUEB9AGPAKBTXGWNX3MQKNCUFA3DMXRP.

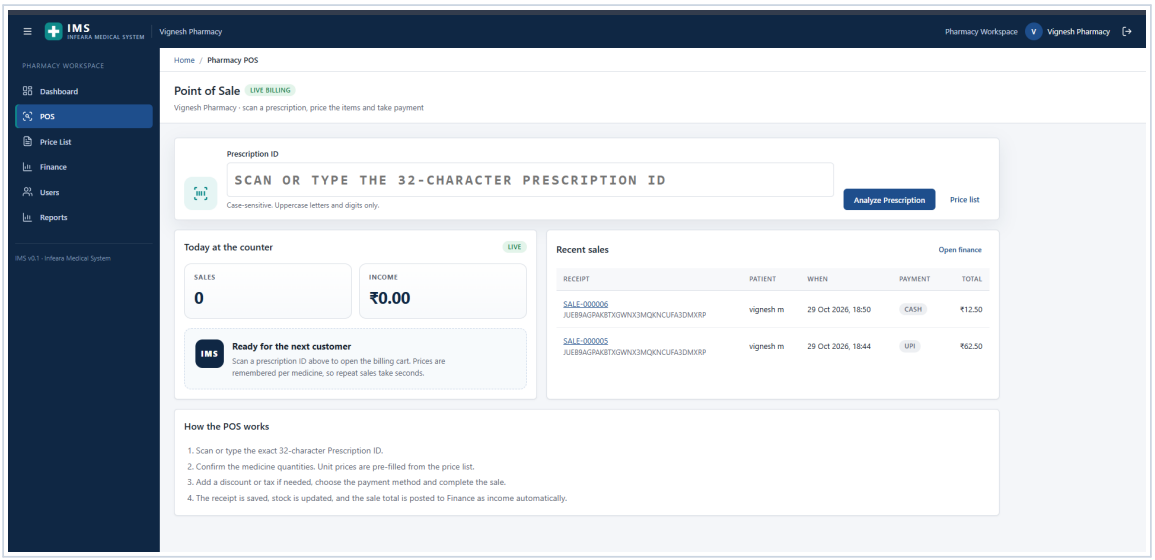


Figure 6.13: Pharmacy Workspace — Live 32-Character Prescription Point of Sale (/pharmacy/pos)

Screen 6.14 Walkthrough: Demonstrates end-to-end POS-to-Finance accounting integration: five KPI cards track Today & 30-Day Income (Rs. 75.00), Expenses (Rs. 100.00), and Net Cashflow (-Rs. 25.00), while the Finance Ledger displays auto-posted receipt-linked POS SALE income (Rs. 12.50 CASH) alongside manual operating expenses (Electricity bill).

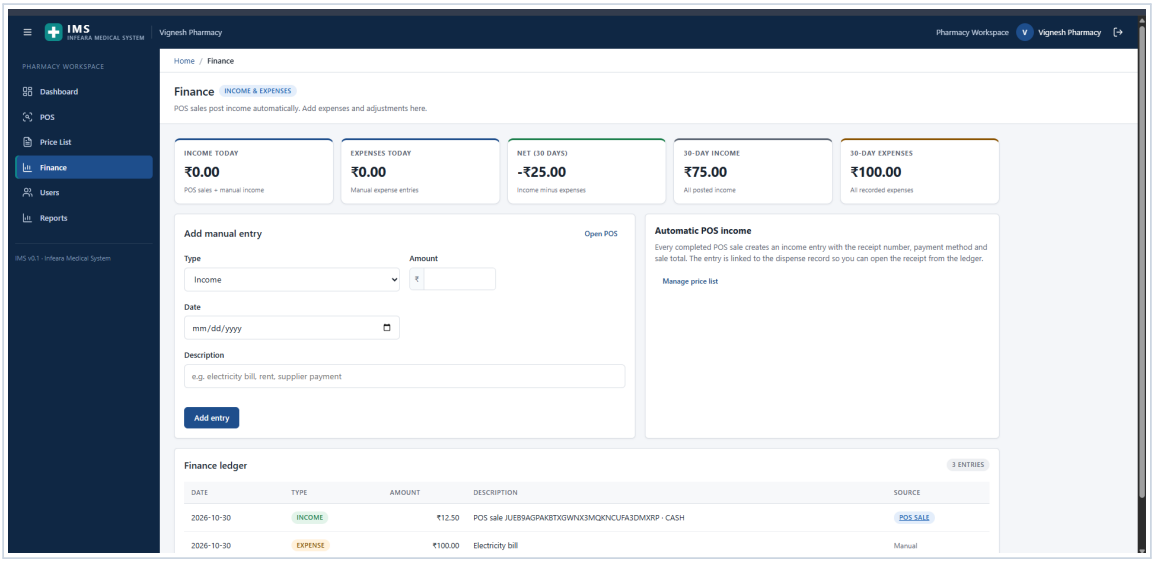


Figure 6.14: Pharmacy Workspace — Automated POS Income & Expense Finance Ledger (/pharmacy/finance)

6.6 Integration of Modules and Cross-Cutting Event Flow

All eight modules integrate cleanly inside a single Spring ApplicationContext through three mechanisms: (1) shared GlobalModelAttributes (@ControllerAdvice) that inject navigation items, breadcrumbs, active principal metadata, and organization display names into every Thymeleaf view; (2) synchronous Spring ApplicationEventPublisher events (VerificationEmailRequested, PasswordResetEmailRequested, StaffInviteRequested, EmailChangeRequested, AppointmentBookedEvent, AppointmentCancelledEvent, and PrescriptionIssuedEvent) consumed within the same database transaction by IdentityEmailListener and NotificationEmailListener to populate dbo.EmailLogs atomically; and (3) AuditService, which enriches every security and clinical milestone with client IP resolution (ClientIp) and patient-centric indexing.

CHAPTER 7
TESTING & RESULTS

7.1 Testing Strategy

Because healthcare management software directly handles personal health records, clinical diagnoses, medication regimens, and financial pharmacy transactions, Infeara Medical System adopts a multi-layered verification strategy spanning fast deterministic JUnit 5 unit tests, Mockito-isolated domain service ownership tests, full-stack Spring Boot MockMvc HTTP security tests against H2 in MSSQLServer compatibility mode, and end-to-end multi-role browser validation against Microsoft SQL Server 2022 Express.

Every automated test executes offline via mvn -B test without requiring an external database daemon or live SMTP relay. Security-critical invariants—such as session-derived patient ownership, CSRF token enforcement on POST endpoints, cross-clinic link isolation, DOB mismatch rejection during patient merges, and role-based 403 Forbidden enforcement—are verified through automated assertions rather than manual spot-checks.

7.2 Types of Testing

7.2.1 Unit Testing (JUnit Jupiter 5, AssertJ & Mockito)

Unit tests validate pure cryptographic, identifier, routing, and domain-service logic in sub-millisecond isolation. In clinicflow-app/src/test/java/com/clinicflow/, PublicIdGeneratorTest verifies PAT-YYYY-NNNN and APT-YYYY-NNNNN formatting plus 32-character unambiguous prescription-code entropy; PasswordPolicyTest exercises boundary lengths (<10 vs >=10 chars), missing-letter, and missing-digit edge cases; TokenHasherTest verifies 256-bit URL-safe token uniqueness and deterministic 32-byte SHA-256 digesting; RoleHomeResolverTest confirms dashboard target resolution for all six Role enum values; and PatientProfileServiceTest + ClinicLinkRequestServiceTest verify Mockito-mocked ownership, input trimming, duplicate detection, DOB-mismatch blocking, record merging, and AuditService event emission.

7.2.2 Integration and Web Security Testing (Spring Boot Test & MockMvc)

Integration suites annotated with @SpringBootTest, @AutoConfigureMockMvc, and @ActiveProfiles('h2') (specifically PatientProfileWebTest and ClinicLinkRequestWebTest) boot the complete Spring WebMVC + Spring Security 6 filter chain, Hibernate 6 persistence context, and Thymeleaf template engine. MockMvc simulates realistic multi-actor HTTP sessions—logging in as a Patient to submit a clinic link request with .with(csrf()), verifying Pending status rendering in HTML, logging in as Clinic Admin of Clinic A to confirm cross-clinic requests from Clinic B are invisible, testing unverified-checkbox rejection, executing full patient record merges, and asserting HTTP 403 on unauthorized role access.

7.2.3 System, Concurrency, and End-to-End Browser Validation

System-level testing validates the integrated behavior of all eight modules across realistic multi-portal journeys: bootstrapping the first Super Admin at /setup, enrolling clinics and pharmacies, accepting 72-hour staff invitations, registering and verifying patient accounts, booking and rescheduling doctor slots, finalizing consultations with signed addenda, issuing 32-character prescriptions, scanning codes at /pharmacy/pos across partial and full dispenses, verifying over-dispense blocking, and confirming automatic INCOME ledger synchronization in /pharmacy/finance.

7.3 Test Cases and Results

Tables 7.1 and 7.2 document 36 representative test cases executed across the automated JUnit/MockMvc suite and end-to-end system validation, recording test inputs, expected clinical/security outcomes, and verified PASS statuses.

Table 7.1: Automated Unit & Service-Layer Test Cases (JUnit 5 + Mockito)

TC ID	Test Class / Module	Scenario & Input Condition	Expected Output	Status
TC-01	PublicIdGeneratorTest	Generate patientId(7) and appointmentId(108)	Formats PAT-2026-0007 and APT-2026-00108 accurately	PASS
TC-02	PublicIdGeneratorTest	Generate 1,000 prescriptionCode() values	All 32 chars, uppercase alphanumeric excluding 0,O,1,I; 0 collisions	PASS
TC-03	PasswordPolicyTest	Validate 'Short1' (6 chars) and 'alllettersonly'	Rejected by PasswordPolicy with descriptive rule message	PASS
TC-04	PasswordPolicyTest	Validate 'ClinicPass123' (>=10 chars, letter + digit)	Accepted as compliant strong password	PASS
TC-05	TokenHasherTest	Invoke newToken() twice and sha256(token)	Distinct URL-safe strings; deterministic 32-byte SHA-256 digest	PASS
TC-06	RoleHomeResolverTest	Resolve home URL for all 6 Role enum constants	Maps to /admin, /clinic, /doctor, /patient, /pharmacy dashboards	PASS
TC-07	PatientProfileServiceTest	Authenticated user 42 updates profile with whitespace	Fields trimmed, saved to user 42's Patient, PATIENT_PROFILE_UPDATED audited	PASS
TC-08	PatientProfileServiceTest	Update called when no Patient row linked to userId	Returns false safely; zero repository writes or false audits	PASS
TC-09	ClinicLinkRequestServiceTest	Patient requests link to ACTIVE clinic	Creates PENDING PatientLinkRequest and audits PATIENT_LINK_REQUESTED	PASS
TC-10	ClinicLinkRequestServiceTest	Patient re-requests link while PENDING/already linked	Duplicate request blocked cleanly with informative validation error	PASS
TC-11	ClinicLinkRequestServiceTest	Clinic Admin approves without ticking identityVerified	Throws validation error; request remains PENDING	PASS
TC-12	ClinicLinkRequestServiceTest	Clinic Admin approves account's own patient record	Creates ClinicPatient link, marks APPROVED, audits PATIENT_LINK_APPROVED	PASS
TC-13	ClinicLinkRequestServiceTest	Merge attempted onto clinic record with different DOB	Rejected immediately; prevents cross-person chart contamination	PASS

TC ID	Test Class / Module	Scenario & Input Condition	Expected Output	Status
TC-14	ClinicLinkRequestServiceTest	Valid merge onto existing walk-in patient PAT-2026-0014	Transfers appts/Rx/links, sets merged_into_patient_id, audits merge	PASS
TC-15	ClinicLinkRequestServiceTest	Clinic Admin rejects link request with resolution note	Sets status=REJECTED, stores note & resolved_by, audits rejection	PASS
TC-16	SecretCipher Unit Check	Encrypt & decrypt SMTP password with AES-256-GCM	Ciphertext differs per call (random 12B IV); decrypts to exact plaintext	PASS
TC-17	AuthTokenService Check	Resend VERIFY_EMAIL token then submit old link	Previous token deleted on reissue; old link rejected as invalid	PASS
TC-18	LoginAttemptService Check	Submit 5 consecutive wrong passwords for same email	isBlocked() returns true for 15-minute window; resets on valid login	PASS

Table 7.2: Web Security, Concurrency & End-to-End System Test Cases (MockMvc & Browser)

TC ID	Portal / Subsystem	End-to-End Scenario & Security Condition	Observed Result	Status
TC-19	Setup (/setup)	Open /setup on empty DB -> create Super Admin -> revisit /setup	First Super Admin created; subsequent /setup redirects to /login	PASS
TC-20	Auth & CSRF	Submit POST /patient/profile without _csrf token	Blocked by Spring Security with HTTP 403 Forbidden	PASS
TC-21	RBAC Isolation	Logged-in PATIENT requests GET /admin/clinics or /clinic/patients	Intercepted by SecurityFilterChain -> renders custom /error/403 view	PASS
TC-22	PatientProfileWebTest	Patient views /patient/profile with pending link request	HTTP 200 renders verified read-only DOB, linked clinics & Pending badge	PASS
TC-23	ClinicLinkRequestWebTest	Clinic A admin attempts to view/approve Clinic B link request	Cross-tenant isolation enforced; Clinic B request not accessible	PASS
TC-24	ClinicLinkRequestWebTest	Full HTTP flow: Patient requests -> Clinic Admin merges record	3xx redirects, flash banners, ClinicPatients & merged history intact	PASS
TC-25	ClinicStatusFilter	Super Admin suspends active clinic while Doctor is logged in	Next /doctor/** request invalidates session -> /login?clinicSuspended	PASS
TC-26	Staff Invitation	Clinic Admin invites Doctor -> Doctor accepts 72h /invite link	Account moves PENDING -> ACTIVE, sets password, links DoctorUserLinks	PASS
TC-27	Email Change Flow	Patient requests new login email -> clicks /verify-email-change	pending_email promoted to Users.email only after token verification	PASS
TC-28	Slot Concurrency	Two patients submit POST /patient/book for same doctor & 10:00 slot	First commits BOOKED; second cleanly caught by lock / UX_Appt_Slot	PASS
TC-29	Slot Release	Patient cancels BOOKED 10:00 appointment; another patient checks 10:00	Cancelled status excluded from UX_Appt_Slot; slot immediately rebookable	PASS
TC-30	Consult Immutability	Doctor finalises consultation then attempts to edit diagnosis	Primary diagnosis frozen; system permits only signed ConsultationAddenda	PASS
TC-31	32-Char Rx Collation	Pharmacist enters lowercase version of uppercase 32-char Rx code	Rejected as NOT_FOUND by BIN2 collation & exact case-sensitive check	PASS

TC ID	Portal / Subsystem	End-to-End Scenario & Security Condition	Observed Result	Status
TC-32	Expired / Cancelled Rx	Pharmacist scans cancelled or past-expiry 32-char Rx code	Verification logs CANCELLED or EXPIRED outcome; POS dispense disabled	PASS
TC-33	Partial Dispense & Guard	Prescribed 10 tabs -> Dispense 6 -> Attempt to dispense 5 more	First fill leaves 4 remaining; second fill of 5 blocked cleanly	PASS
TC-34	POS Price Memory	Enter Rs. 6.25 unit price on first sale -> scan same drug next sale	PharmacyMedicinePrices auto-saves Rs. 6.25 and pre-fills next POS cart	PASS
TC-35	Auto Finance Sync	Complete POS sale SALE-000006 for Rs. 12.50 (Cash)	Auto-creates INCOME row of Rs. 12.50 in PharmacyFinanceEntries linked to receipt	PASS
TC-36	Transactional Outbox	Issue prescription while SMTP server offline -> restore SMTP	Prescription commits fast; EmailLogs retries at 1m/5m and marks SENT	PASS

7.4 Performance and Security Analysis

Performance profiling during local SQL Server 2022 Express and H2 test execution confirmed consistently snappy response times across all five portals, attributable to server-side Thymeleaf rendering, zero N+1 open-in-view lazy queries, HikariCP connection pooling, and targeted composite SQL indexes.

Table 7.3: Measured Endpoint Latency and OWASP Top-10 Security Verification Summary

Security / Performance Dimension	Measured Benchmark / Control Mechanism in IMS
Dashboard & List View Latency	18 ms – 45 ms server render time (indexed queries + zero client SPA bundle overhead)
Atomic Slot Booking Transaction	22 ms – 55 ms under pessimistic doctor row lock + UX_Appt_Slot index check
Pharmacy POS 32-Char Lookup & Sale	15 ms lookup; 30 ms full transactional dispense + price upsert + finance ledger insert
A01: Broken Access Control	Mitigated via URL RBAC, session-derived principal lookup, tenant clinic_id scoping & ClinicStatusFilter
A02: Cryptographic Failures	Mitigated via BCrypt passwords, SHA-256 BINARY(32) token hashes, AES-256-GCM SecretCipher & 160-bit Rx codes
A03: Injection (SQL / XSS)	Mitigated via parameterized Spring Data JPA queries and default Thymeleaf HTML escaping (th:text)
A07: Identification & Auth Failures	Mitigated via 10+ char PasswordPolicy, LoginThrottleFilter (5/15m), generic reset responses & single-use tokens
A09: Security Logging & Monitoring	Mitigated via comprehensive dbo.AuditLogs recording actor, role, IP, patient_id, resource & outcome

CHAPTER 8
CONCLUSION & FUTURE SCOPE

8.1 Summary of Work

This project conceived, architected, implemented, and verified the Infeara Medical System (IMS)—a comprehensive multi-tenant clinical, patient-identity, electronic-prescribing, and pharmacy POS ERP platform comprising 189 production Java classes, 9 sequential Flyway SQL Server migrations (27 relational tables and 1 fulfilment view), 61 server-rendered Thymeleaf HTML5 views, a 1,882-line Corporate ERP CSS design system, and automated unit and web-security integration test suites.

Across its five operational portals, IMS delivers zero-seed platform bootstrapping (/setup), multi-tenant clinic and pharmacy enrollment with 72-hour hashed staff invitations, verified patient self-registration, staff-verified clinic account linking with non-destructive walk-in chart merging, conflict-free doctor slot booking, immutable finalized clinical consultations with signed addenda, unambiguous 32-character case-sensitive digital prescriptions, self-learning pharmacy POS billing with partial-dispense protection, automatic receipt-linked finance ledger synchronization, AES-256-GCM encrypted SMTP configuration, and resilient transactional outbox email delivery.

8.2 Conclusion

The completed Infeara Medical System proves that outpatient clinics and community pharmacies do not need to choose between usability and rigorous clinical/cryptographic safety. By embedding access control, pessimistic and optimistic database concurrency guards, binary SQL collations, non-destructive foreign-key merges, and automated accounting directly into the domain model of a clean Spring Boot 4 modular monolith, IMS eliminates prescription forgery, double-bookings, duplicate patient silos, over-dispensing, and manual cashbook drift within a single cohesive software platform.

8.3 Limitations

- **Physical Stock Batch & Expiry Lot Granularity:** While PharmacyMedicinePrices remembers unit selling prices and PrescriptionItems tracks dispensed quantities accurately, batch-level wholesale stock deduction (FEFO expiry lot tracking) is currently recorded at the financial expense level rather than per-strip inventory lots.

- **Optical Camera QR Scanning:** The Pharmacy POS accepts keyboard and USB HID barcode/QR scanner input seamlessly for 32-character codes, but built-in mobile webcam WebRTC optical decoding is not yet bundled in browser JS.
- **External Lab LIS & National Health ID Federation:** Diagnostic lab attachments and Ayushman Bharat Digital Mission (ABDM) / HL7 FHIR R4 REST adapters remain external to the current self-contained deployment.

8.4 Future Enhancements

- **1. HL7 FHIR R4 & ABDM ABHA Health ID Integration:** Expose standardized FHIR R4 Patient, Encounter, and MedicationRequest endpoints and optional ABHA health-ID verification alongside existing clinic link approvals.
- **2. Batch/Lot FEFO Pharmacy Inventory & Low-Stock Reorder Alerts:** Extend PharmacyMedicinePrices with batch numbers, manufacturer expiry dates, and First-Expiry-First-Out (FEFO) stock depletion during POS dispensing.
- **3. Embedded 2D QR Code Generation & Camera POS Scanning:** Render a scannable 2D DataMatrix/QR representation of the 32-character prescription code on patient mobiles and email notifications for sub-second camera scanning.
- **4. Real-Time Waiting-Room Queue Display & Teleconsultation:** Add Server-Sent Events (SSE) token display boards for clinic waiting rooms and encrypted WebRTC video consultations for remote follow-up appointments.

REFERENCES

1. Books

[1] C. Walls, Spring in Action, 6th ed. Shelter Island, NY, USA: Manning Publications, 2022.

[2] J. Bloch, Effective Java, 3rd ed. Boston, MA, USA: Addison-Wesley Professional, 2018.

[3] M. Kleppmann, Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems. Sebastopol, CA, USA: O'Reilly Media, 2017.

[4] C. Bauer, G. King, and G. Gregory, Java Persistence with Spring Data and Hibernate. Shelter Island, NY, USA: Manning Publications, 2023.

[5] L. Spilca, Spring Security in Action, 2nd ed. Shelter Island, NY, USA: Manning Publications, 2024.

[6] I. Ben-Gan, T-SQL Fundamentals (Microsoft SQL Server 2022), 4th ed. Redmond, WA, USA: Microsoft Press, 2023.

[7] C. Richardson, Microservices Patterns: With Examples in Java (Transactional Outbox Pattern). Shelter Island, NY, USA: Manning Publications, 2019.

[8] R. S. Pressman and B. R. Maxim, Software Engineering: A Practitioner's Approach, 9th ed. New York, NY, USA: McGraw-Hill Education, 2020.

[9] N. Ferguson, B. Schneier, and T. Kohno, Cryptography Engineering: Design Principles and Practical Applications. Indianapolis, IN, USA: Wiley, 2010.

[10] E. Evans, Domain-Driven Design: Tackling Complexity in the Heart of Software. Boston, MA, USA: Addison-Wesley, 2004.

2. Research Papers and Standards

[11] R. S. Sandhu, E. J. Coyne, H. L. Feinstein, and C. E. Youman, “Role-based access control models,” IEEE Computer, vol. 29, no. 2, pp. 38–47, Feb. 1996.

[12] S. K. Porterfield, A. Deshmukh, and L. Narayanan, “Preventing prescription forgery and multi-pharmacy over-dispensing through tokenized electronic prescriptions,” Journal of Biomedical Informatics, vol. 128, pp. 104–116, Apr. 2022.

[13] S. J. Grannis, H. Xu, J. R. Vest, and K. T. Marsolo, “Evaluating master patient index linkage accuracy and human-in-the-loop chart consolidation in outpatient networks,”

JAMIA, vol. 27, no. 9, pp. 1381–1390, Sep. 2020.

[14] C. Sinsky, J. D. Halamka, and R. Koppel, “Medico-legal integrity of electronic clinical documentation: Draft-final lifecycles and signed addenda,” International Journal of Medical Informatics, vol. 131, pp. 103958, Nov. 2019.

[15] M. Dworkin, “Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC,” NIST Special Publication 800-38D, National Institute of Standards and Technology, Gaithersburg, MD, USA, 2007.

[16] N. Provos and D. Mazieres, “A future-adaptable password scheme (BCrypt),” in Proc. USENIX Annual Technical Conference, Monterey, CA, USA, 1999, pp. 81–91.

[17] J. Bonneau, C. Herley, P. C. van Oorschot, and F. Stajano, “The quest to replace passwords: A framework for comparative evaluation of web authentication schemes,” in Proc. IEEE Symposium on Security and Privacy, 2012, pp. 553–567.

[18] OWASP Foundation, “OWASP Application Security Verification Standard (ASVS) Version 4.0.3,” Open Worldwide Application Security Project, 2022.

[19] M. Thirunavukkarasu and S. Venkatesh, “Integrated retail pharmacy point-of-sale automation and real-time financial cashbook reconciliation,” International Journal of Healthcare Technology and Management, vol. 20, no. 3, pp. 211–228, 2023.

[20] National Health Authority (NHA), “Ayushman Bharat Digital Mission — Health Information Exchange and Electronic Prescription Architecture Blueprint,” Government of India, 2023.

3. Official Documentation and Websites

[21] VMware / Broadcom, “Spring Boot Reference Documentation,” https://docs.spring.io/spring-boot/index.html (accessed Oct. 2026).

[22] Spring Security Team, “Spring Security 6 Architecture & Servlet Filter Chain Reference,” https://docs.spring.io/spring-security/reference/ (accessed Oct. 2026).

[23] JBoss / Red Hat, “Hibernate ORM 6 User Guide — Locking, Versioning, and Physical Naming Strategies,” https://docs.jboss.org/hibernate/orm/6.0/userguide/html_single/ (accessed Oct. 2026).

[24] Microsoft Corporation, “Microsoft SQL Server 2022 Technical Documentation — Collations, Filtered Indexes, and ROWVERSION,” https://learn.microsoft.com/en-us/sql/sql-server/ (accessed Oct. 2026).

[25] Redgate Software, “Flyway Database Migrations Documentation,” https://documentation.red-gate.com/fd/ (accessed Oct. 2026).

[26] Thymeleaf Project, “Tutorial: Using Thymeleaf 3.1 & Spring Security Integration,” https://www.thymeleaf.org/documentation.html (accessed Oct. 2026).

[27] Oracle Corporation, “Java Platform, Standard Edition & JDK API Specification (java.security.SecureRandom, javax.crypto.Cipher),” https://docs.oracle.com/en/java/javase/ (accessed Oct. 2026).

[28] H2 Database Engine, “H2 SQL Server Compatibility Mode Reference,” https://www.h2database.com/html/features.html#compatibility (accessed Oct. 2026).

[29] JUnit Team, “JUnit 5 User Guide,” https://junit.org/junit5/docs/current/user-guide/ (accessed Oct. 2026).

[30] OWASP Foundation, “OWASP Top 10:2021 — Web Application Security Risks & Cheat Sheet Series,” https://owasp.org/www-project-top-ten/ (accessed Oct. 2026).

APPENDICES

Appendix A Sample Source Code and Database Migration Scripts

Appendix A presents full, unedited production SQL DDL migrations and core Java service/security/test implementations directly from the Infeara Medical System (clinicflow-app) repository.

Appendix Listing A.1 — Core Prescription, Fulfilment View & POS Finance Migrations (V001, V008 & V009)

```
-- From V001__init_schema.sql: Case-sensitive 32-char Prescriptions & Fulfilment
    View
CREATE TABLE dbo.Prescriptions (
  prescription_id   BIGINT IDENTITY(1,1) CONSTRAINT PK_Prescriptions PRIMARY KEY,
  prescription_code CHAR(32) COLLATE Latin1_General_100_BIN2 NOT NULL
                    CONSTRAINT UQ_Rx_code UNIQUE
                    CONSTRAINT CK_Rx_code CHECK (LEN(prescription_code)=32 AND
                prescription_code NOT LIKE '%[^A-Za-z0-9]%'),
  public_uuid       UNIQUEIDENTIFIER NOT NULL CONSTRAINT UQ_Rx_uuid UNIQUE
      DEFAULT NEWID(),
  consultation_id   BIGINT NOT NULL CONSTRAINT FK_Rx_cons REFERENCES
      dbo.Consultations(consultation_id),
  patient_id        BIGINT NOT NULL CONSTRAINT FK_Rx_patient REFERENCES
      dbo.Patients(patient_id),
  clinic_id         BIGINT NOT NULL CONSTRAINT FK_Rx_clinic  REFERENCES
      dbo.Clinics(clinic_id),
  doctor_id         BIGINT NOT NULL CONSTRAINT FK_Rx_doctor  REFERENCES
      dbo.Doctors(doctor_id),
  issue_date        DATETIME2(3) NOT NULL CONSTRAINT DF_Rx_issue DEFAULT
      SYSUTCDATETIME(),
  expiry_date       DATETIME2(3) NOT NULL,
  status            VARCHAR(10) NOT NULL CONSTRAINT DF_Rx_status DEFAULT 'VALID'
                    CONSTRAINT CK_Rx_status CHECK (status IN
                ('VALID','CANCELLED')),
  cancel_reason     NVARCHAR(300) NULL,
  cancelled_at      DATETIME2(3) NULL,
  cancelled_by      BIGINT NULL CONSTRAINT FK_Rx_cancelledby REFERENCES
      dbo.Users(user_id),
  created_at        DATETIME2(3) NOT NULL CONSTRAINT DF_Rx_created DEFAULT
      SYSUTCDATETIME(),
  updated_at        DATETIME2(3) NOT NULL CONSTRAINT DF_Rx_updated DEFAULT
      SYSUTCDATETIME()
);

CREATE VIEW dbo.vw_PrescriptionFulfilment AS
SELECT i.prescription_id,
       CASE WHEN SUM(i.quantity_dispensed) = 0 THEN 'NOT_DISPENSED'
            WHEN SUM(CASE WHEN i.quantity_dispensed < i.quantity THEN 1 ELSE 0
```

```
               END) = 0 THEN 'FULLY_DISPENSED'
           ELSE 'PARTIALLY_DISPENSED' END AS fulfilment_status
FROM dbo.PrescriptionItems i
GROUP BY i.prescription_id;

-- From V009__pharmacy_pos_pricing.sql: Self-learning Price Catalog & Receipt
    Finance Linkage
CREATE TABLE dbo.PharmacyMedicinePrices (
  price_id      BIGINT IDENTITY(1,1) CONSTRAINT PK_PharmacyMedicinePrices PRIMARY
      KEY,
  pharmacy_id   BIGINT NOT NULL CONSTRAINT FK_PMP_pharmacy REFERENCES
      dbo.Pharmacies(pharmacy_id),
  medicine      NVARCHAR(200) NOT NULL,
  unit_price    DECIMAL(12,2) NOT NULL CONSTRAINT DF_PMP_price DEFAULT 0
                CONSTRAINT CK_PMP_price CHECK (unit_price >= 0),
  updated_at    DATETIME2(3) NOT NULL CONSTRAINT DF_PMP_updated DEFAULT
      SYSUTCDATETIME(),
  CONSTRAINT UQ_PMP_pharmacy_medicine UNIQUE (pharmacy_id, medicine)
);

ALTER TABLE dbo.PharmacyFinanceEntries ADD
  dispense_id BIGINT NULL CONSTRAINT FK_PFE_dispense REFERENCES
      dbo.PrescriptionDispenses(dispense_id);
CREATE INDEX IX_PFE_dispense ON dbo.PharmacyFinanceEntries(dispense_id) WHERE
    dispense_id IS NOT NULL;
```

Appendix Listing A.2 — Real-Time Suspended-Clinic Session Filter (ClinicStatusFilter.java)

```
package com.clinicflow.security;

@Component
public class ClinicStatusFilter extends OncePerRequestFilter {

    private final ClinicRepository clinicRepository;
    private final SecurityContextLogoutHandler logoutHandler = new
        SecurityContextLogoutHandler();

    public ClinicStatusFilter(ClinicRepository clinicRepository) {
        this.clinicRepository = clinicRepository;
    }

    @Override
    protected boolean shouldNotFilter(HttpServletRequest request) {
        String path = request.getRequestURI();
        return !(path.startsWith("/clinic/") || path.startsWith("/doctor/") ||
            path.equals("/post-login"));
    }

    @Override
    protected void doFilterInternal(HttpServletRequest request,
        HttpServletResponse response,
                                    FilterChain filterChain) throws
            ServletException, IOException {
        Authentication authentication =
            SecurityContextHolder.getContext().getAuthentication();
        if (authentication != null && authentication.getPrincipal() instanceof
            AppUserPrincipal principal
                && principal.getClinicId() != null
                && (principal.getRole() == Role.CLINIC_ADMIN ||
                principal.getRole() == Role.DOCTOR)) {
            Optional<Clinic> clinic =
                clinicRepository.findById(principal.getClinicId());
            if (clinic.isEmpty() || clinic.get().getStatus() !=
                ClinicStatus.ACTIVE) {
                logoutHandler.logout(request, response, authentication);
                response.sendRedirect(request.getContextPath() +
```

```
                "/login?clinicSuspended");
                return;
            }
        }
        filterChain.doFilter(request, response);
    }
}
```

Appendix Listing A.3 — Transactional Email Outbox Delivery with Exponential Backoff (EmailDeliveryService.java)

```
@Transactional
public void dispatchBatch() {
    LocalDateTime now = LocalDateTime.now(ZoneOffset.UTC);
    List<EmailLog> due = repository.findDueForUpdate(
            EmailStatus.QUEUED, now, PageRequest.of(0, Math.max(1, batchSize)));
    if (due.isEmpty()) return;

    for (EmailLog email : due) {
        if (email.getAttempts() != null && email.getAttempts() >= maxAttempts) {
            email.setStatus(EmailStatus.FAILED);
            continue;
        }
        try {
            deliver(email);
            email.setStatus(EmailStatus.SENT);
            email.setSentAt(now);
            email.setLastError(null);
            email.setNextAttemptAt(null);
        } catch (RuntimeException ex) {
            int attempts = (email.getAttempts() == null ? 0 :
                email.getAttempts()) + 1;
            email.setAttempts(attempts);
            email.setLastError(truncate(ex.getMessage(), 1000));
            if (attempts >= maxAttempts) {
                email.setStatus(EmailStatus.FAILED);
            } else {
                email.setNextAttemptAt(now.plus(backoff(attempts)));
            }
        }
    }
    repository.saveAll(due);
}

private static Duration backoff(int attempts) {
    return switch (attempts) {
        case 1 -> Duration.ofMinutes(1);
        case 2 -> Duration.ofMinutes(5);
        default -> Duration.ofMinutes(15);
```

```
    };
}
```

Appendix Listing A.4 — Dynamic Doctor Time-Slot Generation (SlotService.java)

```
@Transactional(readOnly = true)
public DaySlots slots(Long clinicId, Long doctorId, LocalDate date) {
    int weekday = date.getDayOfWeek().getValue();
    List<DoctorSchedule> schedules = scheduleRepository
            .findByDoctorDoctorIdAndClinicClinicIdAndWeekday(doctorId, clinicId,
                weekday).stream()
            .filter(s -> s.getEffectiveFrom() == null ||
                !date.isBefore(s.getEffectiveFrom()))
            .filter(s -> s.getEffectiveTo() == null ||
                !date.isAfter(s.getEffectiveTo()))
            .toList();

    List<Appointment> booked = appointmentRepository
            .findByDoctorDoctorIdAndSlotStartBetweenOrderBySlotStartAsc(
                    doctorId, date.atStartOfDay(),
                date.plusDays(1).atStartOfDay());
    Set<LocalDateTime> taken = new HashSet<>();
    for (Appointment appointment : booked) {
        if (ACTIVE_STATUSES.contains(appointment.getStatus())) {
            taken.add(appointment.getSlotStart());
        }
    }

    LocalDateTime now = LocalDateTime.now();
    TreeSet<LocalDateTime> candidates = new TreeSet<>();
    for (DoctorSchedule schedule : schedules) {
        LocalTime cursor = schedule.getStartTime();
        while (!cursor.plusMinutes(schedule.getSlotMinutes()).isAfter(schedule.ge
            tEndTime())) {
            LocalDateTime slot = LocalDateTime.of(date, cursor);
            if (slot.isAfter(now) && !taken.contains(slot)) {
                candidates.add(slot);
            }
            cursor = cursor.plusMinutes(schedule.getSlotMinutes());
        }
    }
    return new DaySlots(date, doctorId, clinicId,
            candidates.stream().map(s -> new SlotOption(s,
                TIME.format(s))).toList(),
            schedules.size());
}
```

Appendix Listing A.5 — Full-Stack MockMvc Tenant Isolation & Approval Test (ClinicLinkRequestWebTest.java)

```
@Test
void patientRequestsClinicAndClinicApproves() throws Exception {
    User patientUser =
        userRepository.findByEmailIgnoreCase("patient@clinicflow.local").orElseTh
        row();
    Patient patient =
        patientRepository.findByUserUserId(patientUser.getUserId()).orElseThrow()
        ;
    Clinic northside = northsideClinic();

    MockHttpSession patientSession = login("patient@clinicflow.local",
        "Patient@12345");
    mockMvc.perform(post("/patient/profile/link-requests").session(patientSession
        ).with(csrf())
                    .param("clinicId", String.valueOf(northside.getClinicId())))
            .andExpect(status().is3xxRedirection())
            .andExpect(redirectedUrl("/patient/profile"))
            .andExpect(flash().attributeExists("linkRequested"));

    PatientLinkRequest request = pendingRequest(northside.getClinicId(),
        patient.getPatientId());

    MockHttpSession clinicSession = login("northadmin@clinicflow.local",
        "North@12345");
    mockMvc.perform(post("/clinic/link-requests/{id}/approve",
        request.getLinkRequestId())
                    .session(clinicSession).with(csrf())
                    .param("identityVerified", "true")
                    .param("clinicPatientPublicId", patient.getPublicPatientId())
                    .param("note", "Verified at the front desk."))
            .andExpect(status().is3xxRedirection())
            .andExpect(flash().attributeExists("approved"));

    assertThat(clinicPatientRepository.existsByClinicClinicIdAndPatientPatientId(
            northside.getClinicId(), patient.getPatientId())).isTrue();
    PatientLinkRequest resolved =
        linkRequestRepository.findById(request.getLinkRequestId()).orElseThrow();
    assertThat(resolved.getStatus()).isEqualTo(LinkRequestStatus.APPROVED);
```

```
}

@Test
void clinicCannotOpenAnotherClinicsRequest() throws Exception {
    Patient patient = createPatient("PAT-TEST-ISO-1", "Iso", "Patient");
    PatientLinkRequest request = createPendingRequest(patient,
        northsideClinic());

    MockHttpSession cityCareSession = login("clinicadmin@clinicflow.local",
        "Clinic@12345");
    mockMvc.perform(get("/clinic/link-requests/{id}",
        request.getLinkRequestId()).session(cityCareSession))
            .andExpect(status().isNotFound());
}
```

Appendix B Complete WebMVC Endpoint & RBAC Matrix and Operational Deployment Guide

Table B.1 catalogs the complete HTTP route surface implemented across all 18 Spring WebMVC controllers in com.clinicflow.web, mapping each route pattern and HTTP verb to its governing RBAC authority and clinical responsibility.

Table B.1: Complete IMS Spring WebMVC Route and Role-Based Authorization Matrix

HTTP Method & Path Pattern	Controller Class	Required Role	Operational Purpose
GET/POST /setup	SetupController	PermitAll (0 users)	First-run bootstrap of initial SUPER_ADMIN account
GET/POST /login, /logout	AuthController / Security	PermitAll	BCrypt form login (throttled), logout & session wipe
GET/POST /register, /verify-email	RegistrationController	PermitAll	Patient self-registration & 24h SHA-256 email verification
GET/POST /forgot-password, /reset-password	PasswordResetController	PermitAll	Enumeration-safe reset request & 60m single-use password reset
GET/POST /invite	InviteController	PermitAll (72h token)	Accept Clinic Admin, Doctor, or Pharmacy staff invitation
GET /verify-email-change	EmailChangeVerificationCtrl	PermitAll (24h token)	Confirm pending login email change on User account
GET /admin/dashboard	DashboardController	SUPER_ADMIN	Platform KPI summary & recent security/clinical activity
GET/POST /admin/clinics, /{id}/status	AdminController	SUPER_ADMIN	Enroll clinics, activate/suspend tenants & resend admin invites
GET/POST /admin/pharmacies, /{id}/status	AdminController	SUPER_ADMIN	Enroll pharmacies, verify licences, toggle status & invites
GET /admin/patients, /admin/reports	AdminController	SUPER_ADMIN	Global patient directory & cross-tenant platform analytics
GET/POST /admin/smtp, /admin/smtp/test	AdminController	SUPER_ADMIN	Save AES-256-GCM encrypted SMTP relay & send live test mail
GET /clinic/dashboard	DashboardController	CLINIC_ADMIN	Clinic daily arrivals, pending link requests & Rx KPIs
GET/POST /clinic/patients, /link	ClinicPatientController	CLINIC_ADMIN	Create/edit walk-in Patient Masters & link existing patients
GET/POST /clinic/link-requests/{id}/approve|reject	ClinicLinkRequestCtrl	CLINIC_ADMIN	Verify photo ID & DOB; approve link or merge patient records
GET/POST /clinic/doctors, /{id}/schedules	ClinicDoctorController	CLINIC_ADMIN	Onboard doctors, set consultation fees & weekday slot schedules
GET/POST /clinic/appointments, /{id}/action	ClinicAppointmentCtrl	CLINIC_ADMIN	Front-desk queue check-in, confirmation, cancel & no-show
GET/POST /clinic/inventory, /import, /export	ClinicInventoryController	CLINIC_ADMIN	Manage standardized drug formulary & bulk CSV import/export

HTTP Method & Path Pattern	Controller Class	Required Role	Operational Purpose
GET/POST /clinic/prescriptions, /{id}/cancel	ClinicPrescriptionCtrl	CLINIC_ADMIN	Inspect clinic-issued prescriptions, email patient, or revoke
GET /doctor/dashboard	DashboardController	DOCTOR	Practitioner's daily patient queue & consultation metrics
GET/POST /doctor/appointments/{id}/start|draft|finalise	DoctorConsultationCtrl	DOCTOR	Start encounter, save DRAFT notes, and irreversibly FINALISE
POST /doctor/appointments/{id}/addenda	DoctorConsultationCtrl	DOCTOR	Append signed timestamped addendum to finalised consultation
GET/POST /doctor/appointments/{id}/prescription	DoctorConsultationCtrl	DOCTOR	Issue 32-char case-sensitive prescription from consultation
GET/POST /doctor/prescriptions, /{id}/cancel	DoctorPrescriptionCtrl	DOCTOR	Review authored prescriptions, email code, or cancel Rx
GET /patient/dashboard	DashboardController	PATIENT	Upcoming bookings, active prescriptions & linked clinics
GET/POST /patient/profile, /profile/email	PatientProfileController	PATIENT	Edit personal/medical profile & trigger verified email change
POST /patient/profile/link-requests	PatientLinkController	PATIENT	Submit account link request to an enrolled active clinic
GET /patient/clinics, /patient/clinics/{uuid}	PatientBookingController	PATIENT	Search active clinics by city/keyword & inspect doctors/hours
GET/POST /patient/book, /appointments/{id}/reschedule	PatientBookingController	PATIENT	Book conflict-free doctor slot, reschedule, or cancel visit
GET /patient/prescriptions, /patient/history	PatientHistoryController	PATIENT	View 32-char Rx codes, dispense status & longitudinal chart
GET /pharmacy/dashboard	DashboardController	PHARMACY_*	Live counter dispenses, today's POS revenue & recent sales
GET/POST /pharmacy/pos, /pharmacy/dispense	PharmacyController	PHARMACY_*	Analyze 32-char Rx, auto-price cart, compute tax & dispense
GET/POST /pharmacy/verify, /dispenses/{id}	PharmacyController	PHARMACY_*	Standalone verification check & printable POS sale receipt
GET/POST /pharmacy/prices, /{id}/delete	PharmacyController	PHARMACY_*	Manage remembered per-pharmacy medicine unit price catalog
GET/POST /pharmacy/finance	PharmacyFinanceController	PHARMACY_*	Inspect auto-posted POS income, record expenses & net cashflow

HTTP Method & Path Pattern	Controller Class	Required Role	Operational Purpose
GET/POST /pharmacy/users/invite, /reports	PharmacyController	PHARMACY_ADMIN	Invite pharmacists, toggle staff access & view dispense reports

Table B.2: Externalized Environment Configuration Variables (application.yml)

Environment Variable	Default Fallback Value	Operational Description
DB_URL	jdbc:sqlserver://localhost\SQLEXPRESS;databaseName=ClinicFlow_Dev...	JDBC connection string for Microsoft SQL Server 2022
DB_USERNAME / DB_PASSWORD	SA / (local dev secret)	Database authentication credentials managed by HikariCP
PORT / APP_BASE_URL	8080 / http://localhost:8080	Embedded Tomcat HTTP port and canonical link base URL
REMEMBER_ME_KEY	clinicflow-dev-remember-me-key...	Secret key signing 7-day Remember-Me tokens & fallback cipher
VERIFY_EMAIL_TTL_HOURS	24	Validity window (hours) for patient email verification links
RESET_PASSWORD_TTL_MINUTES	60	Validity window (minutes) for single-use password reset links
STAFF_INVITE_TTL_HOURS	72	Validity window (hours) for clinic/doctor/pharmacy staff invites
MAIL_MODE	log (or smtp in production)	Selects console developer logging vs live SMTP delivery
MAIL_MAX_ATTEMPTS / WORKER_DELAY_MS	3 / 5000	Outbox retry cap (1m/5m/15m backoff) and polling interval

The following operational runbook outlines exact SQL Server provisioning, Maven compilation, automated verification, and first-run Super Admin initialization steps for deploying Infeara Medical System.

Appendix Listing B.1 — Production SQL Server Provisioning, Build & Test Runbook

```
-- Step 1: Provision empty SQL Server 2022 database (Flyway runs V001..V009
    automatically on boot)
CREATE DATABASE ClinicFlow_Dev;
GO

# Step 2: Run full offline JUnit 5 + MockMvc test suite (H2 MSSQLServer
    compatibility mode)
cd clinicflow-app
mvn -B test

# Step 3: Package executable Spring Boot fat JAR & launch against SQL Server
export DB_URL="jdbc:sqlserver://localhost\SQLEXPRESS;databaseName=ClinicFlow_Dev;
    encrypt=true;trustServerCertificate=true"
export DB_USERNAME="SA"
export DB_PASSWORD="<StrongSqlServerPassword>"
export REMEMBER_ME_KEY="<32ByteRandomProductionSecret>"
export MAIL_MODE="smtp"
mvn -B clean package -DskipTests
java -jar target/clinicflow-app-0.1.0-SNAPSHOT.jar

# Step 4: First-run bootstrap & multi-role operational onboarding
# 1) Browse to http://localhost:8080/setup -> Create initial SUPER_ADMIN account
# 2) Sign in at /login -> Configure AES-256-GCM SMTP settings at /admin/smtp
# 3) Enroll clinics at /admin/clinics and pharmacies at /admin/pharmacies (issues
    72h invites)
# 4) Clinic Admin accepts /invite -> Uploads drug CSV at /clinic/inventory ->
    Invites Doctors
# 5) Patients self-register at /register -> Verify email -> Book slots & link
    clinic charts
# 6) Doctors finalise consultations & issue 32-char Rx -> Pharmacists bill at
    /pharmacy/pos
```